

Core Competencies

- ✓ Theory of Computation
- ✓ Data Structure & Algorithm
- ✓ Computer Networking
- ✓ Machine Learning
- ✓ Data Science

Prof. P. R. Navghare

M. Tech in Information Technology from National Institute of Technology, Raipur
B. E. in Computer Engineering from RTMNU, Nagpur

Research Domains

- ✓ Wireless Sensor Network
- ✓ Machine Learning



Society for Computer Technology and Research's
PUNE INSTITUTE OF COMPUTER TECHNOLOGY

Department of Computer Engineering

Discrete Mathematics

Course Code: 1303103

Prof. P. R. Navghare

Module -III

Module III :Graph Theory and Applications (10 Hrs)

- **Graph:** Graph Terminology and Special Types of Graphs,
- Representing Graphs and Graph Isomorphism,
- Euler and Hamilton Paths,
- the handshaking lemma,
- Single source shortest path-Dijkstra's Algorithm,
- Planar Graphs.
- **Trees:** Trees Introduction,
- properties of trees,
- Binary tree,
- Binary search tree, tree traversal, Expression tree,
- Huffman coding, Minimum Spanning Tree, Kruskal's and Prim's algorithms

Introduction

- Graphs are discrete structures consisting of vertices and edges that connect these vertices.

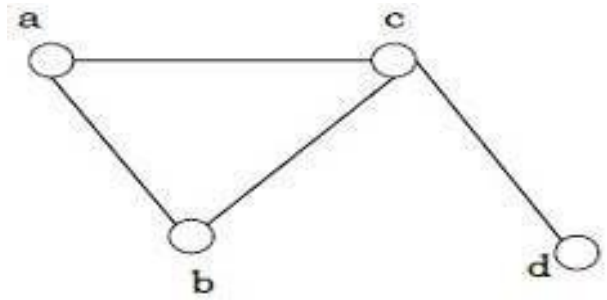
- There are different kinds of graphs, depending on whether edges have directions, whether multiple edges can connect the same pair of vertices, and whether loops are allowed.

- Definition: A Graph G = (V, E) consists of V, a nonempty set of vertices (or nodes) and E, a set of edges. Each edge has either one or two vertices associated with it, called its endpoints. An edge is said to connect its endpoints.

❖**Definition:** A Graph G = (V, E) consists of V, a nonempty set of vertices (or nodes) and E, a set of edges. Each edge has either one or two vertices associated with it, called its endpoints. An edge is said to connect its endpoints.

❖**Example 1:** Let us consider, a Graph is G = (V, E) where V = {a, b, c, d} and

E ={{a,b},{a,c},{b,c},{c,d}}



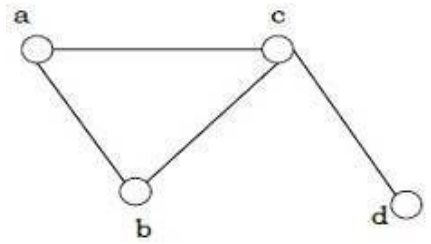
❖**Degree of a Vertex** - The degree of a vertex V of a graph G (denoted by deg (V)) is the number of edges incident with the vertex V.

❖**Even and Odd Vertex** - If the degree of a vertex is even, the vertex is called an even vertex and if the degree of a vertex is odd, the vertex is called an odd vertex.

❖**The Handshaking Lemma-** In a graph, the sum of all the degrees of vertices is equal to twice the number of edges.

$$2m = \sum_{v \in V} \deg(v).$$

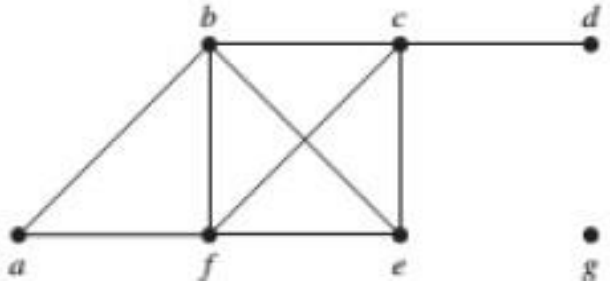
Vertex	Degree	Even / Odd
a	2	even
b	2	even
c	3	odd
d	1	odd



Graph Terminology

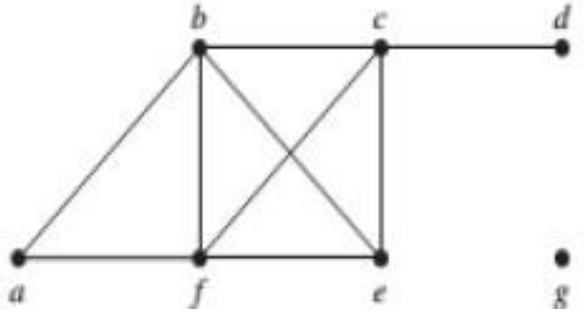
❖**Adjacent Vertex:** Two vertices u and v in an undirected graph G are called adjacent (or neighbors) in G if u and v are endpoints of an edge e of G. Such an edge e is called incident with the vertices u and v and e is said to connect u and v.

❖**Neighborhood:** The set of all neighbors of a vertex v of G = (V, E), denoted by N(v), is called the neighborhood of v. If A is a subset of V, we denote by N(A) the set of all vertices in G that are adjacent to at least one vertex in A.



❖**Isolated vertex :** A vertex of degree zero is called **isolated**. It follows that an isolated vertex is not adjacent to any vertex. In fig vertex g in graph G is isolated vertex.

❖**Pendent vertex :** A vertex is **pendant** if and only if it has degree one. Consequently, a pendant vertex is adjacent to exactly one other vertex. In fig vertex d in graph G is pendant vertex.



❖**Degree of a Vertex :** The degree of a vertex in an undirected graph is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex. The degree of the vertex v is denoted by deg(v).

❖**Initial and Terminal Vertex :** When (u,v) is an edge of the graph G with directed edges, u is said to be adjacent to v and v is said to be adjacent from u. The vertex u is called the initial vertex of (u,v), and v is called the terminal or end vertex of (u,v). The initial vertex and terminal vertex of a loop are the same.

❖**In-degree :** In a graph with directed edges the **in-degree of a vertex** v, denoted by $\deg^-(v)$, is the number of edges with v as their terminal vertex.

❖**Out-degree :** In a graph with directed edges the **out-degree of v**, denoted by $\deg^+(v)$, is the number of edges with v as their initial vertex.

❖**Imp Note:** that a loop at a vertex contributes 1 to both the in-degree and the out-degree of this vertex.).

Theorem

❖**Theorem 1: The Handshaking Theorem::**

$$2m = \sum_{v \in V} \deg(v).$$

➢ Let G = (V , E) be an undirected graph with m edges. Then

➢ (Note that this applies even if multiple edges and loops are present.)

❖**Theorem 2: An undirected graph has an even number of vertices of odd degree.**

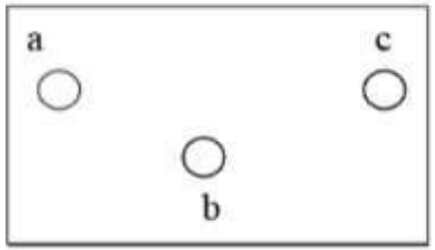
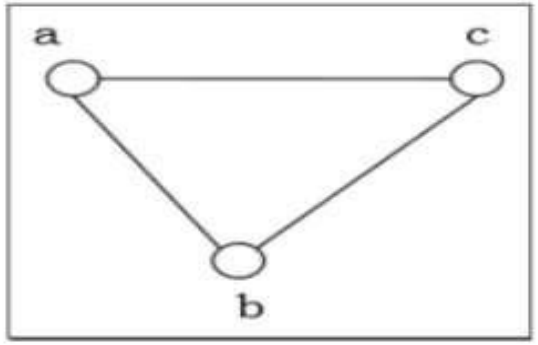
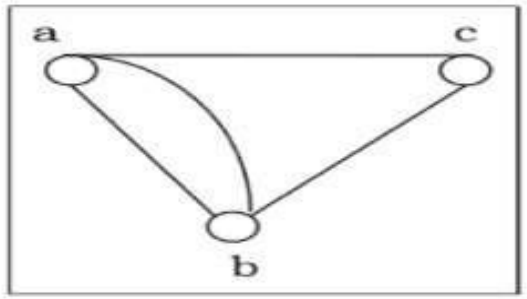
➢ Proof: Let V1 and V2 be the set of vertices of even degree and the set of vertices of odd degree, respectively, in an undirected graph G = (V,E) with m edges. Then

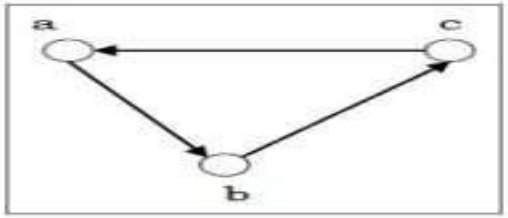
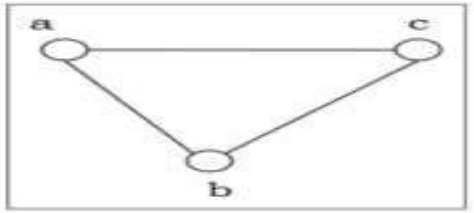
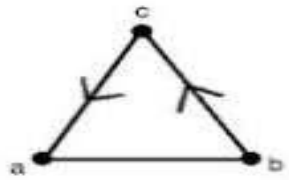
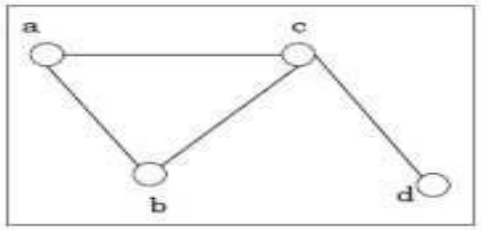
$$2m = \sum_{v \in V} \deg(v) = \sum_{v \in V_1} \deg(v) + \sum_{v \in V_2} \deg(v).$$

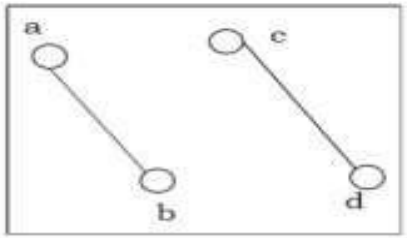
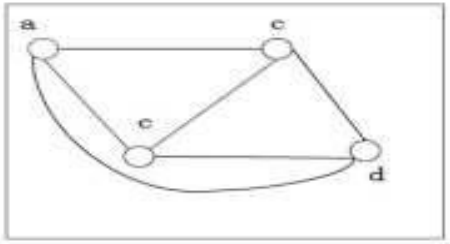
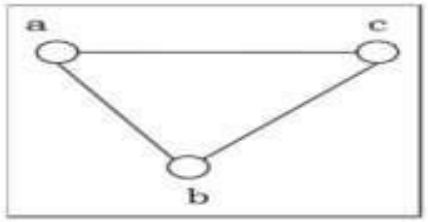
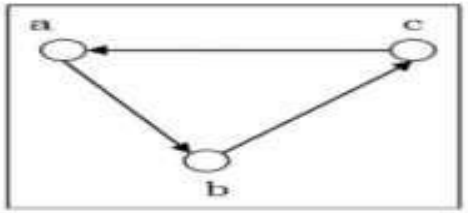
❖**Theorem 3: Let G = (V, E) be a graph with directed edges. Then**

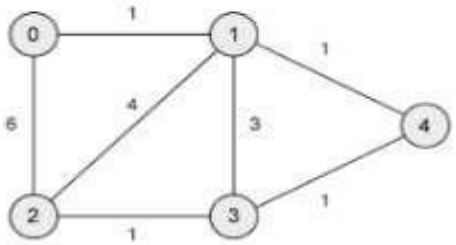
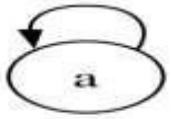
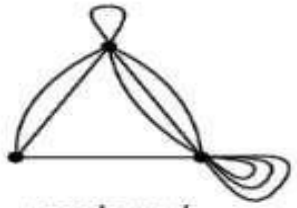
$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|.$$

Type of Graph

Null Graph	A null graph has no edges. The null graph of n vertices is denoted by Nn	
Simple Graph	A graph is called simple graph/strict graph if the graph is undirected and does not contain any loops or multiple edges. In a simple graph each edge connects two different vertices and no two edges connect the same pair of vertices.	
Multi-Graph	If in a graph multiple edges between the same set of vertices are allowed, it is called Multigraph. When m different edges connect the vertices u and v, we say that {u,v} is an edge of multiplicity m.	

Directed Graph	A graph G = (V, E) is called a directed graph if the edge set is made of ordered vertex pair.	
Undirected Graph	A graph G = (V, E) is called a undirected if the edge set is made of unordered vertex pair.	
Mixed Graph	A graph with both directed and undirected edges is called a mixed graph.	
Connected Graph	A graph is connected if any two vertices of the graph are connected by a path	

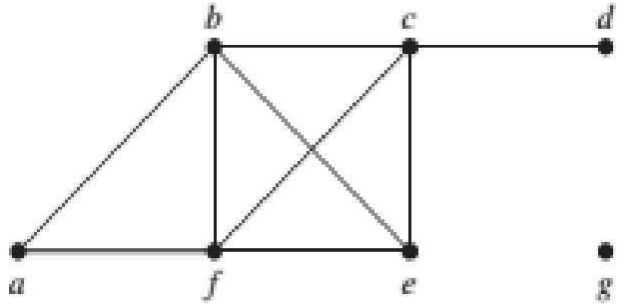
Disconnected Graph	A graph is disconnected if at least two vertices of the graph are not connected by a path. If a graph G is unconnected, then every maximal connected subgraph of G is called a connected component of the graph G.	
Regular Graph	A graph is regular if all the vertices of the graph have the same degree. In a regular graph G of degree r, the degree of each vertex of G is r.	
Complete Graph	A graph is called complete graph if every two vertices pair are joined by exactly one edge. The complete graph with n vertices is denoted by Kn	
Cycle Graph	If a graph consists of a single cycle, it is called cycle graph. The cycle graph with n vertices is denoted by Cn	

Infinite graph	A graph with an infinite vertex set or an infinite number of edges is called an infinite graph	
Finite graph	A graph with a finite vertex set and a finite edge set is called a finite graph	
Weighted Graph	A graph having a weight, or number, associated with each edge.	
Loop	An edge that connects a vertex to itself is called a loop.	
Pseudograph	A pseudograph may include loops, as well as multiple edges connecting the same pair of vertices.	 *pseudograph*

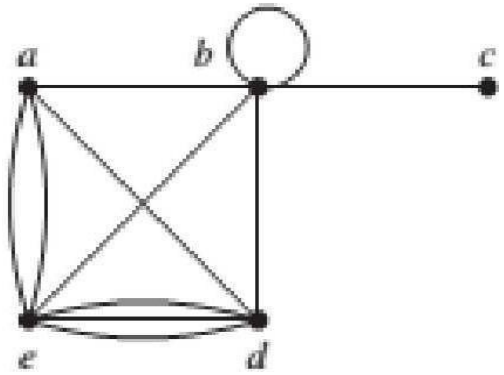
Example

❖**Example 1:** What are the degrees and what are the neighborhoods of the vertices in the graphs G and H displayed in figure below.

In Graph G,	In Graph H,
deg(a) = 2, deg(b) = deg(c) = deg(f) = 4, deg(d) = 1, deg(e) = 3, and deg(g) = 0.	deg(a) = 4, deg(b) = deg(e) = 6, deg(c) = 1, and deg(d) = 5.
The neighborhoods of these vertices are N(a)={b, f}, N(b)={a, c, e, f}, N(c)={b, d, e, f}, N(d)={c}, N(e)={b, c, f}, N(f)={a, b, c, e}, and N(g)=∅.	The neighborhoods of these vertices are N(a)={b, d, e}, N(b)={a, b, c, d, e}, N(c)={b}, N(d)={a, b, e}, and N(e)={a, b, d}.

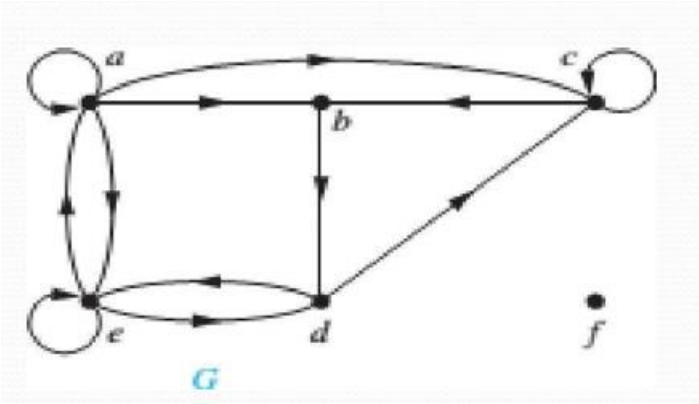


G



H

❖**Example 2:** Find the in-degree and out-degree of each vertex in the graph G with directed edges shown in figure below.

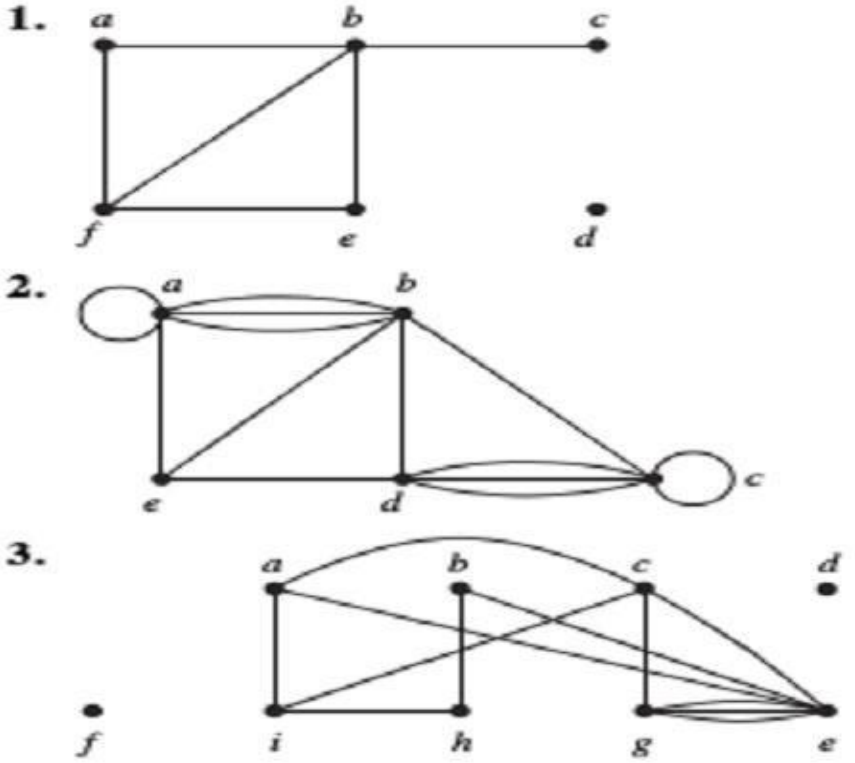


Solution:

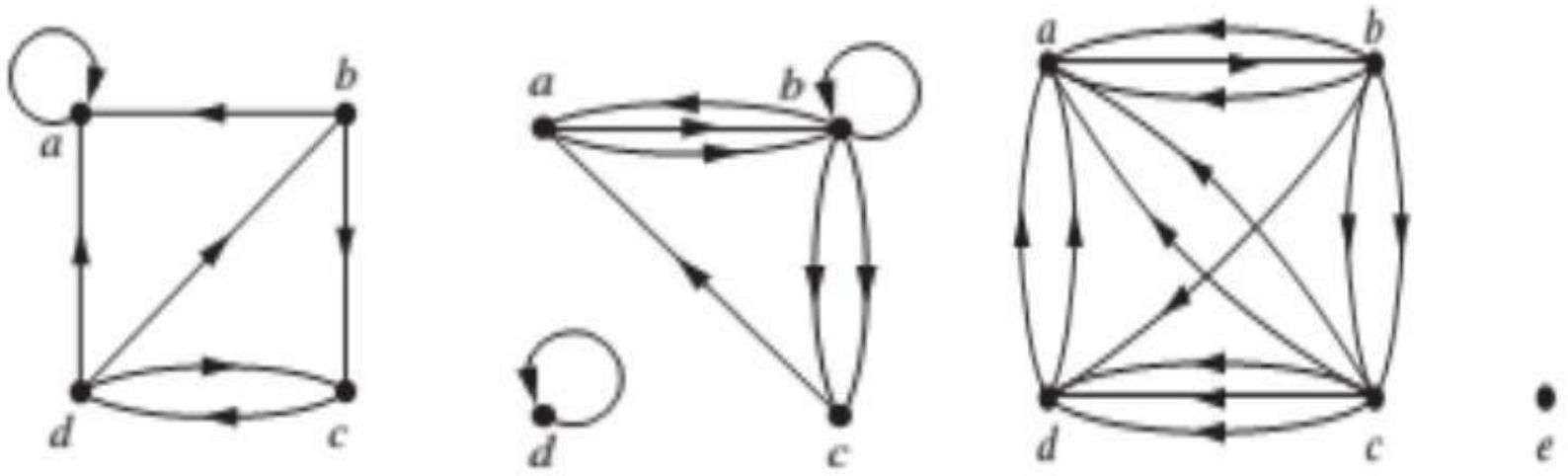
The in-degrees in G are ::: $\deg^-(a) = 2$, $\deg^-(b) = 2$,
$\deg^-(c) = 3$, $\deg^-(d) = 2$,
$\deg^-(e) = 3$, $\deg^-(f) = 0$.

The out-degrees in G are ::: $\deg^+(a) = 4$, $\deg^+(b) = 1$,
$\deg^+(c) = 2$, $\deg^+(d) = 2$,
$\deg^+(e) = 3$, $\deg^+(f) = 0$.

❖**Example 3:** Find the number of vertices, the number of edges, and the degree of each vertex in the given undirected graph. Identify all isolated and pendant vertices.



❖**Example 4:** Determine the number of vertices and edges and find the in- degree and out-degree of each vertex for the given directed multi-graph. Also determine sum of the in-degrees of the vertices and the sum of the out-degrees of the vertices directly.



❖**Example 5:** How many edges are there in a graph with 10 vertices each of degree six?

❖**Example 6:** How many vertices does a regular graph of degree four with 10 edges have?

According to handshaking lemma for Regular graph

❖In a k-regular graph with n vertices and $m$edges, $kn = 2m$

❖**Example 5:** How many edges are there in a graph with 10 vertices each of degree six?

Solution: Because the sum of the degrees of the vertices is **6 * 10 = 60**, it follows that **2m = 60** where m is the number of edges. Therefore, **m=30.**

❖**Example 6:** How many vertices does a regular graph of degree four with 10 edges have?

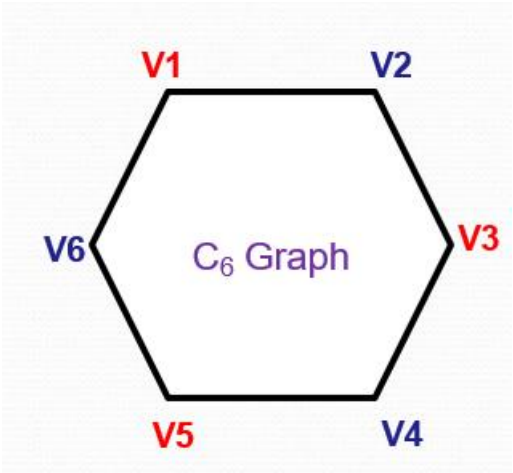
Solution: If a graph is regular of degree 4 and has n vertices, then by the handshaking theorem it has **4n/2 = 2n edges**. Since we are told that there are 10 edges, we just need to solve **2n = 10**. Thus the graph has 5 vertices. The complete graph K5 is one such graph.

Bipartite Graph

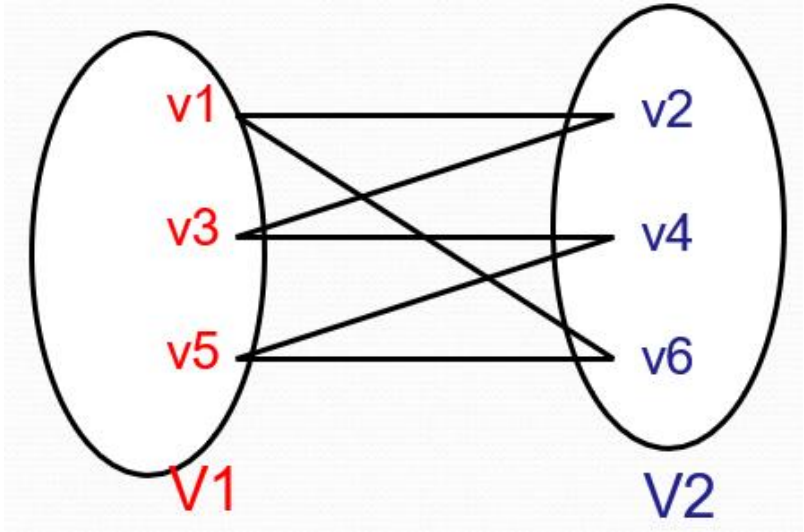
- A simple graph G is called **bipartite** if its vertex set V can be partitioned into two disjoint sets V1 and V2 such that every edge in the graph connects a vertex in V1 and a vertex in V2.
- (so that no edge in G connects either two vertices in V1 or two vertices in V2).
- When this condition holds, we call the pair (V1,V2) a bipartition of the vertex set V of G.

❖**C6 is bipartite**, as shown in below, because its vertex set can be partitioned into the two sets

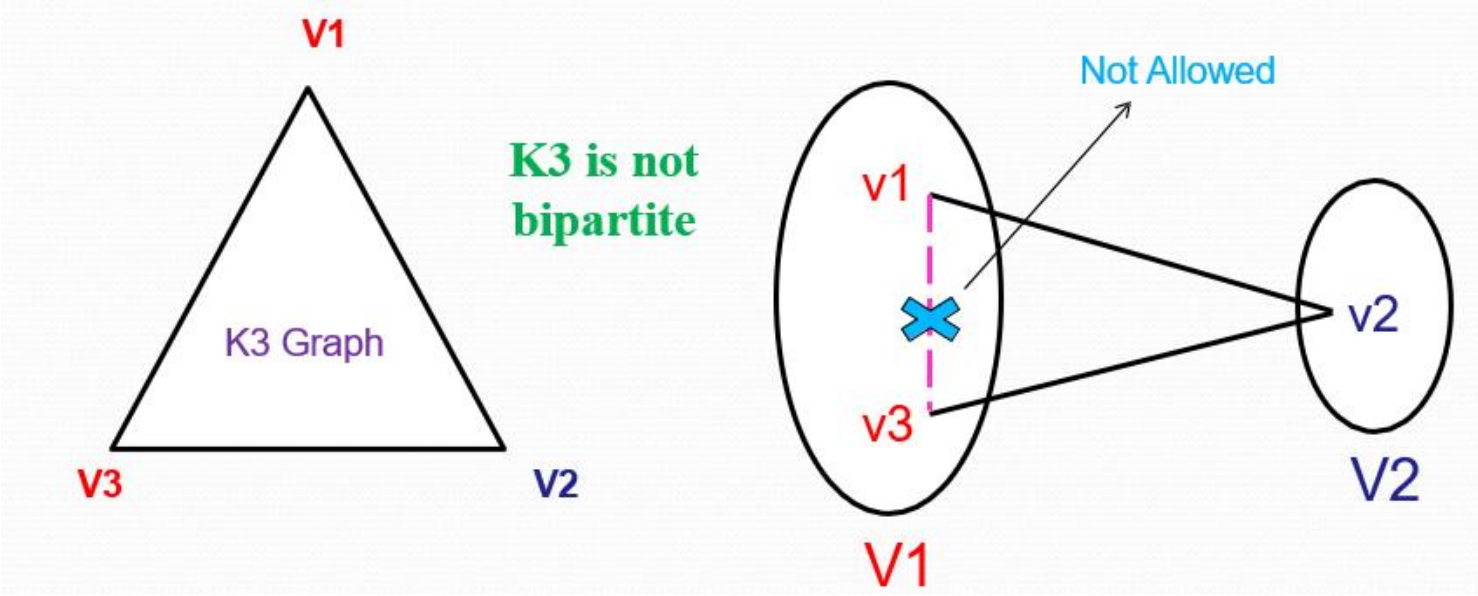
❖V1 = {v1, v3, v5} and V2 = {v2, v4, v6}, and every edge of C6 connects a vertex in V1 and a vertex in V2 .



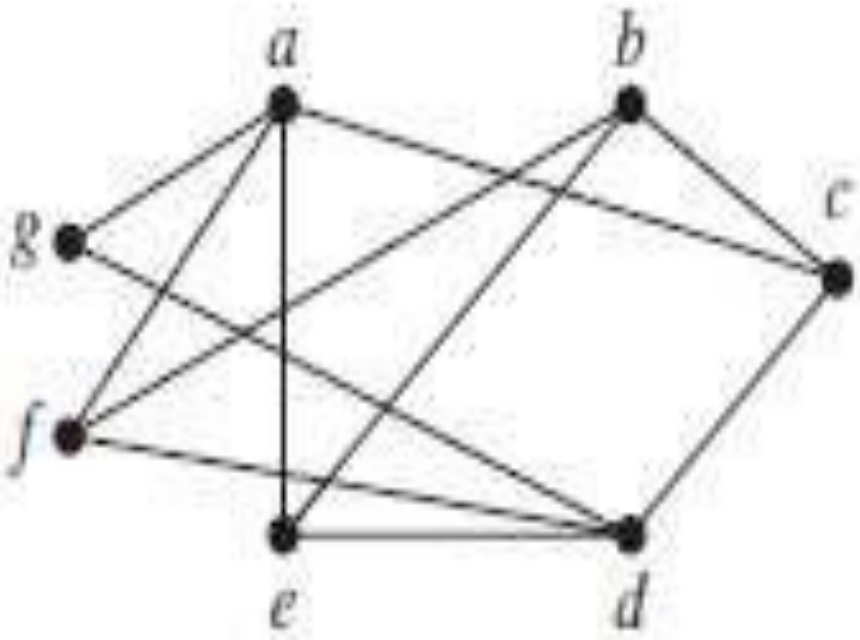
C_6 bipartite



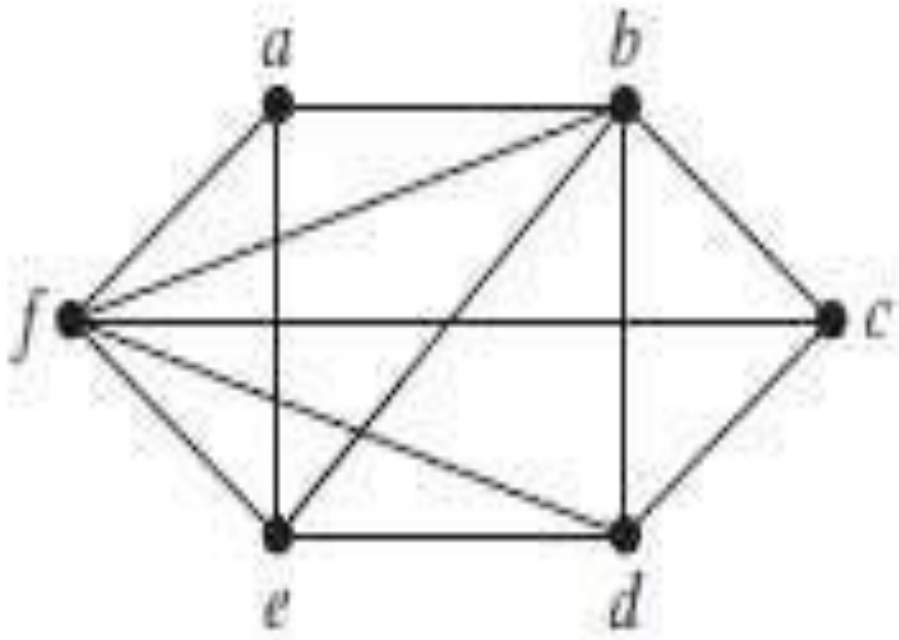
❖**K3 is not bipartite**. To verify this, note that if we divide the vertex set of K3 into two disjoint sets, one of the two sets must contain two vertices. If the graph were bipartite, these two vertices could not be connected by an edge, but in K3 each vertex is connected to every other vertex by an edge.



❖**Example 15:** Are the graphs G and H displayed in figure bipartite?

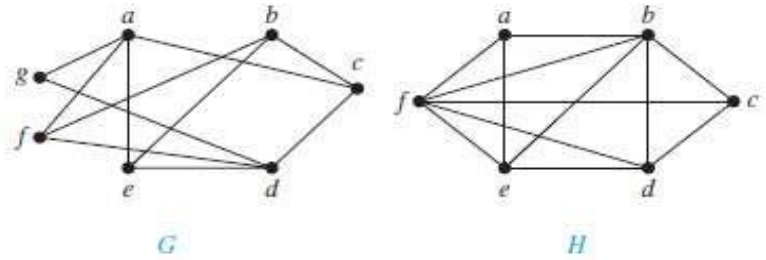


G



H

❖**Example 15:** Are the graphs G and H displayed in figure bipartite?



Solution: Graph **G is bipartite** because its vertex set is the union of two disjoint sets, {a, b, d} and {c, e, f, g}, and each edge connects a vertex in one of these subsets to a vertex in the other subset.

(Note that for G to be bipartite it is not necessary that every vertex in {a, b, d} be adjacent to every vertex in {c, e, f, g}. For instance, b and g are not adjacent.)

Graph H is not bipartite because its vertex set cannot be partitioned into two subsets so that edges do not connect two vertices from the same subset. (The reader should verify this by considering the vertices a, b, and f .)

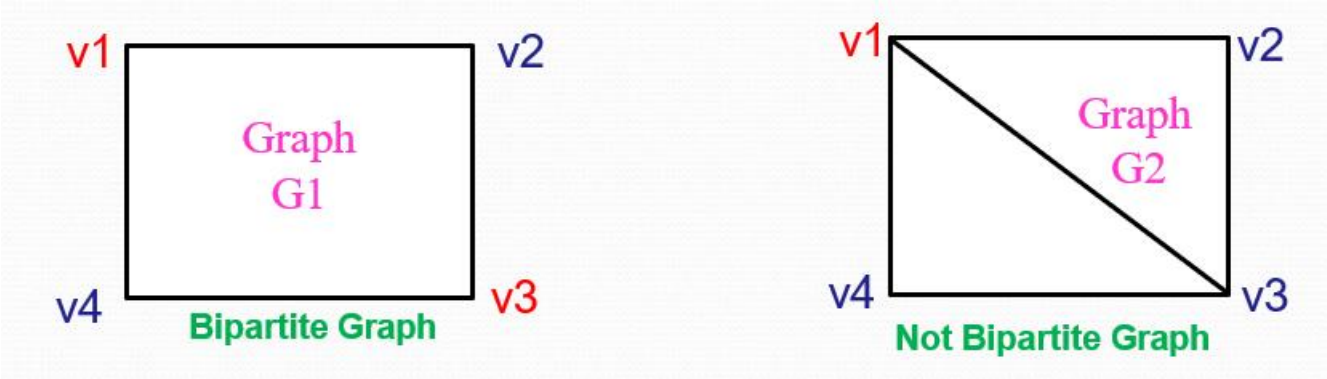
❖**THEOREM:** A simple graph is bipartite if and only if it is possible to assign one of two different colors to each vertex of the graph so that no two adjacent vertices are assigned the same color.

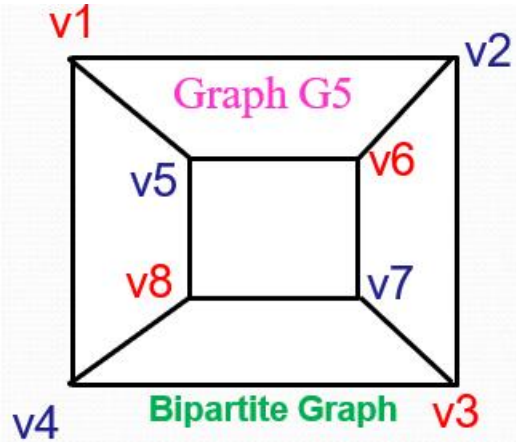
❖**Step:**

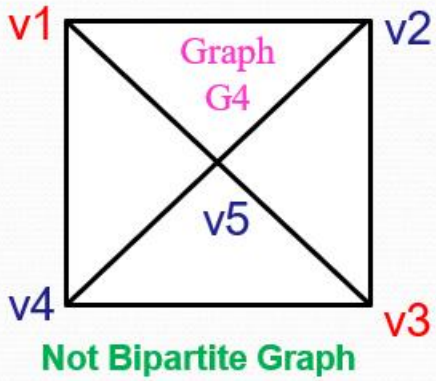
1. Assign RED color to the source vertex (putting into set V1).
2. Color all the neighbors with BLUE color (putting into set V2).
3. Color all neighbor's neighbor with RED color (putting into set V1).
4. This way, assign color to all vertices such that it satisfies all the constraints of m ways of coloring the problem where m=2.
5. While assigning colors, if we find a neighbor which is colored with same color as current vertex, then the graph cannot be colored with 2 vertices (or graph is not Bipartite).

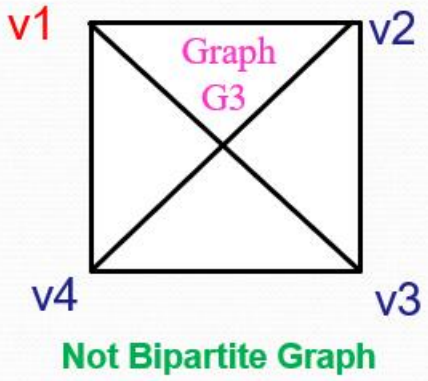
Example on Bipartite Graph

❖**Example 16:** Determine whether the graph is bipartite:

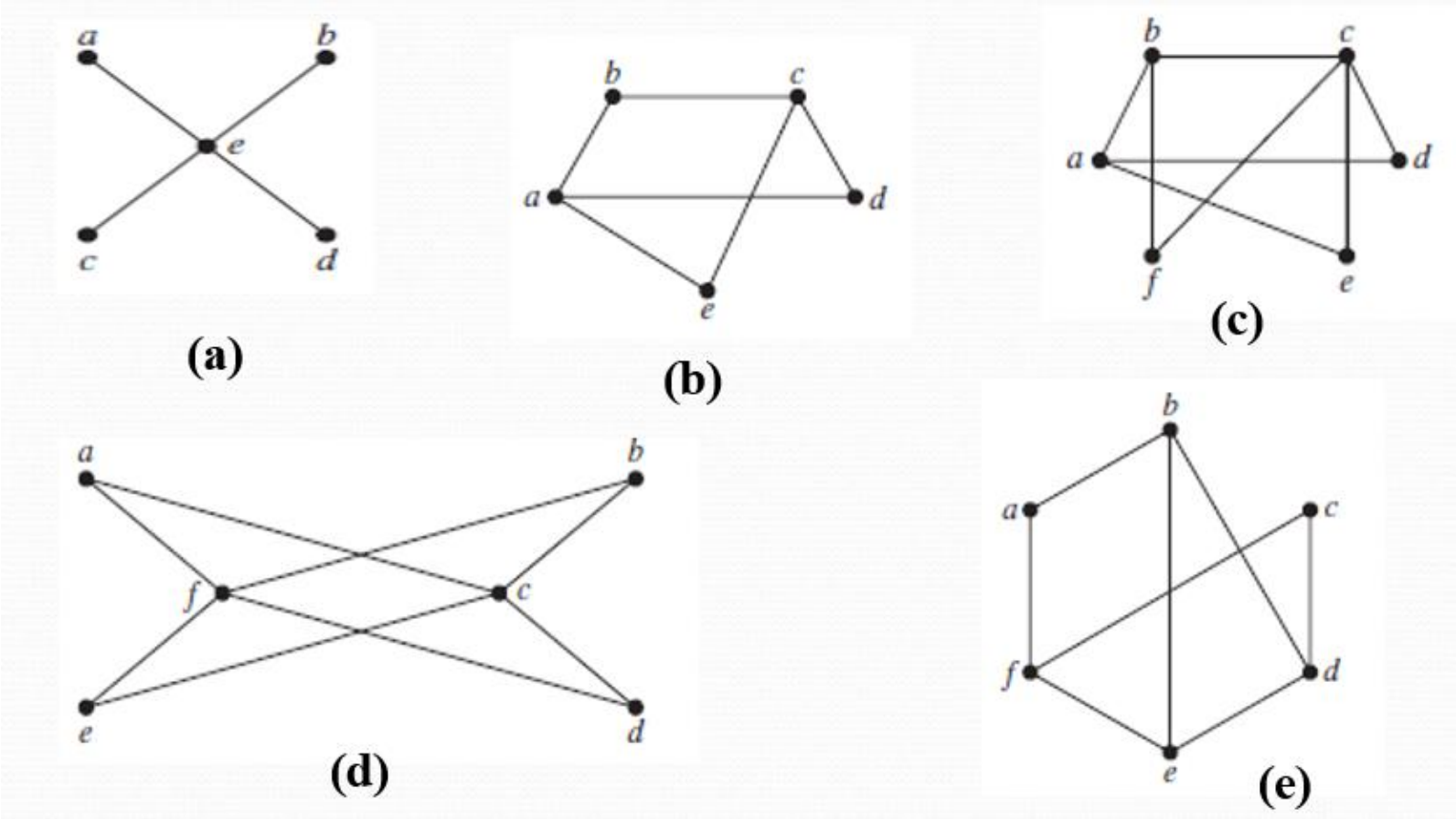




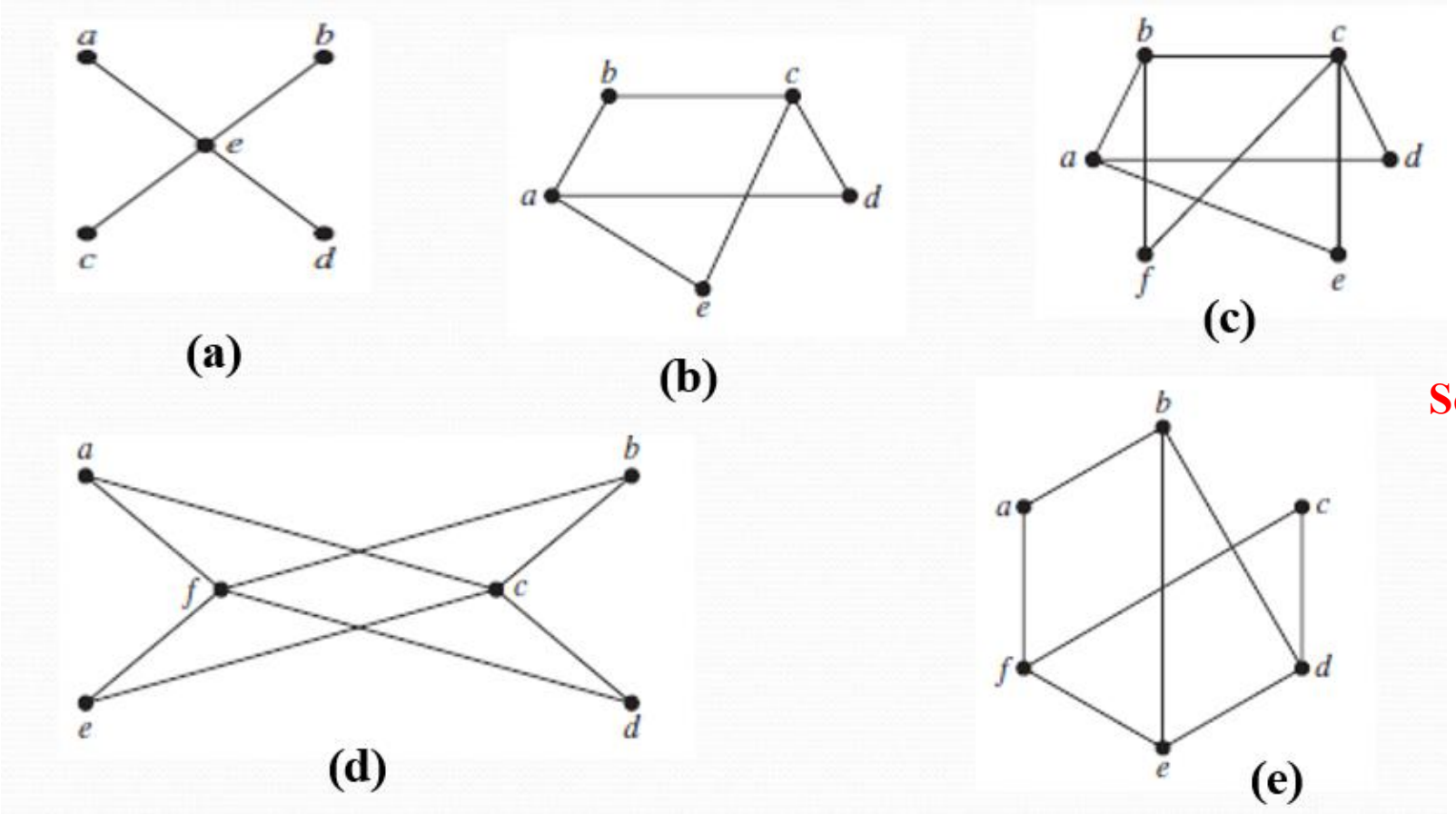




❖**Example 17:** Determine whether the graph is bipartite:



❖**Example 17:** Determine whether the graph is bipartite:



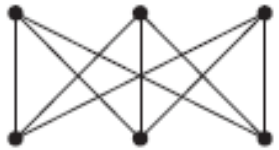
Solution: Bipartite Graph:: a, b, d

Complete Bipartite Graphs

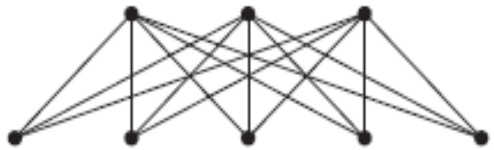
- A **Complete Bipartite Graph $K_{m,n}$** is a graph that has its vertex set partitioned into two subsets of m and n vertices, respectively with an edge between two vertices if and only if one vertex is in the first subset and the other vertex is in the second subset.
- i.e. each vertex of Vi is joined to every vertex of Vj by an unique edge.
- The complete bipartite graphs $K_{2,3}$, $K_{3,3}$, $K_{3,5}$, and $K_{2,6}$ are displayed below:



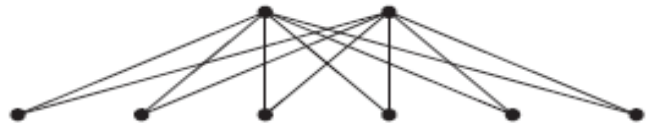
$K_{2,3}$



$K_{3,3}$

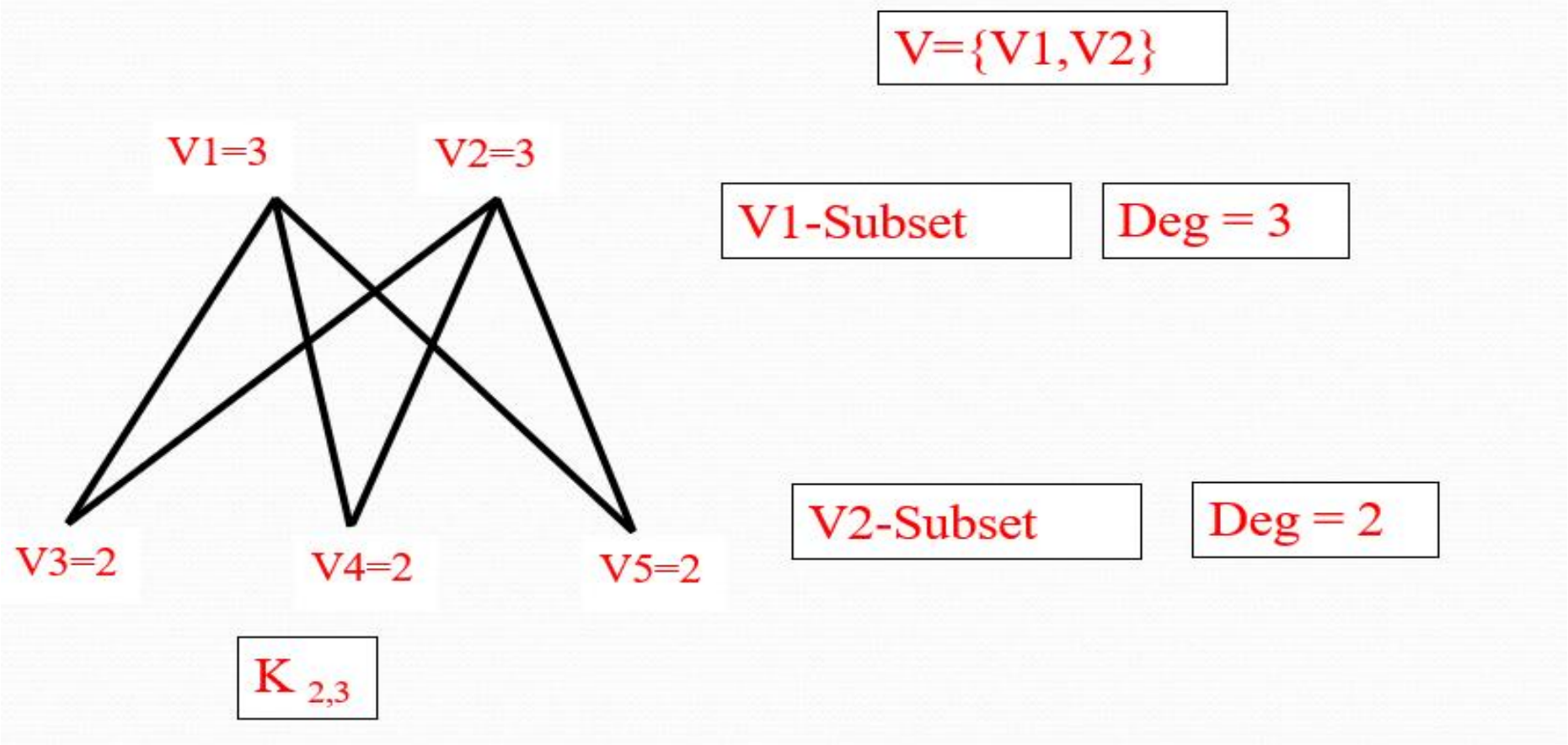


$K_{3,5}$



$K_{2,6}$

- It is denoted by $K_{m,n}$,where
  $| U | = m,$
  $| V | = n.$



Properties of a complete Bipartite graph

- Number of vertices = $m + n$.
- Number of edges = $m \times n$.
- Bipartite by definition.
- Contains no odd cycle.

Example on Complete Bipartite Graphs

❖**Example 18:** For which values of n are these graphs bipartite?

a)K_n b) C_n c) W_n d) Q_n

Solution:

- a) K_n is bipartite if and only if $n = 2$
- b) C_n is bipartite if and only if n is even.
- c) W_n is not bipartite for any n.
- d) Q_n is bipartite for all $n \geq 2$.

❖**Example 18:** For which values of n are these graphs bipartite?

a)K_n b) C_n c) W_n d) Q_n

a) K_n —**bipartite iff** $n \leq 2$.

•K_1 and K_2 are bipartite. For $n \geq 3$ K_n contains a triangle, so not bipartite.

b) C_n —**bipartite iff** n**is even**.

•An odd cycle is not bipartite; even cycles are.

c) W_n)standard wheel with $n$vertices = a cycle on $n-1$vertices plus a hub) — **not bipartite for the usual** $n \geq 4$.

•The hub joined to two adjacent cycle vertices creates a triangle, so wheels (for the standard definition) contain odd cycles and are not bipartite. (No nontrivial W_n is bipartite under the usual definition.)

d) Q_n)n-dimensional hypercube) — **bipartite for every** $n \geq 1$.

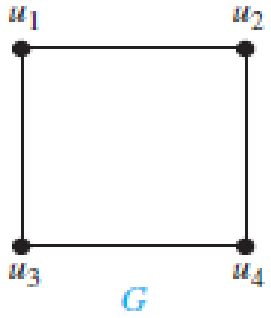
•Partition vertices by parity of the number of 1's in their binary label; all edges join different parities.

Isomorphism of Graphs

- We often need to know whether it is possible to draw two graphs in the same way. That is, do the graphs have the same structure when we ignore the identities of their vertices?
- The simple graphs **G1 = (V1,E1)** and **G2 = (V2,E2)** are isomorphic if there exists a one to-one and onto function f from V1 to V2 with the property that a and b are adjacent in G1 if and only if f(a) and f(b) are adjacent in G2, for all a and b in V1. Such a function f is called an **isomorphism**.
- Two simple graphs that are not isomorphic are called **nonisomorphic**.
- In other words, when two simple graphs are isomorphic, there is a one-to-one correspondence between vertices of the two graphs that preserves the adjacency relationship.

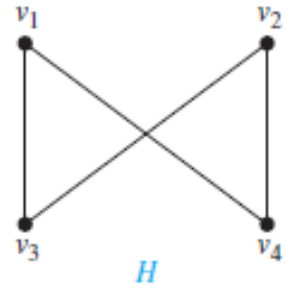
Example: Isomorphism of Graphs

❖**Example 19:** Show that the graphs G = (V ,E) and H = (W, F), displayed in Figure below , are isomorphic.



Solution: The function f with f (u1) = v1, f (u2) = v4, f (u3) = v3, and f (u4) = v2 is a one to-one correspondence between V and W.

Adjacent vertices in G are u1 and u2, u1 and u3, u2 and u4, and u3 and u4, and each of the pairs f (u1) = v1 and f (u2) = v4, f (u1) = v1 and f (u3) = v3, f (u2) = v4 and f (u4) = v2, and f (u3) = v3 and f (u4) = v2 consists of two adjacent vertices in H.



- It is often difficult to determine whether two simple graphs are isomorphic.
- There are ***n!*** possible one-to-one correspondences between the vertex sets of two simple graphs with n vertices.
- Testing each such correspondence to see whether it **preserves adjacency** and non adjacency is impractical if n is at all large.
- Sometimes it is not hard to show that two graphs are not isomorphic. In particular, we can show that two graphs are not isomorphic if we can find a property only one of the two graphs has, but that is preserved by isomorphism.
- A property preserved by isomorphism of graphs is called a **graph invariant**.

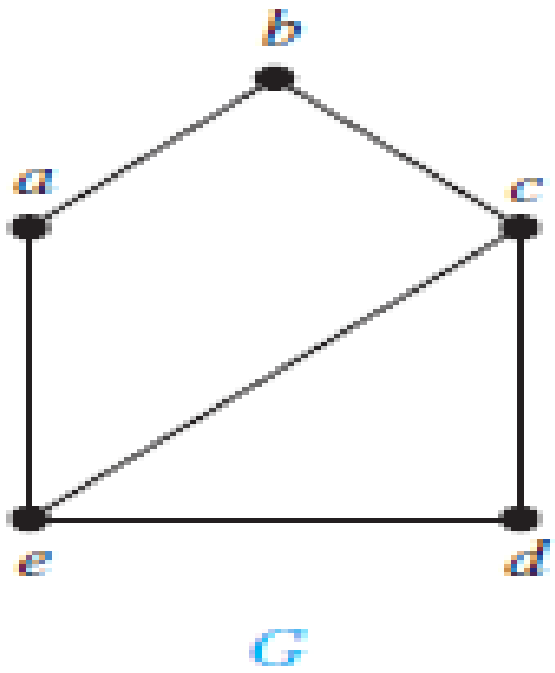
- Isomorphic simple graphs must have the **same number of vertices**, because there is a one-to-one correspondence between the sets of vertices of the graphs.
- Isomorphic simple graphs also must have the **same number of edges**, because the one-to-one correspondence between vertices establishes a one- to-one correspondence between edges.
- The degrees of the vertices in isomorphic simple graphs **must be the same**.
- That is, a vertex v of degree d in G must correspond to a vertex $f(v)$ of degree d in H, because a vertex w in G is adjacent to v if and only if $f(v)$ and $f(w)$ are adjacent in H.

❖Note: If two graphs are isomorphic, they must have:

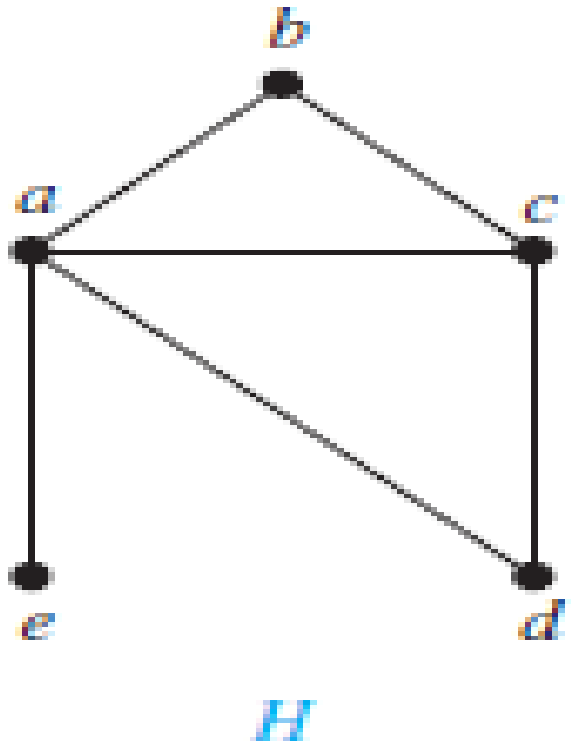
- Must have same number of vertices
- Must have same number of edges
- Must have equal number of vertices with same degree.
- Must have equal number of loops
- Must have equal number of pendent
- G1 and G2 must have equal number of pendent edges.
- If u and v are adjacent in G1 then the corresponding vertices in G2 are also adjacent.

❖In general, it is easier to prove two graphs are not isomorphic by proving that one of the above properties fails.

❖**Example 20:** Show that the graphs G = (V, E) and H = (W, F), displayed in figure below, are isomorphic or not.



G

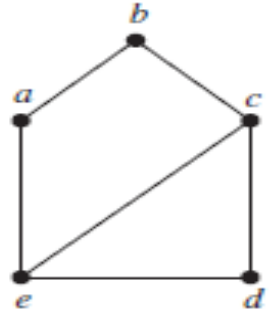


H

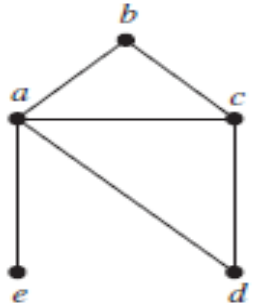
❖**Example 20:** Show that the graphs G = (V, E) and H = (W, F), displayed in figure below, are isomorphic or not.

Solution:

- Both G and H have five vertices and six edges.
- However, H has a vertex of degree one, namely, e, whereas G has no vertices of degree one.
- It follows that G and H are not isomorphic

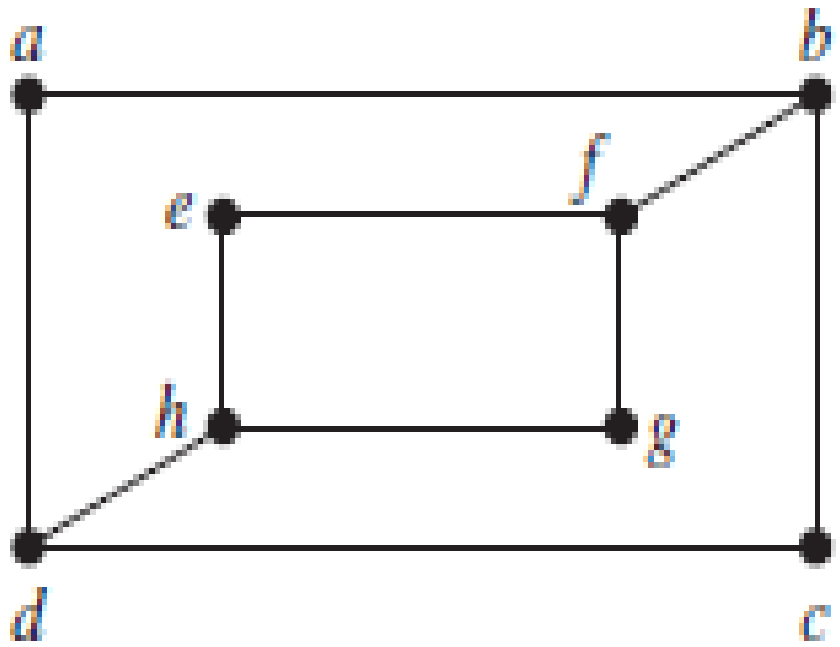


G

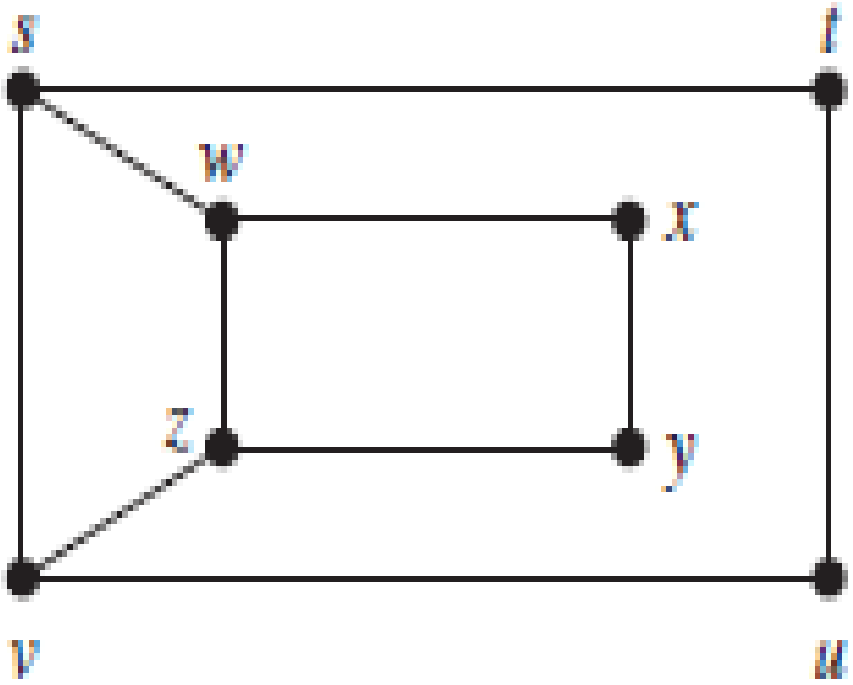


H

❖**Example 21:** Determine whether the following graphs are isomorphic.



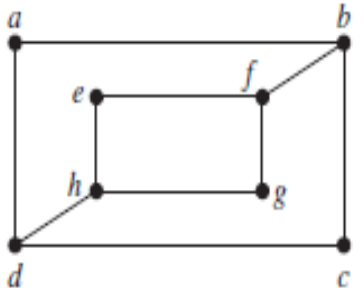
G



H

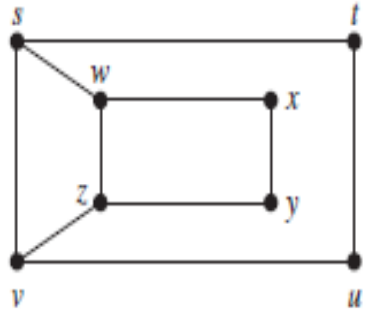
❖**Example 21:** Determine whether the following graphs are isomorphic.

➢**Solution:** The graphs G and H both have eight vertices and 10 edges. They also both have four vertices of degree two and four of degree three. Because these invariants all agree, it is still conceivable that these graphs are isomorphic.



G

➢However, G and H are not isomorphic. To see this, note that because deg(a) = 2 in G, a must correspond to either t , u, x, or y in H, because these are the vertices of degree two in H. However, each of these four vertices in H is adjacent to another vertex of degree two in H, which is not true for a in G.



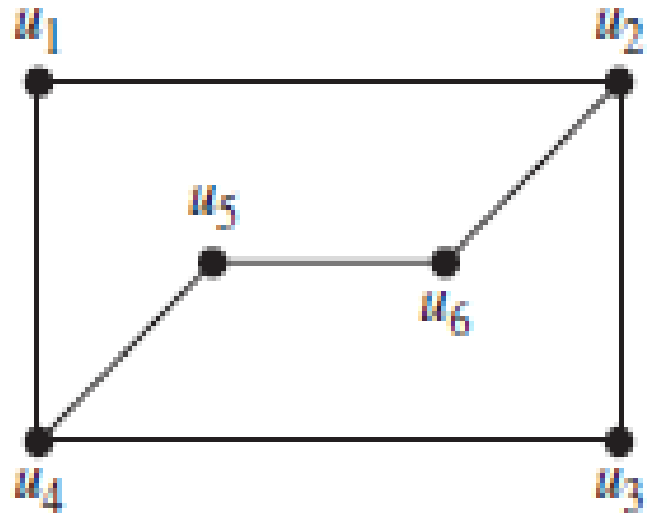
H

Solution:

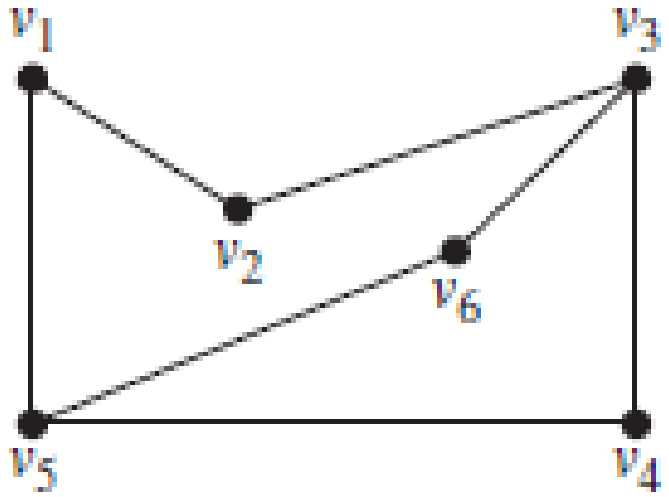
- In G vertex a → should map in H vertex t, u, x, y
- Adjacent to vertex a in G → b=3 and d=3
- In Graph H

t → s=3 and u=2	u → t=2 and v=3
x → w=3 and y=2	y → x=2 and z=3

❖**Example 22:** Determine whether the following graphs are isomorphic.



G

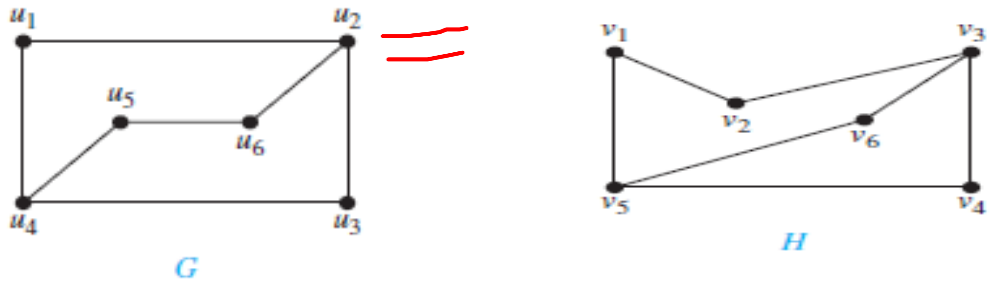


H

❖**Example 22:** Determine whether the following graphs are isomorphic.

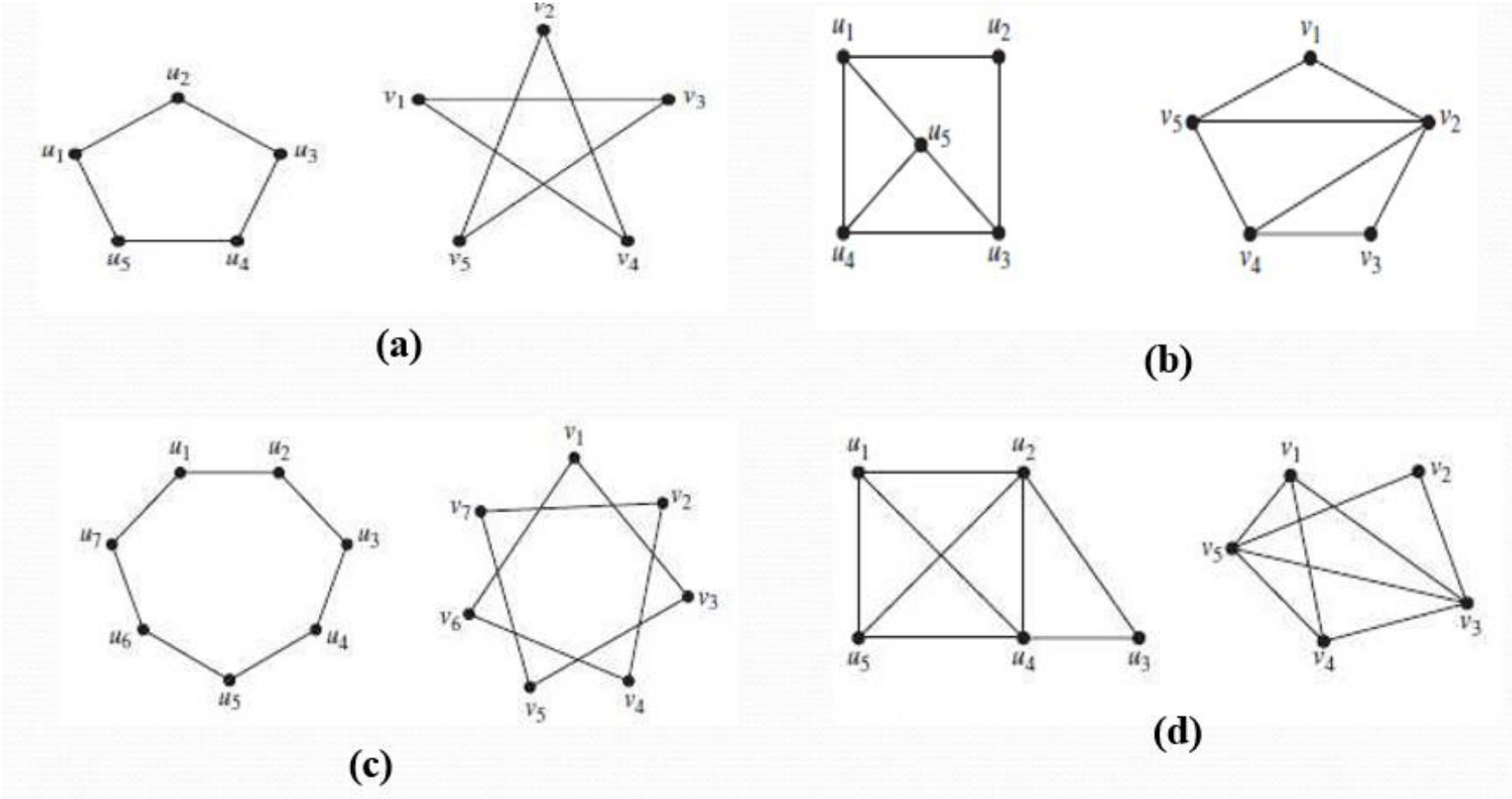
Solution:

- Both G and H have six vertices and seven edges.
- Both have four vertices of degree two and two vertices of degree three.
- It is also easy to see that the subgraphs of G and H consisting of all vertices of degree two and the edges connecting them are isomorphic.



G *H*

❖**Example 23:** Determine whether the following graphs are isomorphic or not.



❖**Example 24:** Are the simple graphs with the following adjacency matrices isomorphic?

Solution:

a. Both graphs consist of 2 sides of a triangle; they are clearly isomorphic.

b. The graphs are not isomorphic, since the first has 4 edges and the second has 5 edges,

c. The graphs are not isomorphic, since the first has 4 edges and the second has 3 edges,

a) $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$

Connectivity-Paths

❖A **path** is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex along edges of the graph.

✓Simple path : No repetition of edges.

✓ Elementary path: No repetition of vertices .

❖As the **path travels** along its edges, it visits the vertices along this path, that is, the endpoints of these edges.

❖The **path is a circuit** if it begins and ends at the same vertex, that is, if u = v, and has length greater than zero.

❖A **path or circuit** is simple if it does not contain the same edge more than once.

❖A **circuit** in a graph is also called as cycle in a graph.

✓*Simple Circuit:* No repetition of edges.

✓*Elementary circuit:* No repetition of vertices

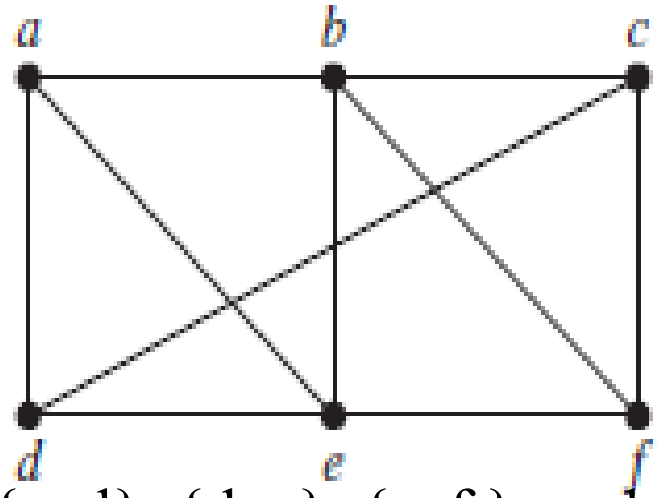
❖A **walk** is an alternating sequence of vertices and edges of a graph.

❖A **path is a walk** that does not include any vertex twice, except that its first vertex might be the same as its last.

❖A **trail is a walk** that does not pass over the same edge twice.

❖A trail might visit the **same vertex twice**, but only if it comes and goes from a different edge each time.

❖**Example 25 :** In the simple graph shown in figure below:



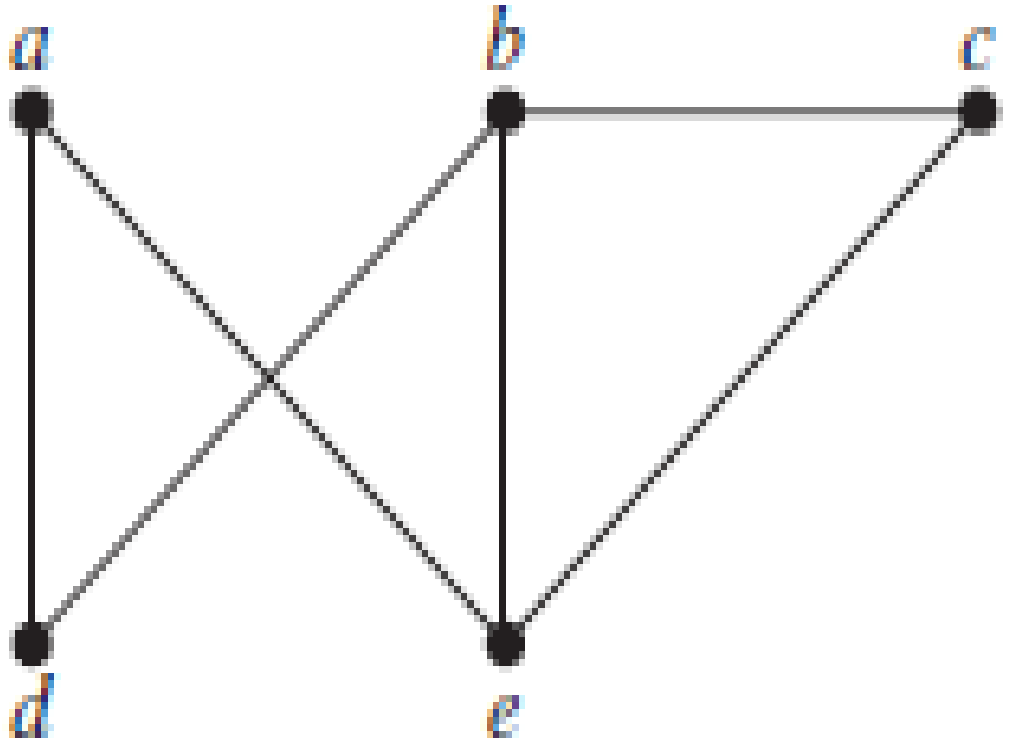
- In graph a, d, c, f , e is a simple path of length 4, because {a, d}, {d, c}, {c, f }, and {f, e} are all edges.
- However, d, e, c, a is not a path, because {e, c} is not an edge.
- Note that b, c, f , e, b is a circuit of length 4 because {b, c}, {c, f }, {f, e}, and {e, b} are edges, and this path begins and ends at b.
- The path a, b, e, d, a, b, which is of length 5, is not simple path because it contains the edge {a, b} twice.

❖**Example 26:** Does each of these lists of vertices form a path in the following graph? Which paths are simple paths? Which are circuits? What are the lengths of those that are paths?

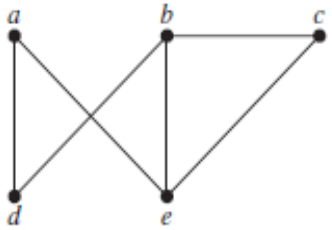
a) a, e, b, c, b

b) a, e, a, d, b, c, a

c) e, b, a, d, b, e



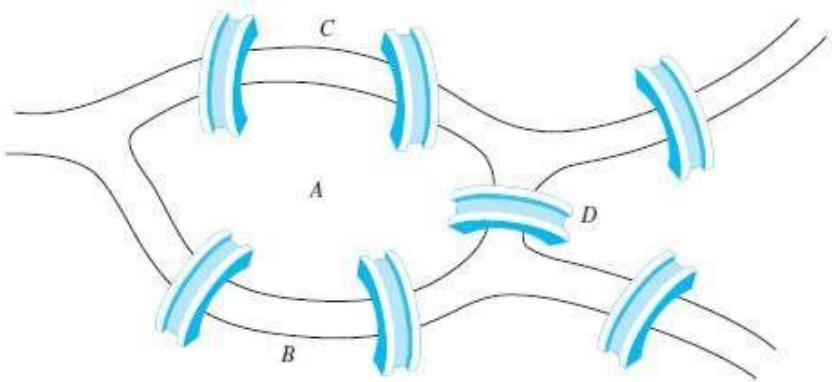
❖**Example 26:** Does each of these lists of vertices form a path in the following graph? Which paths are simple paths? Which are circuits? What are the lengths of those that are paths?

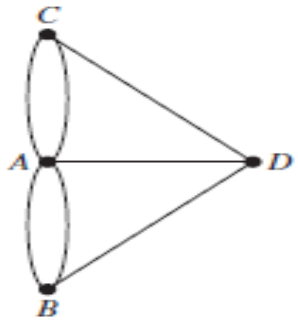


Solution:

- a) a, e, b, c, b **:** This is a path of length 4, but it is not simple, since edge {b, c} is used twice. It is not a circuit, since it ends at a different vertex from the one at which it began.
- b) a, e, a, d, b, c, a : This is not a path, since there is no edge from c to a.
- c) e, b, a, d, b, e : This is not a path, since there is no edge from b to a.
- d) c, b, d, a, e, c : This is a path of length 5 (it has 5 edges in it). It is simple, since no edge is repeated. It is a circuit since it ends at the same vertex at which it began.

Euler Paths & Euler Circuits





- The towns people took long walks through town on Sundays. They wondered whether it was possible to start at some location in the town, travel across all the bridges once without crossing any bridge twice, and return to the starting point.
- The Swiss mathematician Leonhard Euler solved this problem. His solution, published in 1736, may be the first use of graph theory.
- Euler studied this problem using the multigraph obtained when the four regions are represented by vertices and the bridges by edges.

Euler Paths	Euler Circuits
❖ An **Euler path** is a path that uses every edge of a graph exactly once. ❖ An Euler path starts and ends at **different vertices**. ❖ If a graph G has an Euler path, then it must have exactly **two odd vertices**. OR ❖ If the number of odd vertices in G is anything other than 2, then G cannot have an **Euler path**.	❖ An **Euler circuit** is a circuit that uses every edge of a graph exactly once. ❖ An Euler circuit starts and ends at the **same vertex**. ❖ If a graph G has an Euler circuit, then all of its vertices must be **even vertices.** OR ❖ If the number of odd vertices in G is anything other than 0, then G cannot have an Euler circuit.

In **Euler paths and Euler circuits,** the game is to find paths or circuits that include **every edge** of the graph once (and only once).

❖In **Euler paths and Euler circuits,** the game is to find paths or circuits that include **every edge** of the graph once (and only once).

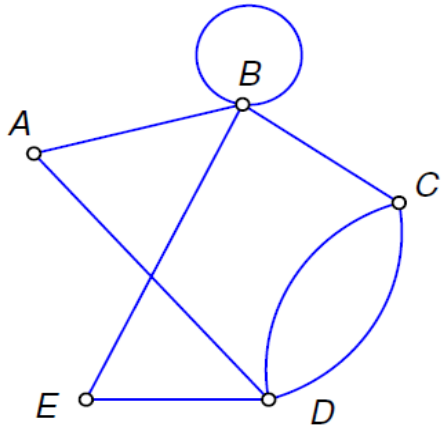
#Odd Vertices	Euler Path?	Euler Circuit?
0	Yes*	Yes*
2	Yes*	No
4, 6, 8,……	No	No
1, 3, 5,……	No such graph exist	

(* Provided the graph is connected)

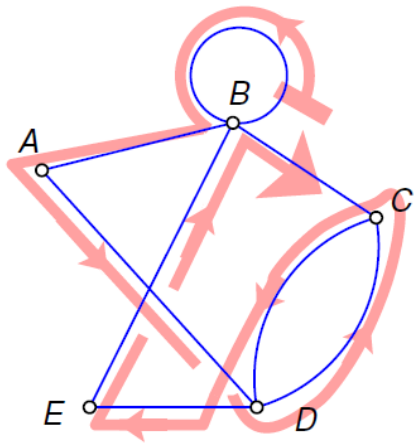
❖**Bridges -** Removing a single edge from a connected graph can make it disconnected. Such an edge is called a **bridge**.

❖Loops cannot be bridges, because removing a loop from a graph cannot make it disconnected.

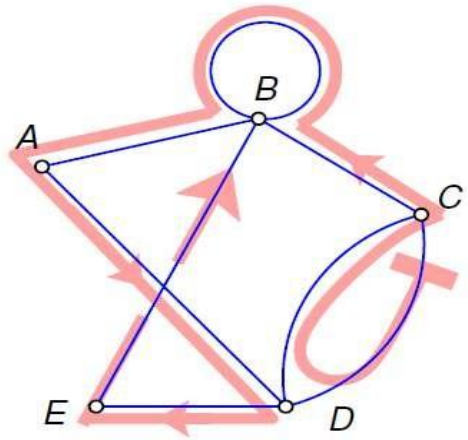
❖If two or more edges share both endpoints, then removing any one of them cannot make the graph disconnected. Therefore, none of those edges is a bridge.



Vertex	Degree	Odd/Even
A	2	Even
B	5	Odd
C	3	Odd
D	4	Even
E	2	Even



Euler Path:
BBADCDEBC



Euler Path:
CDCBBADEB

Under what condition K_{mn}(Complete bipartite graph) will have an eulerian ckt ?

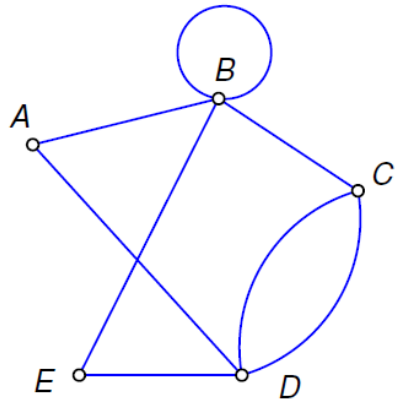
Case1: m=n & both are even. True(Ex. K2,2)

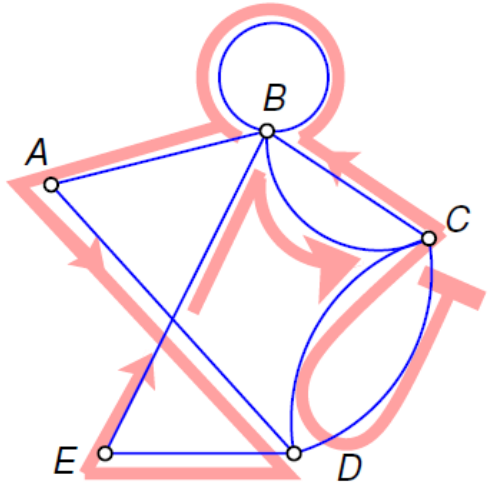
Case2: m=n & both are odd. False (Ex. K3,3)

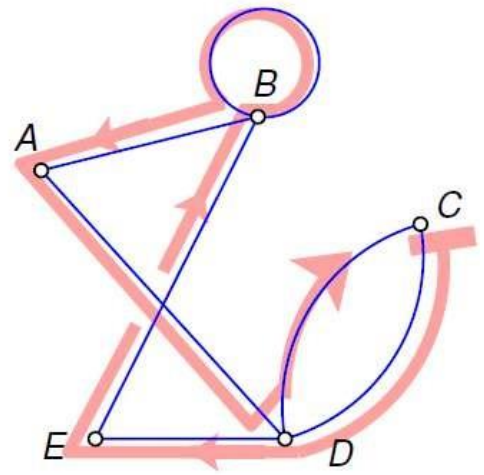
Case3: m≠n & both are even. True (Ex. K2,4)

Case4: m ≠n & one is even & one is odd. False (Ex. K2,3)

Note:For Bipartite K4,3 complete graph , there are 4 vertices of degree 3 and 3 vertices of degree 4. Hence neither Euler's path or circuit exists.







Vertex	Degree	Odd/Even
A	2	Even
B	5	Odd
C	3	Odd
D	4	Even
E	2	Even

Euler Circuit :
CDCBBADEBC

Euler Circuit :
CDEBBADC

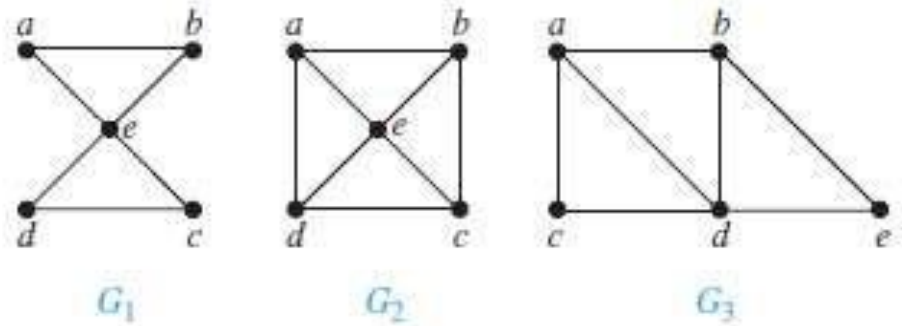
Euler path & Euler circuit for directed graphs

- For a directed graph to have Euler path, on each node, number of incoming edges should be equal to number of outgoing nodes except start node where out degree is one more than in degree and end node where incoming is one more than outgoing.

- To have Euler circuit, all nodes should have in degree equal to out degree.

- We have to keep in mind that for both directed and undirected graphs, above conditions hold when all nodes with non-zero degree are part of strongly connected component of graph.

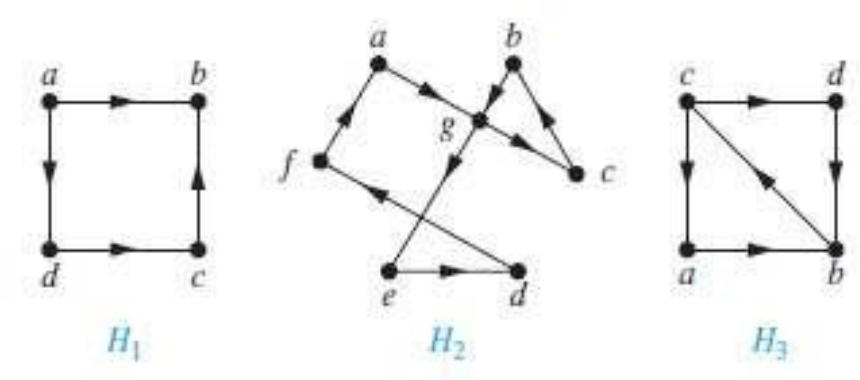
❖**Example 27 :** Which of the undirected graphs in Figure have an Euler circuit? Of those that do not, which have an Euler path?



Solution:

- The graph G1 has an Euler circuit, for example, a, e, c, d, e, b, a.
- Neither of the graphs G2 or G3 has an Euler circuit.
- G2 does not have an Euler path.
- G3 has an Euler path, namely, a, c, d, e, b, d, a, b.

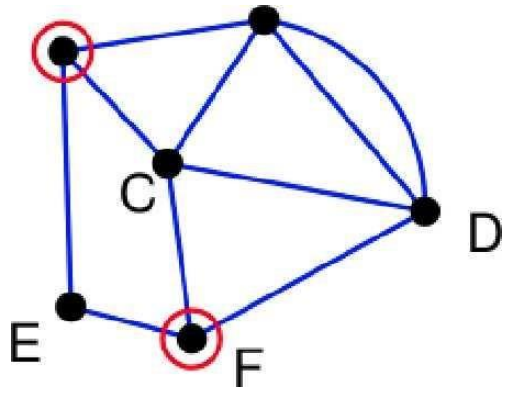
❖**Example 28 :** Which of the directed graphs in Figure have an Euler circuit? Of those that do not, which have an Euler path?



Solution:

- The graph H2 has an Euler circuit, for example a, g, c, b, g, e, d, f, a.
- Neither H1 nor H3 has an Euler circuit.
- H3 has an Euler path, namely, c, a, b, c, d, b, but H1 does not.

❖**Example 29:** Finding Euler Circuits and Paths



Solution: Euler Path: FEACBDCFDBA

Hamilton Paths and Circuits

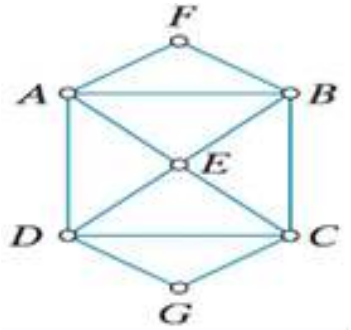
❖A **Hamilton path** in a graph is a path that includes **each vertex** of the graph once and only once.

❖A **Hamilton circuit** is a circuit that includes **each vertex** of the graph once and only once.

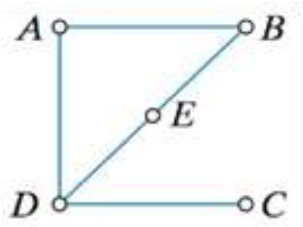
✓*Theorem1:* If the sum of the degree for each pair of vertex is ≥ n-1, then there exist a hamilton path.

✓*Theorem2:* If the degree of each vertex in G ie d(v) ≥ n/2 , then G will contain hamilton ckt.

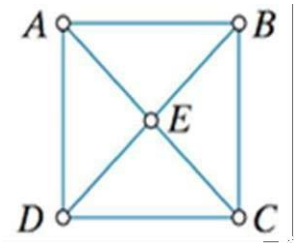
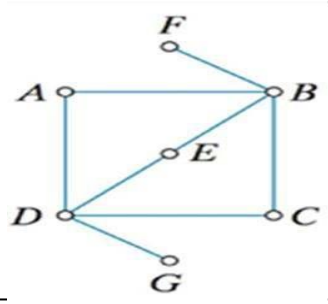
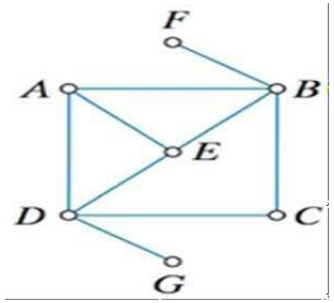
Hamilton versus Euler



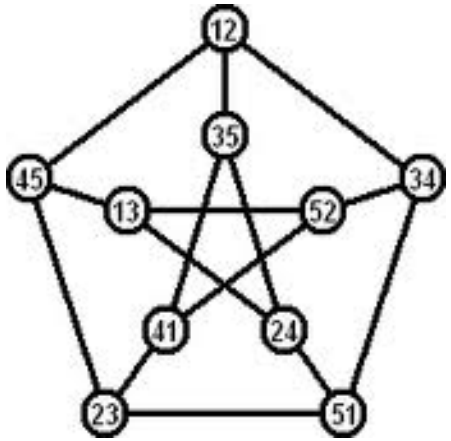
- If a graph has a Hamilton circuit, then it automatically has a Hamilton path-(the Hamilton circuit can always be truncated into a Hamilton path by dropping the last vertex of the circuit.)
- Contrast this with the mutually exclusive relationship between Euler circuits and paths: If a graph has an Euler circuit it cannot have an Euler path and vice versa.
- Hamilton circuit is A, F, B, C, G, D, E, A &
- Hamilton path is A, F, B, C, G, D, E

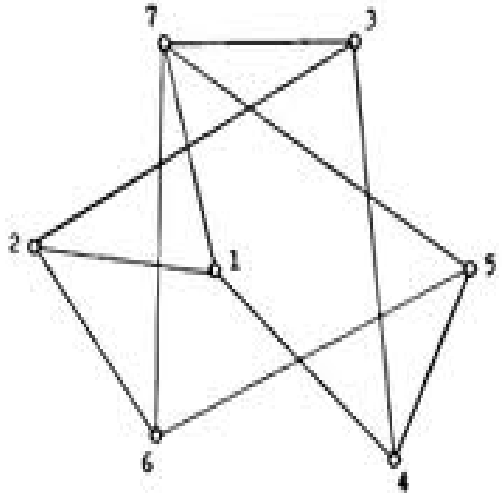


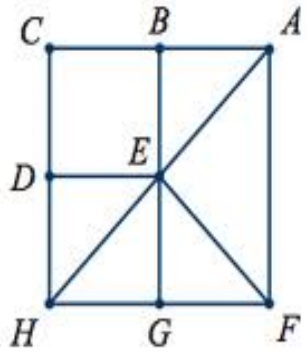
- Has no Euler circuits but does have Euler paths as C, D, E, B, A, D.
- Has no Hamilton circuits (sooner or later you have to go to C, and then you are stuck) but does have Hamilton paths as A, B, E, D, C
- This shows that a graph can have a Hamilton path but no Hamilton circuit!.

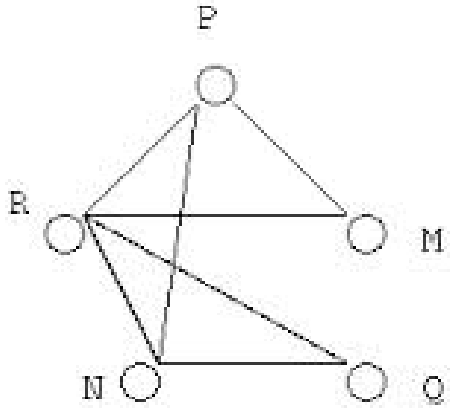
	Has neither Euler circuits nor paths (it has four odd vertices) Has Hamilton circuits as A,B,C,D,E,A - there are plenty more) and consequently has Hamilton paths as A, B, C, D, E.
	▪ Has no Euler circuits but has Euler paths (F and G are the two odd vertices) and ▪ Has neither Hamilton circuits nor Hamilton paths.
	▪ Has neither Euler circuits nor Euler paths (too many odd vertices) and ▪ Has neither Hamilton circuits nor Hamilton paths

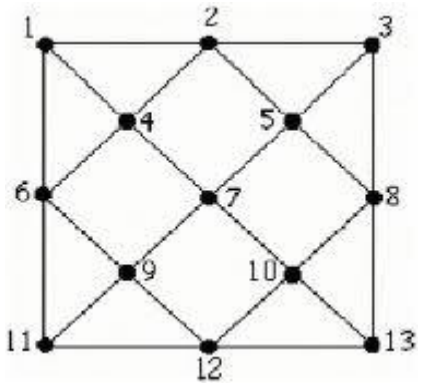
Verify


12
35
45
34
13
52
41
24
23
51

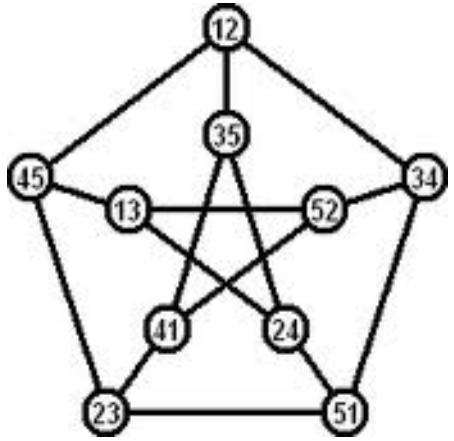

7
3
2
1
5
6
4


C
B
A
E
D
H
G
F

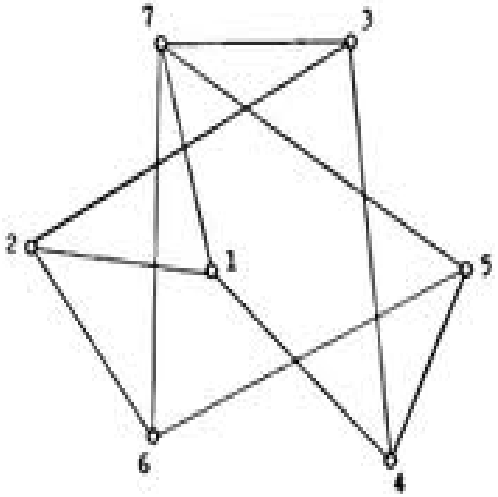

P
R
M
N
Q

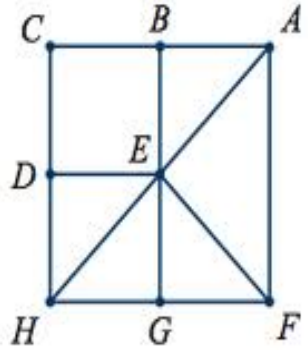

1
2
3
4
5
6
7
8
9
10
11
12
13

- Answer 1) Hamilton path(12,45,23,51,34,52,41,35,24,13) is possible but not Hamilton circuit

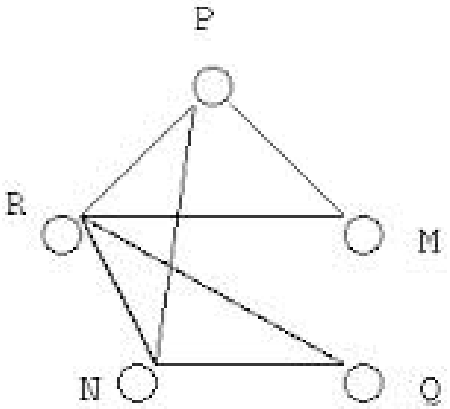


- Answer 2) Hamiltonian Circuit($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 1$)
- Hamiltonian path ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$)



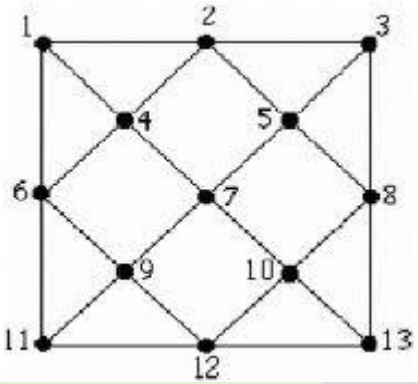


- Answer 1) Hamilton path(C → B → E → A → F → G → H → D)
- Hamilton circuit(A → B → C → D → E → H → G → F → A)



- Answer 2) Hamiltonian Circuit does not exist
- Hamiltonian path (M → P → R → O → N)

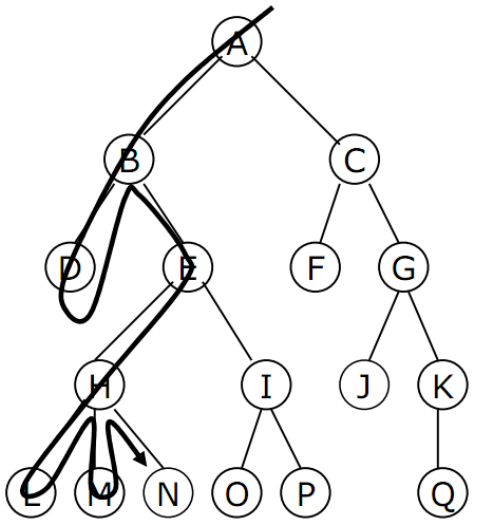
- Answer 3) Hamiltonian Circuit (1 → 4 → 2 → 3 → 5 → 8 → 13 → 12 → 10 → 7 → 9 → 11 → 6 → 1)
- Hamiltonian path (4 → 2 → 3 → 5 → 8 → 13 → 12 → 10 → 7 → 9 → 11 → 6 → 1)



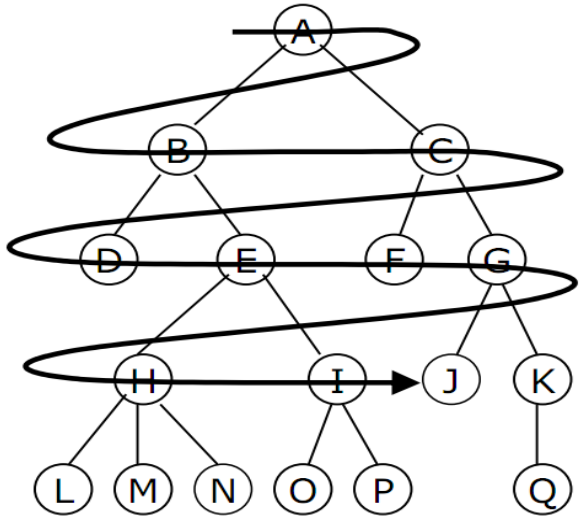
- Find a hamilton path and a hamilton circuit in $K_{4,3}$.
- It has a hamilton path but not the hamilton ckt.

Graph Traversing

- Given a graph G(V,E), explore every vertex and every edge
- Using adjacency list is more efficient
- Example algorithms:
  - Depth-first search (DFS)
  - Breadth-first search (BFS)

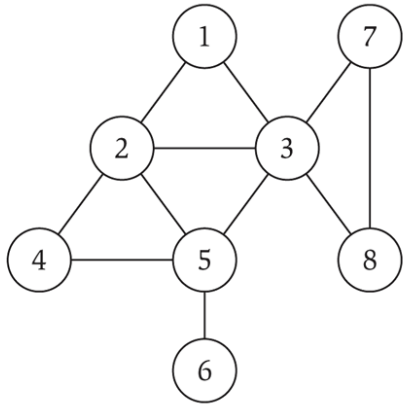


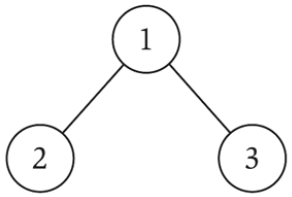
DFS



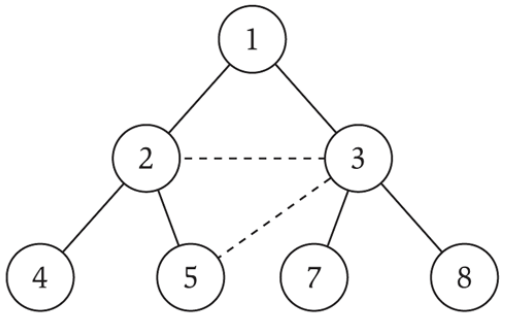
BFS

- BFS example:

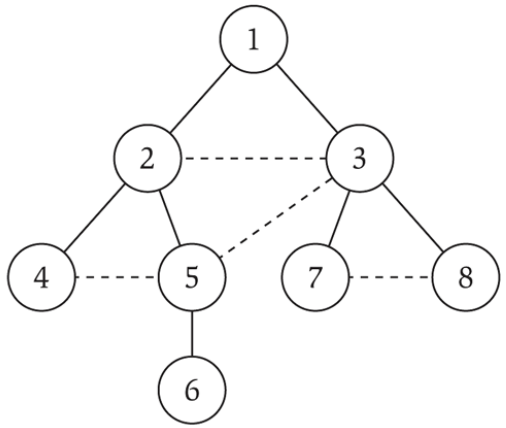




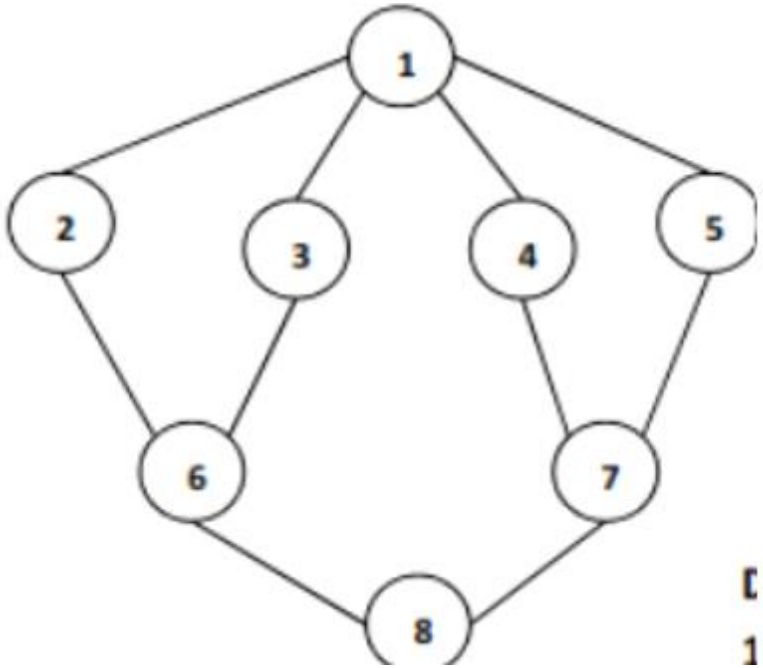
(a)



(b)



(c)



DFS: 1,2,6,3,8,7,4,5

BFS:1,2,3,4,5,6,7,8

- DFS applications:
  - Determines whether G is connected
  - Computes the connected components of G (strongly connected components of a *digraph* = directed graph)
  - Path / cycle finding
  - Topological sort (ordering of vertices of digraph G(V,E) such that for every edge *(u,v)* in E, *u* appears before *v* in the ordering)
  - Linear running time
- BFS applications:
  - Computes the **distance** from *s* to each reachable vertex in unewighted G
  - Finds **shortest paths** from *s* to all other nodes in unweighted G
  - Finds a simple cycle, if there is one
  - Computes the connected components of G

Shortest Path Problem

- In **graph theory**, the shortest path problem is the problem of finding a path between two vertices (or nodes) in a graph such that the sum of the weights of its constituent edges is minimized.
- The problem of finding the shortest path between two intersections on a road map (the graph's vertices correspond to intersections and the edges correspond to road segments, each weighted by the length of its road segment) may be modeled by a special case of the shortest path problem in graphs.
- There are several different algorithms that find a shortest path between two vertices in a weighted graph.
- **Dijkstra's Algorithm::** is an algorithm for finding the shortest paths between nodes in a graph, which may represent, for example, road networks. It was conceived by computer scientist Edsger W. Dijkstra in 1956 and published three years later.

Dijkstra's Algorithm- Step:

- Dijkshtra's algorithm to find the shortest path from vertex a to z of a graph G. Let G(V,E) be a simple graph and **a, z ∈ V**.
- Suppose L(x) is the label of the vertex which represents the length of the shortest path from vertex a. Wij=Weight of an edge eij=(vi,vj).
- **Consider following Steps:**
- **Step 1:** Let P be the set of those vertices which have permanent labels and T be set of all vertices of G.

$$\text{Set } L(a) = 0, \qquad L(x) = \infty \ \forall \ x \in T \quad \text{and } x \neq a$$

$$P = \emptyset \text{ and } T = v.$$

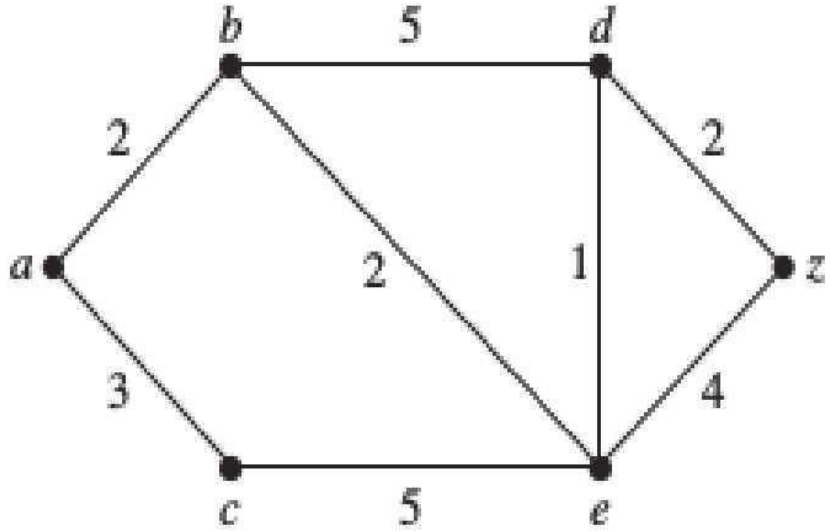
Dijkstra's Algorithm- Step:

- **Step 2:** Select the vertex v in T which has smallest label. This label is called the permanent label of v. Also set P as P U {v} and T - {v}
  - ✓ If v = z then L(z) is the length of the shortest path from the vertex a to z and stop the procedure.
- **Step 3:** If $v \neq z$, then revise the labels of the vertices of T. i.e. The vertices which do not have permanent labels.
  - ✓ The new label of x in T is given by

$$L(x) = \min\{\text{old } L(x), L(v) + w(v,x)\}$$

  - ✓ Where w(v,x) is the weight of the edge joining v and x. If there is no edge joining v and x then take $w(v,x) = \infty$.
- **Step 4:** Repeat the step 2 and 3 until z gets permanent label.

❖**Example 30:** Use Dijkstra's algorithm to find a shortest path between a and z.



Solution: Possible Path

a-b-d-z = 2+5+2=9

a-c-e-z = 3+5+4=12

a-b-e-z = 8

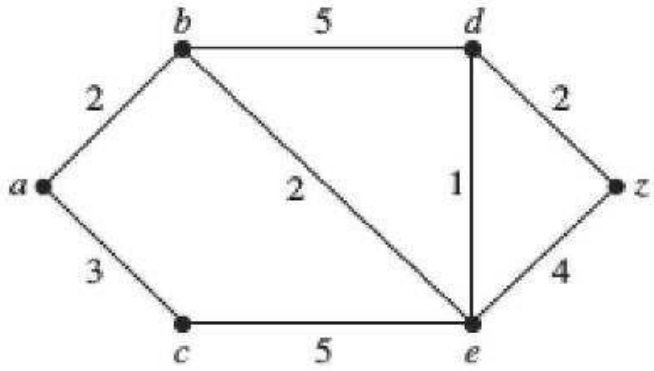
a-b-e-d-z=7

a-c-e-d-z =11

a-c-e-b-d-z = 17

❖**Example 30:** Use Dijkstra's algorithm to find a shortest path between a and z.

Solution: Step 1: Let P be the set of vertices which have permanent labels and T be set of all vertices of G.



$P = \emptyset$ and Set $L(a) = 0$, $T = \{a, b, c, d, e, z\}$.

$L(x) = \infty$

Step 2: v = a the permanent label of a = 0. $P = \{a\}$ and $T=\{b, c, d, e, z\}$

$L(b) = \min\{\text{old } L(b), L(a) + w(a,b)\} = \min\{\infty, 0 + 2\} = 2$

$L(c) = \min\{\text{old } L(c), L(a) + w(a,c)\} = \min\{\infty, 0 + 3\} = 3$

$L(d) = \min\{\text{old } L(d), L(a) + w(a,d)\} = \min\{\infty, 0 + \infty\} = \infty$

$L(e) = \min\{\text{old } L(e), L(a) + w(a,e)\} = \min\{\infty, 0 + \infty\} = \infty$

$L(z) = \min\{\text{old } L(z), L(a) + w(a,z)\} = \min\{\infty, 0 + \infty\} = \infty$

Therefore L(b) = 2 is minimum label.

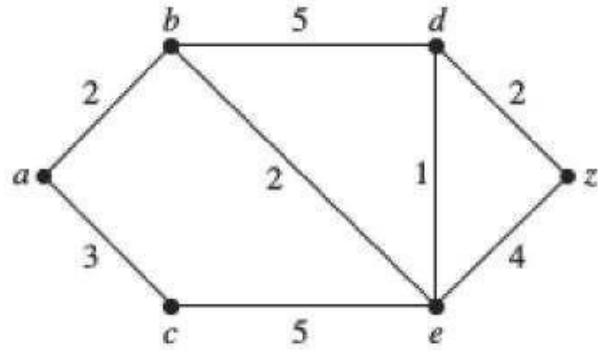
Step 3: v = b the permanent label of b = 2. P = {a, b} and T={c, d, e, z}

L(c) = min{old L(c), L(b) + w(b,c)} = min{3, 2 + ∞} = 3

L(d) = min{old L(d), L(b) + w(b,d)} = min{∞, 2 + 5} = 7

L(e) = min{old L(e), L(b) + w(b,e)} = min{∞, 2 + 2} = 4

L(z) = min{old L(z), L(b) + w(b,z)} = min{∞, 2 + ∞} = ∞



Therefore L(c) = 3 is minimum label.

Step 4: v = c the permanent label of c = 3. P = {a, b, c} and T={d, e, z}

L(d) = min{old L(d), L(c) + w(c,d)} = min{7, 3 + ∞} = 7

L(e) = min{old L(e), L(c) + w(c,e)} = min{4, 3 + 5} = 4

L(z) = min{old L(z), L(c) + w(c,z)} = min{∞, 3 + ∞} = ∞

No labels are changed. Then e is put into P Therefore L(e) = 4 is minimum label.

Step 5: v = e the permanent label of e = 4. P = {a, b, c, e} and T={d, z}

L(d) = min{old L(d), L(e) + w(e,d)} = min{7, 4 + 1} = 5

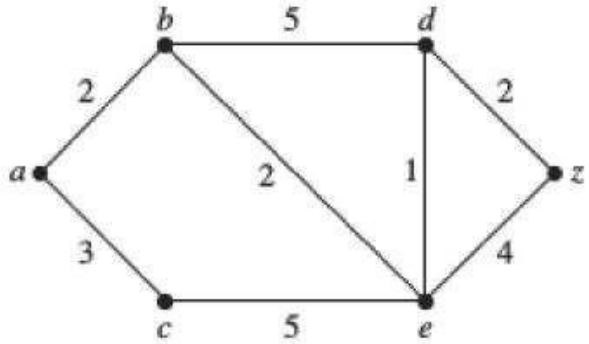
L(z) = min{old L(z), L(e) + w(e,z)} = min{∞, 4 + 4} = 8 Therefore

L(d) = 5 is minimum label.

Step 6: v = d the permanent label of d = 5. P = {a, b, c, e, d} and T={z}

L(z) = min{old L(z), L(d) + w(d,z)} = min{8, 5 + 2} = 7

Therefore L(z) = 7 is minimum label.



Step 7: v = z the permanent label of z is 7.

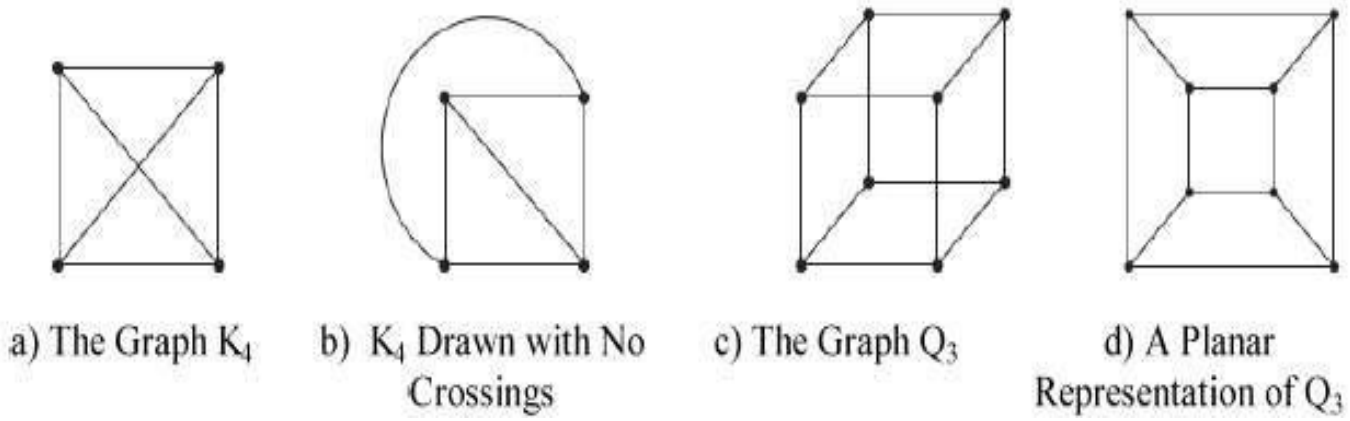
Therefore a shortest path is a, b, e, d, z, with length 7.

Planner Graph

- A graph is called **planar** if it can be drawn in the plane without any edges crossing (where a crossing of edges is the intersection of the lines or arcs representing them at a point other than their common endpoint). Such a drawing is called a planar representation of the graph.
- A graph may be planar even if it is usually drawn with crossings, because it may be possible to draw it in a different way without crossings.
- We can show that a graph is planar by displaying a planar representation. It is harder to show that a graph is nonplanar.

❖**Example 31:** Is K4 and Q3 (shown in Figure a and c) planar?

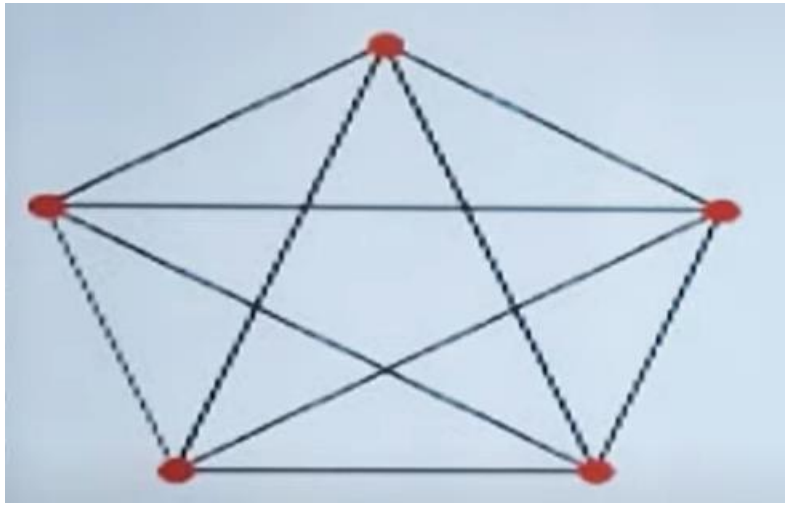
Solution: K4 and Q3 are planar because it can be drawn without crossings, as shown in figure b and d.

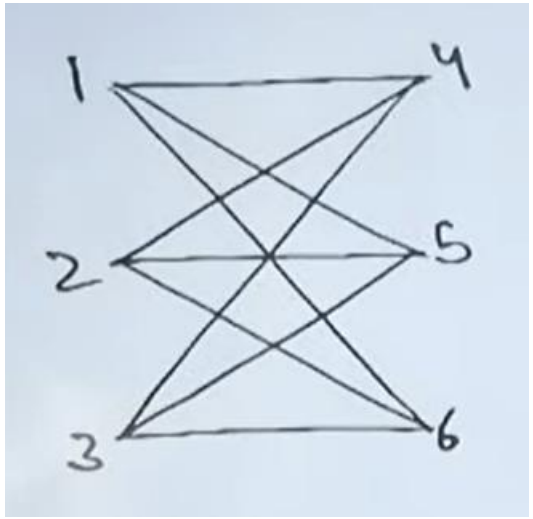


a) The Graph K_4 b) K_4 Drawn with No Crossings c) The Graph Q_3 d) A Planar Representation of Q_3

❖**Example 31:** Is K5, K3,3 planar?

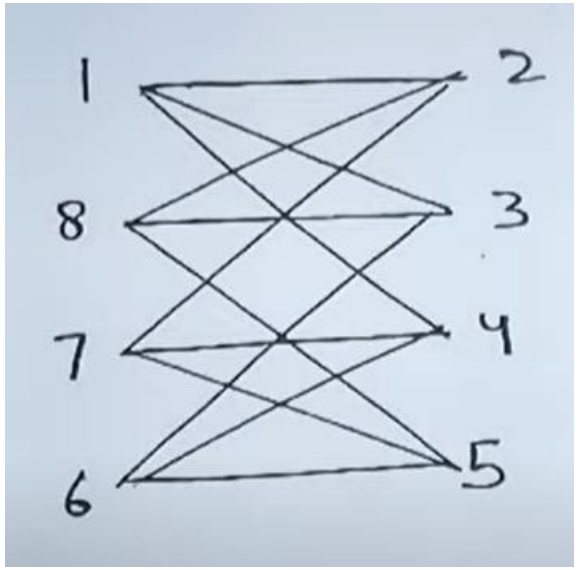
Solution:

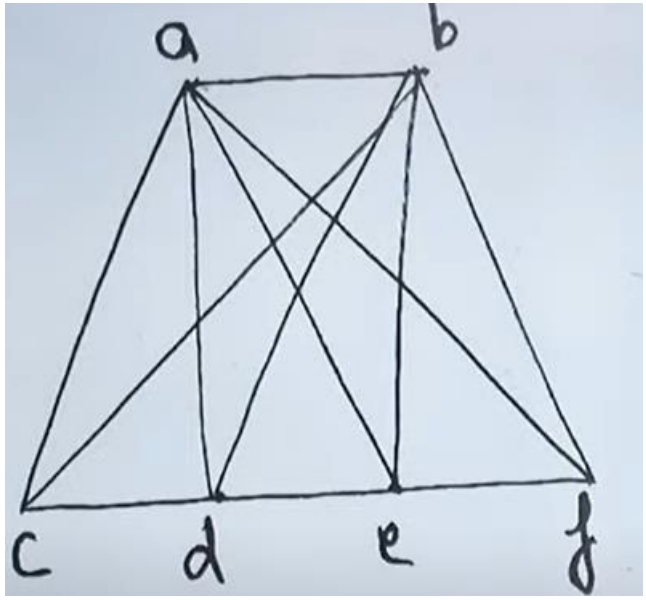


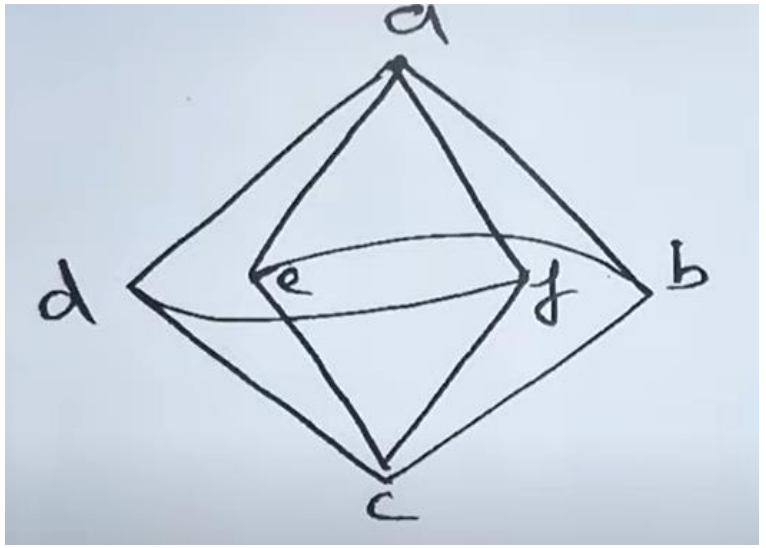


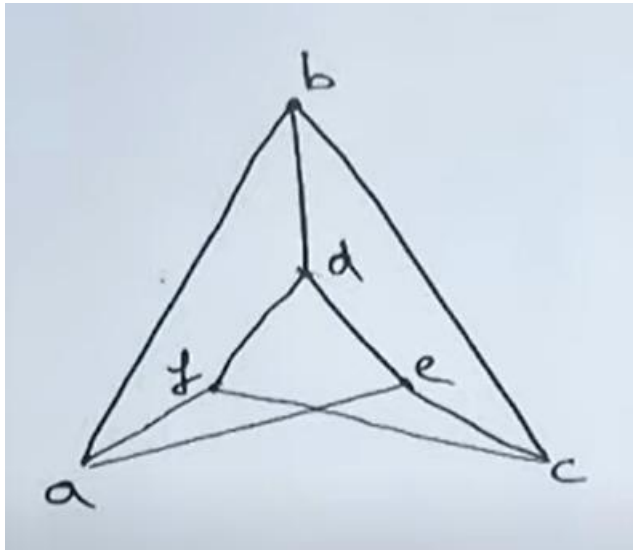
❖**Example 31:** Is the graph planar?

Solution:



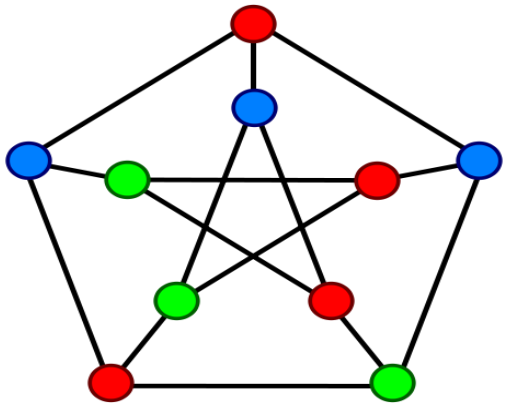






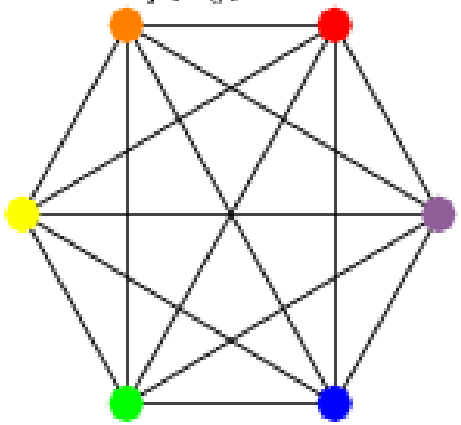
Coloring of a Graph

- One of the main way is to color the vertices of G such that no two adjacent vertices have the same color. It is called proper coloring.
- **Chromatic number:** The chromatic number of a graph G is the minimum number of colors required for the proper coloring of the graph G.
- In this case =3
- This graph is called 3- chromatic

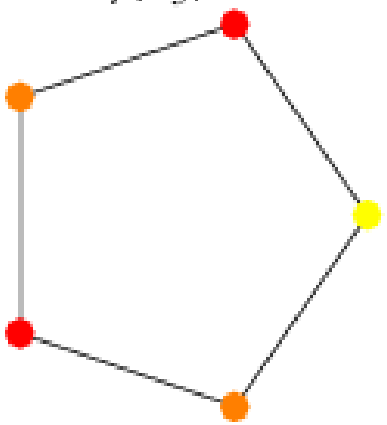


Sr. No.	Type of graph G	Chromatic no. K(G)
1.	Complete graph Kn	n
2.	Star graph Cn, n>1	2
3.	Cycle graph Cn , n>1	2 for n is even 3 for n is odd
4.	Wheel graph Wn, n >2	3 for n is odd 4 for n is even

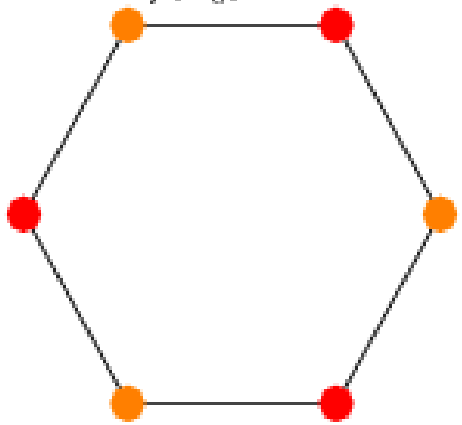
$\gamma(K_6) = 6$



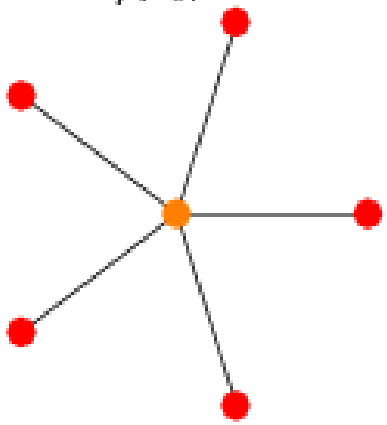
$\gamma(C_5) = 3$



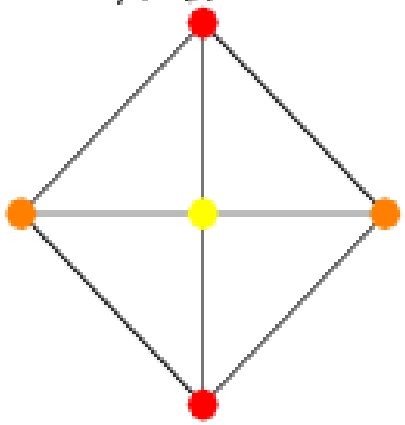
$\gamma(C_6) = 2$



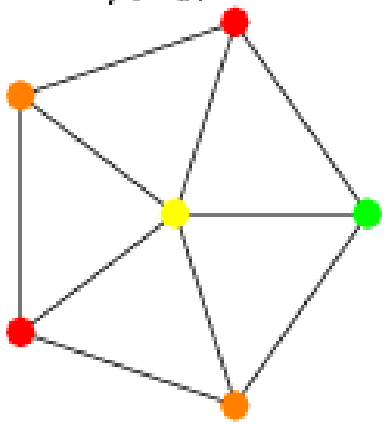
$\gamma(S_6) = 2$

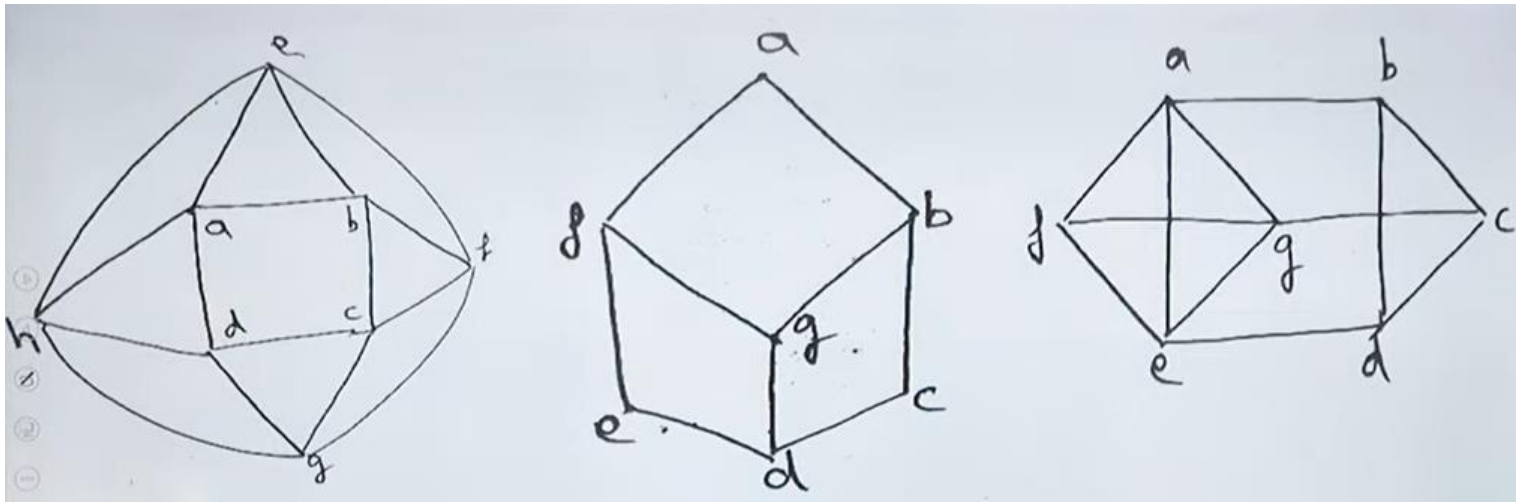


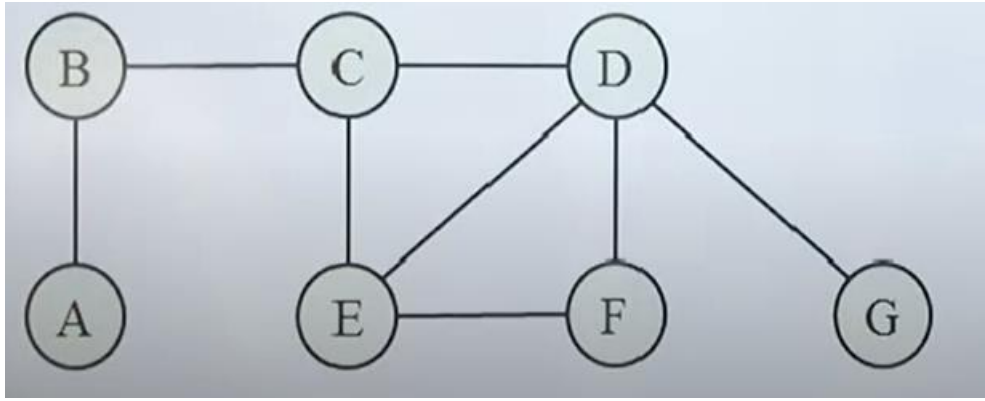
$\gamma(W_5) = 3$

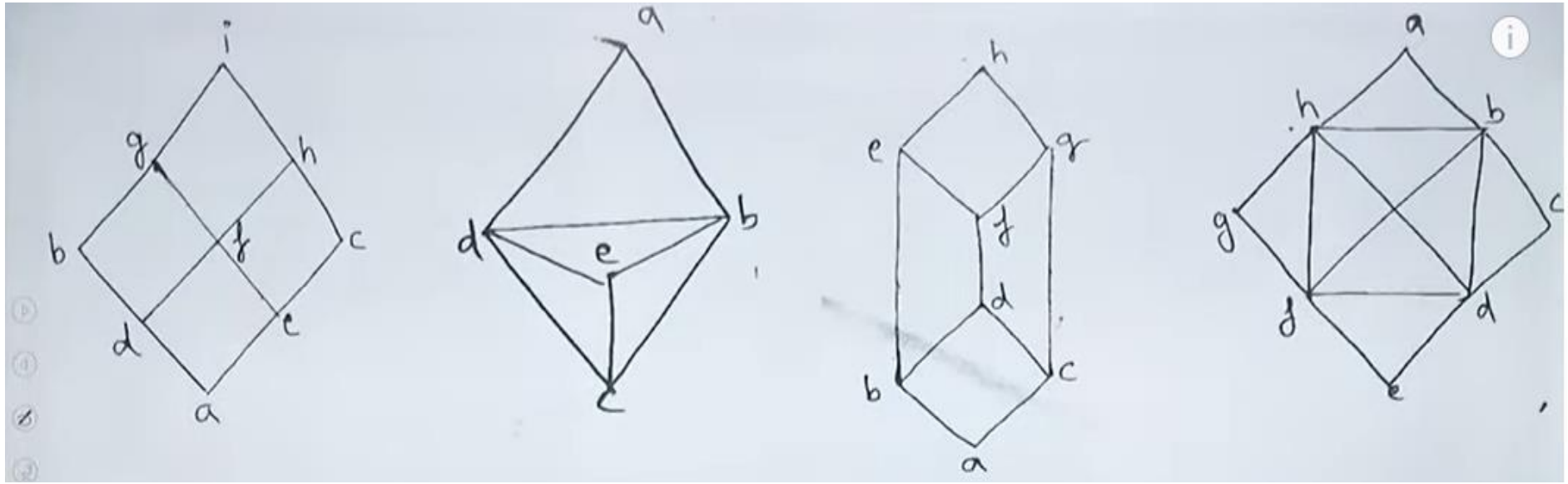


$\gamma(W_6) = 4$



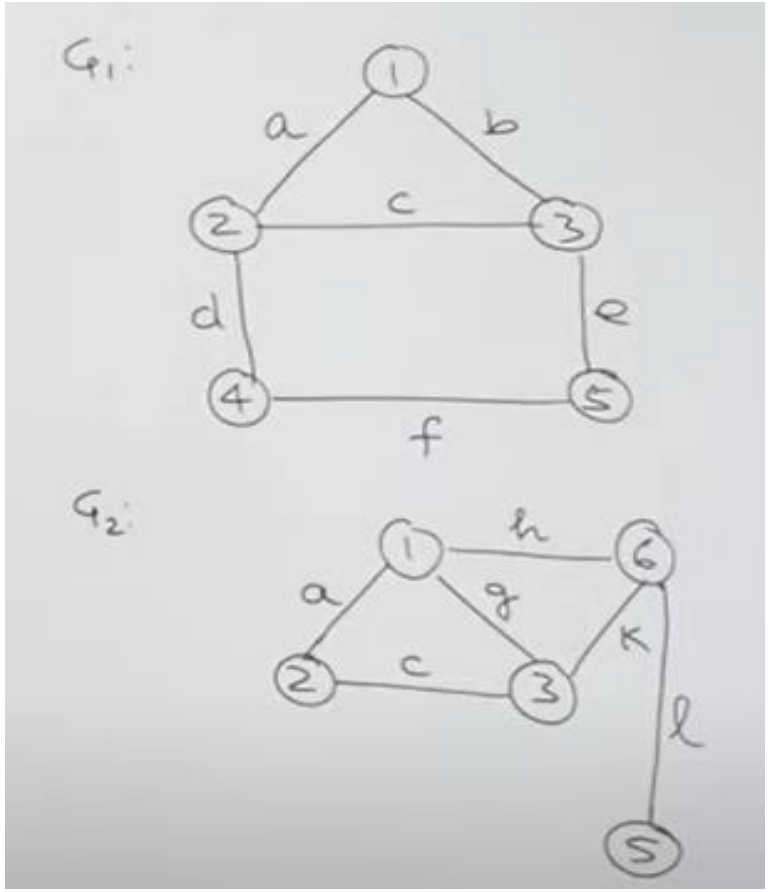

e
a
b
f
h
d
c
g
a
b
g
g
e
d
c
a
b
f
g
c
e
d


B
C
D
A
E
F
G

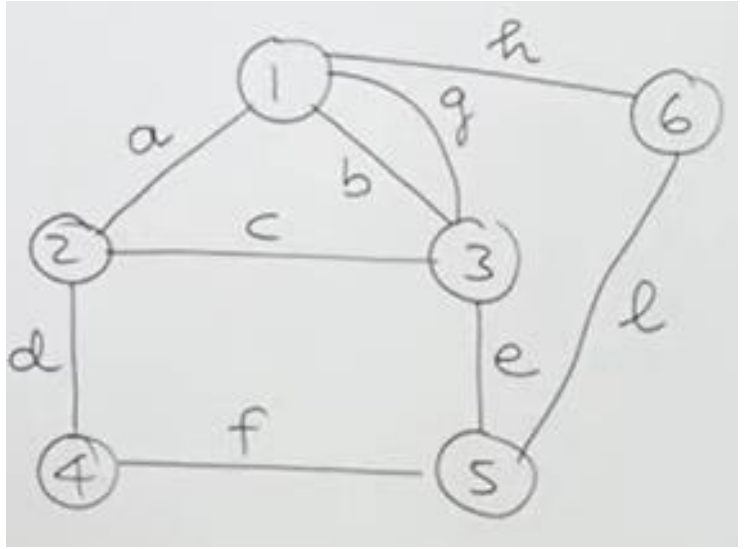

i
g
h
b
f
c
d
e
a
a
d
b
e
c
h
e
g
f
d
b
c
a
a
h
b
g
c
f
d
e

Operations on Graph

Union: If G1(V1,E1) and G2(V2,E2) are two graphs then union G3=G1UG2,whose vertex set is V3=V1UV2 and edge set E3 = E1UE2.

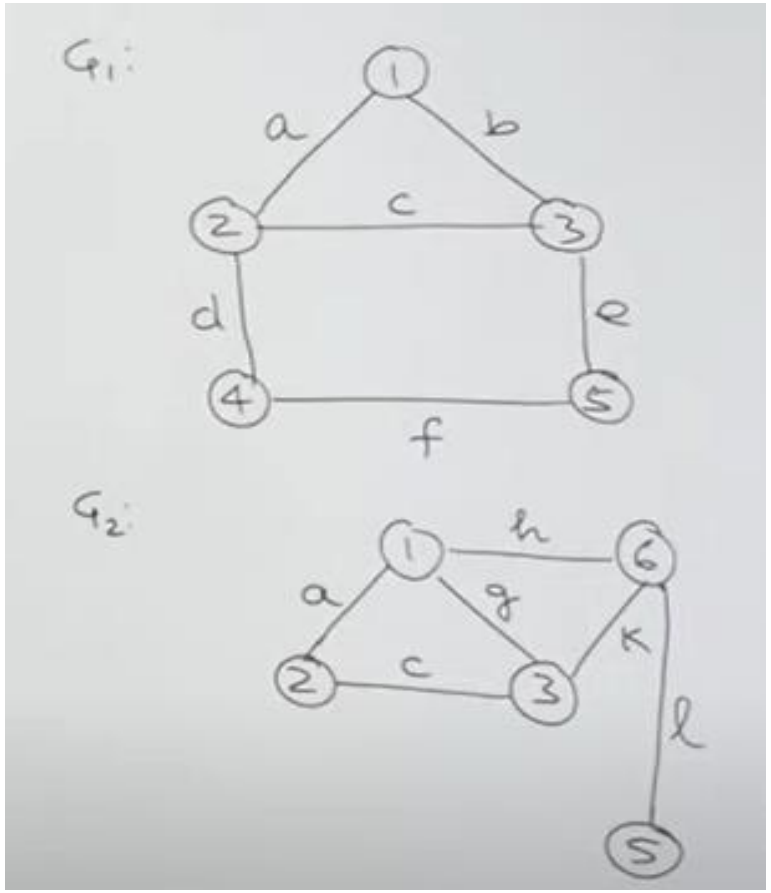


Union of G1 and G2

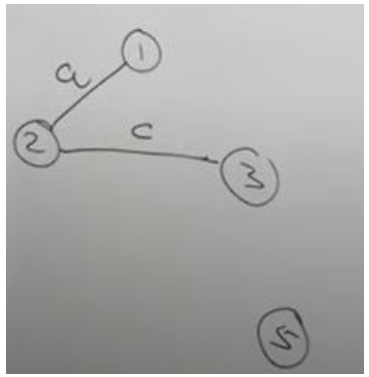


Operations on Graph

Intersection: If G1(V1,E1) and G2(V2,E2) are two graphs then intersection G3=G1∏G2,whose vertex set is V3= V1 ∏ V2 and edge set E3 = E1 ∏ E2.

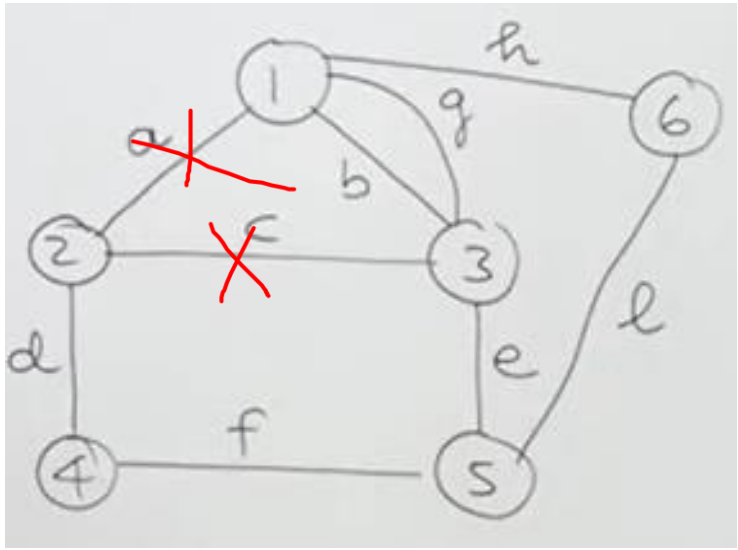


G1 Intersection G2



Operations on Graph

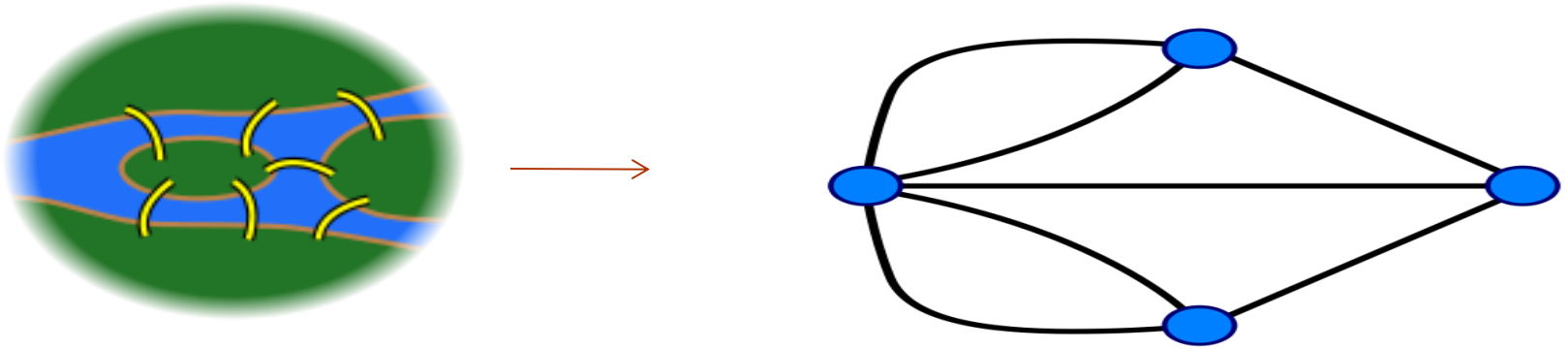
Ring sum: If G1(V1,E1) and G2(V2,E2) are two graphs then ringsum G3=G1⊕G2,whose vertex set is V3=V1 ⊕ V2 and edge set E3 = E1 ⊕ E2.



Application of graph Theory

✓ *Konigsberg Bridge Problem:*

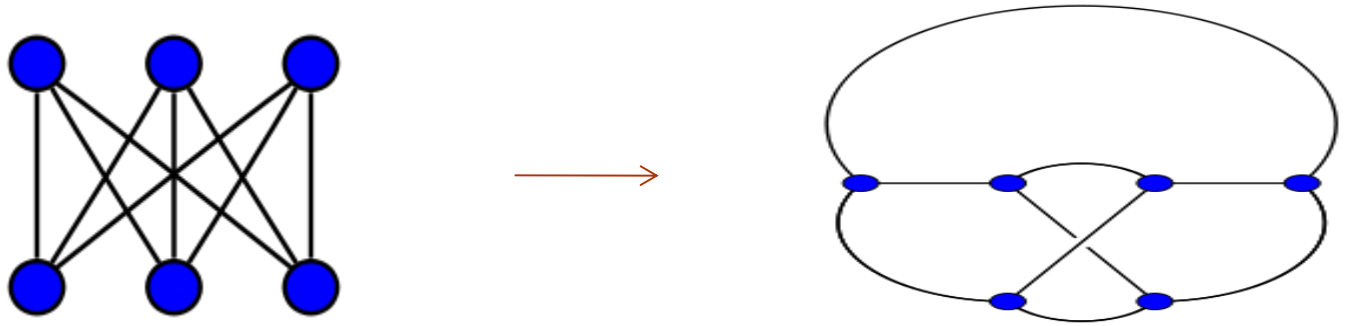
Start from any four land area walkover the seven bridges exactly once.



Vertices represent the land area and edges represent the bridge.

Euler prove that solution of this problem does not exist.

✓*Utility problem:* Three houses H1 ,H2,H3 & three utilities W , G & E.Is it possible to provide such a connection without any crossover.



✓This graph can't be drawn on a plane without crossing the edges.

Graph Theory

Q1: Which of the following statements is/are TRUE for undirected graphs? [GATE 2013]

P: Number of odd degree vertices is even.

Q: Sum of degrees of all vertices is even.

A. P only

B. Q only

C. Both P and Q

D. Neither P nor Q

Graph Theory

Q1: Which of the following statements is/are TRUE for undirected graphs? [GATE 2013]

P: Number of odd degree vertices is even.

Q: Sum of degrees of all vertices is even.

A. P only

B. Q only

C. Both P and Q

D. Neither P nor Q

Hint: Handshaking Lemma:

In any undirected graph, the sum of the degrees of all vertices equals $2 \mid E \mid$.

Hence, the sum of degrees is always even.

Also, the number of odd-degree vertices must be even.

Answer: C

Q2: Let G be a simple undirected planar graph on 10 vertices with 15 edges. If G is a connected graph, then the number of bounded faces in any embedding of G on the plane is equal to [GATE2012]

A. 3
B. 4
C. 5
D. 6

Q2: Let G be a simple undirected planar graph on 10 vertices with 15 edges. If G is a connected graph, then the number of bounded faces in any embedding of G on the plane is equal to [GATE2012]

A. 3
B. 4
C. 5
D. 6

Hint: Use Euler's formula for connected planar graphs:
$V - E + F = 2$.

Answer: D

Q3: What is the chromatic number of an n-vertex simple connected graph which does not contain any odd length cycle? Assume n ≥ 2[GATE 2009]

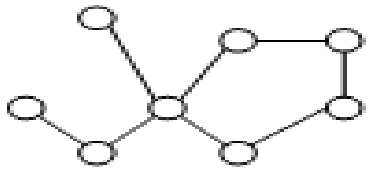
A.2
B.3
C.n-1
D. n

Q3: What is the chromatic number of an n-vertex simple connected graph which does not contain any odd length cycle? Assume n ≥ 2[GATE 2009]

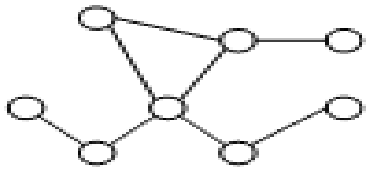
A.2
B.3
C.n-1
D. n

Answer : A

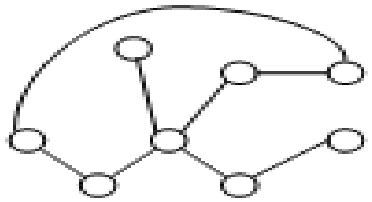
Q4. Which of the following graph is isomorphic to [GATE 2012]



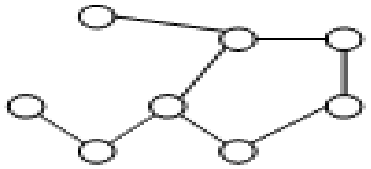
(A)



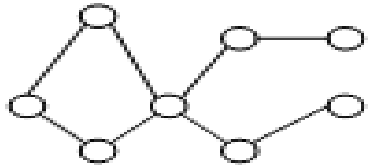
(B)



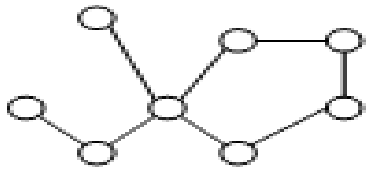
(C)



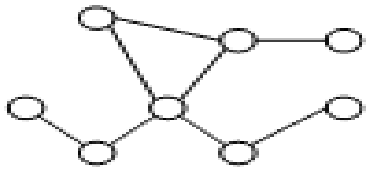
(D)



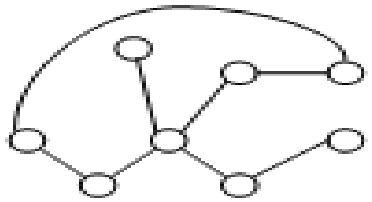
Q4. Which of the following graph is isomorphic to [GATE 2012]



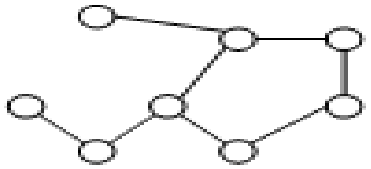
(A)



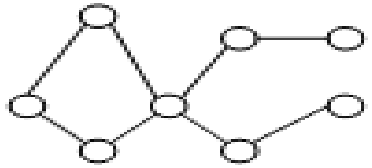
(B)



(C)



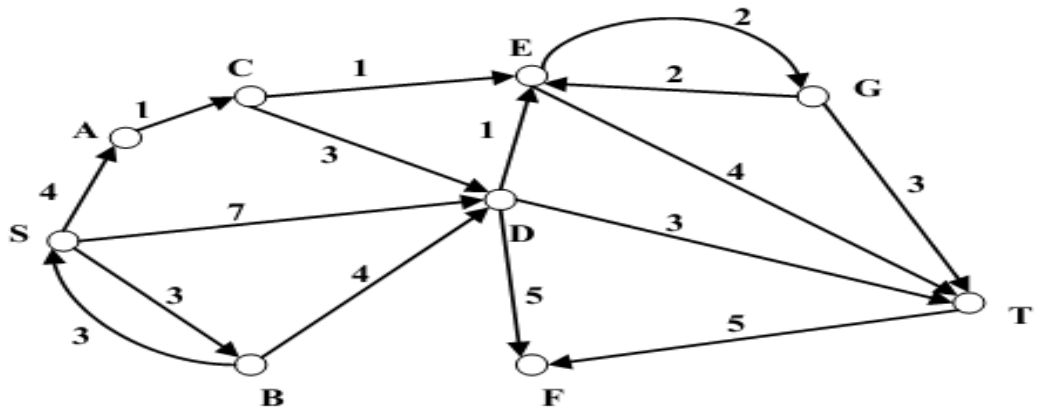
(D)



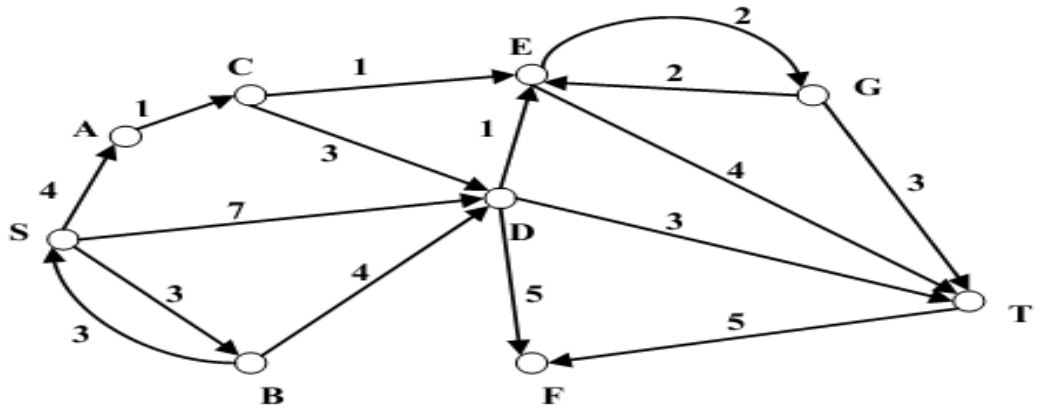
Answer : B

Q5. Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices S and T. Which one will be reported by Dijkstra's shortest path algorithm? Assume that, in any iteration, the shortest path to a vertex v is updated only when a strictly shorter path to v is discovered. [GATE 2012]

Ans: A.SDT
B.SBDT
C.SACDT
D.SACET



Q5. Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices S and T. Which one will be reported by Dijkstra's shortest path algorithm? Assume that, in any iteration, the shortest path to a vertex v is updated only when a strictly shorter path to v is discovered. [GATE 2012]



Ans: A.SDT
B.SBDT
C.SACDT
D.SACET

Answer : D

Q6. Which one of the following is TRUE for any simple connected undirected graph with more than 2 vertices? [GATE 2009]

Ans: A. No two vertices have the same degree.

B. At least two vertices have the same degree.

C. At least three vertices have the same degree.

D. All vertices have the same degree.

Q6. Which one of the following is TRUE for any simple connected undirected graph with more than 2 vertices? [GATE 2009]

Ans: A. No two vertices have the same degree.

B. At least two vertices have the same degree.

C. At least three vertices have the same degree.

D. All vertices have the same degree.

Answer: B

Q7. A graph with one vertex and no edges is

A. Multi graph
B. Digraph
C. Isolated graph
D. Trivial graph

Q8. Maximum number of edges in a n-Node undirected graph without self loop is

A. n2
B. $n(n-1)$
C. $n(n+1)$
D. $n(n-1)/2$

Q7. A graph with one vertex and no edges is

A. Multi graph
B. Digraph
C. Isolated graph
D. Trivial graph

Answer: D

Hint: A graph with **one vertex and no edges** is called a **Trivial graph**

Q8. Maximum number of edges in a n-Node undirected graph without self loop is

A. n2
B. $n(n-1)$
C. $n(n+1)$
D. $n(n-1)/2$

Answer: D

Q9. A path in which every edge of the graph appears exactly once, is called

A. Hamiltonian path
B. Eulerian path
C. Hamiltonian circuit
D. None of these

Q10. A path in which every vertex of the graph appears exactly once is called as

A. Eulerian path
B. Hamiltonian path
C. Eulerian circuit
D. None of these

Q9. A path in which every edge of the graph appears exactly once, is called

A. Hamiltonian path
B. Eulerian path
C. Hamiltonian circuit
D. None of these

Answer: B

Q10. A path in which every vertex of the graph appears exactly once is called as

A. Eulerian path
B. Hamiltonian path
C. Eulerian circuit
D. None of these

Answer: B

Q11. The complete bipartite graph K2,3 has

A. Eulerian circuit
B. Hamiltonian circuit
C. Both Eulerian circuit and Hamiltonian circuit
D. Neither Eulerian circuit nor Hamiltonian circuit

Q12. A graph which can be drawn on the plane such that no edges cross each other, is called

A. Planar graph
B. Complete graph
C. Complete bipartite graph
D. None of these

Q11. The complete bipartite graph K2,3 has

A. Eulerian circuit

B. Hamiltonian circuit

C. Both Eulerian circuit and Hamiltonian circuit

D. Neither Eulerian circuit nor Hamiltonian circuit

Answer: D

Q12. A graph which can be drawn on the plane such that no edges cross each other, is called

A. Planar graph

B. Complete graph

C. Complete bipartite graph

D. None of these

Answer: A

Q13. For any connected planar graph, if v, e and r are total number of vertices, edges and regions in the graph respectively, then

A. $v - e + r = 2$
B. $v + e + r = 2$
C. $v - e + r = 1$
D. $v + e + r = 1$

Q14. The minimum number of colors needed to produce a proper coloring of a simple connected graph G is called

A. Chromatic number of G
B. Edge connectivity of G
C. Vertex connectivity of G
D. None of these

Q13. For any connected planar graph, if v, e and r are total number of vertices, edges and regions in the graph respectively, then

A. $v - e + r = 2$
B. $v + e + r = 2$
C. $v - e + r = 1$
D. $v + e + r = 1$

Answer: A

Q14. The minimum number of colors needed to produce a proper coloring of a simple connected graph G is called

A. Chromatic number of G
B. Edge connectivity of G
C. Vertex connectivity of G
D. None of these

Answer: A

Q15. The chromatic number of complete bipartite graph Km,n is

Ans. A. m

B. n

C. 2

D. 1

Q15. The chromatic number of complete bipartite graph Km,n is

Ans. A. m

B. n

C. 2

D. 1

Answer: C

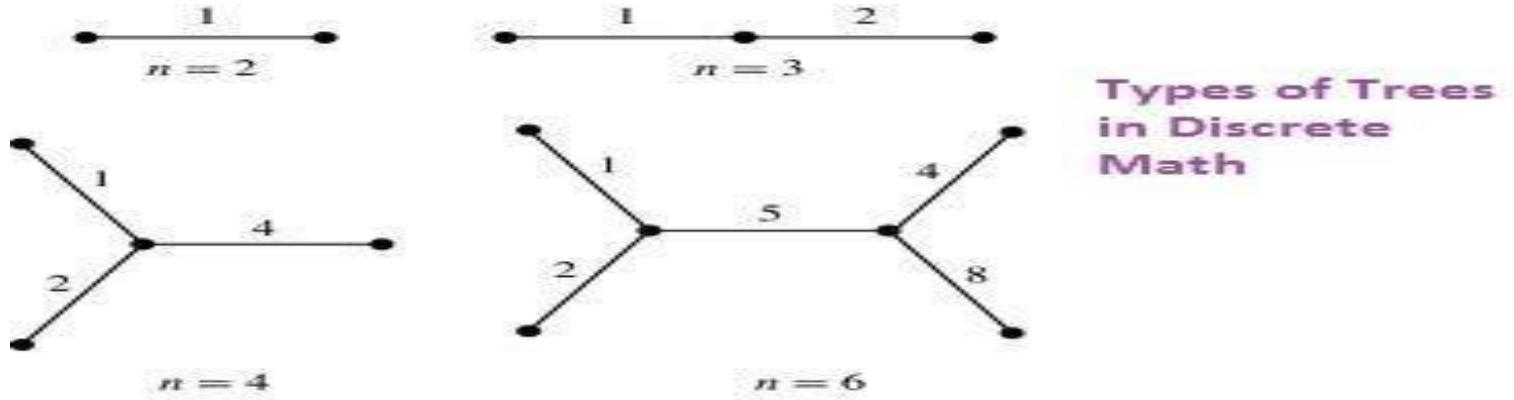
Module -III

Module III :Graph Theory and Applications (10 Hrs)

- **Trees:** Trees Introduction
- Properties of trees(1)
- Binary tree
- Binary search tree, tree traversal, Expression tree(2)
- Huffman coding(1)
- Minimum Spanning Tree
- Kruskal's and Prim's algorithms(2)

Trees Introduction

- A Tree is a connected undirected graph with no simple circuits.
- Because a tree cannot have a simple circuit, a tree cannot contain multiple edges or loops. Therefore any tree must be a simple graph.



Types of Trees in Discrete Math

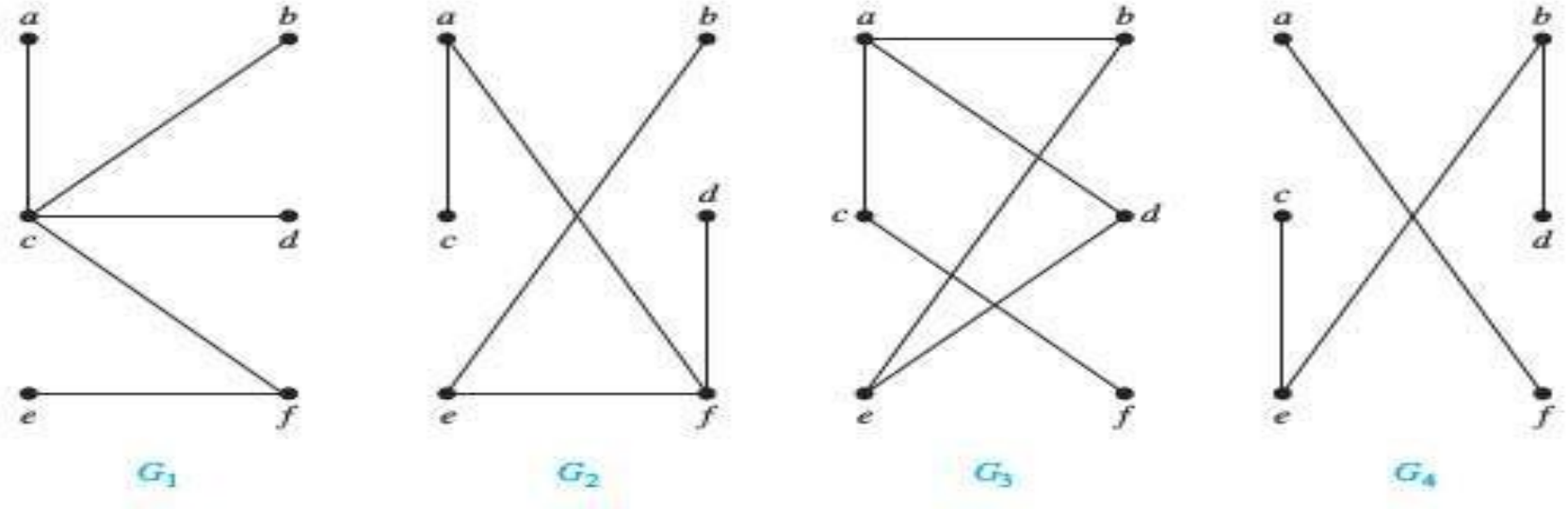
- **Theorem 1:** An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.
- **Theorem 2:** Every tree with n vertices has exactly n - 1 edges.

❖ **Example 1:** Which of the graphs shown in Figure are trees?

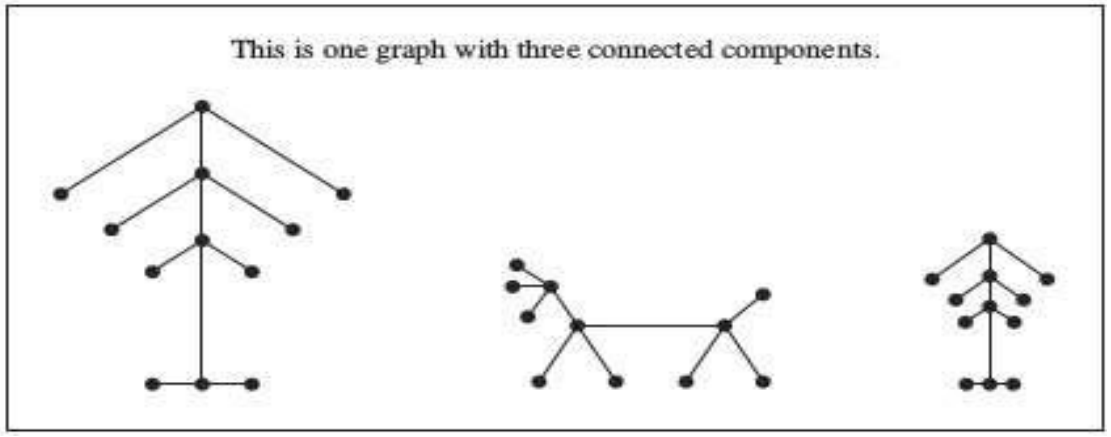
Solution: G1 and G2 are trees, because both are connected graphs with no simple circuits.

G3 is not a tree because e, b, a, d, e is a simple circuit in this graph.

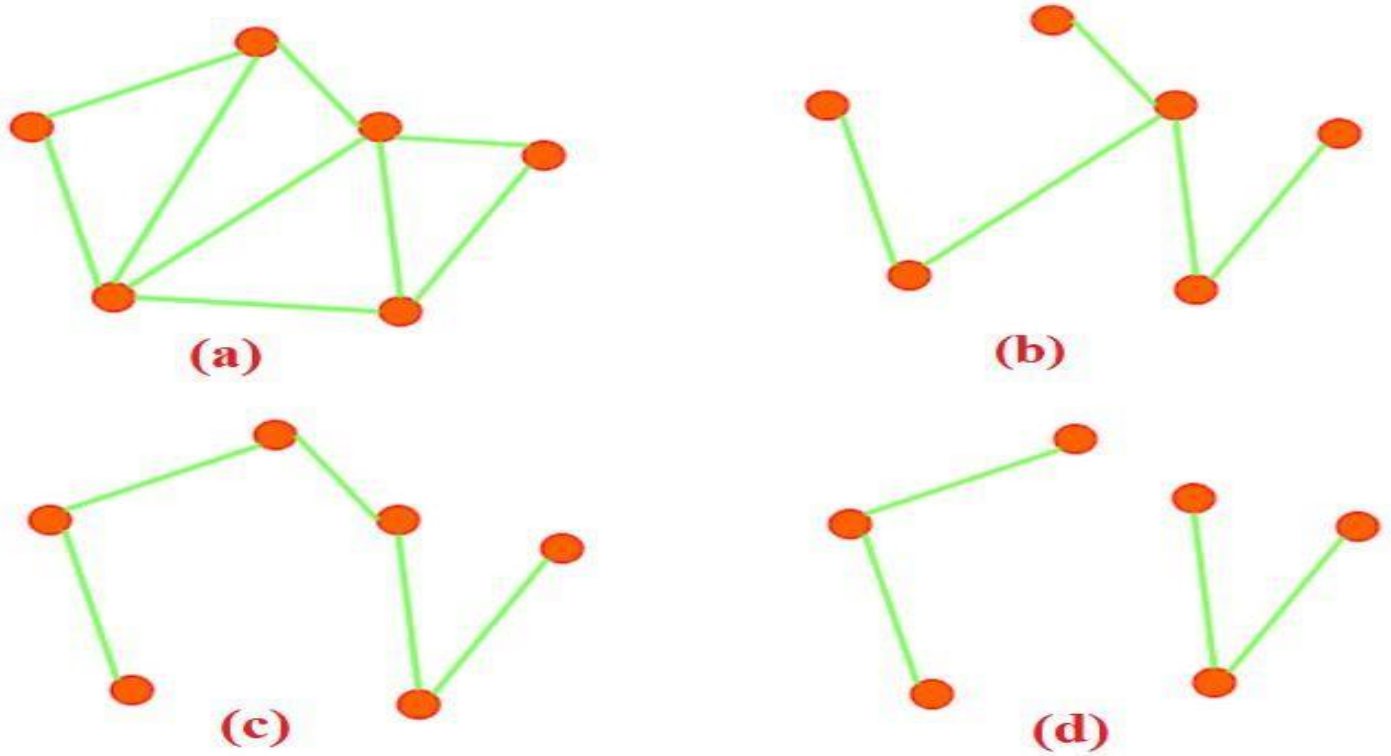
Finally, G4 is not a tree because it is not connected



- Any connected graph that contains no simple circuits is a **tree**.
- **What about graphs containing no simple circuits that are not necessarily connected?.**
- These graphs are called **forests** and have the property that each of their connected components is a tree.
- A forest is composed of one tree or some disconnected trees. A forest is a disjoint union of trees.



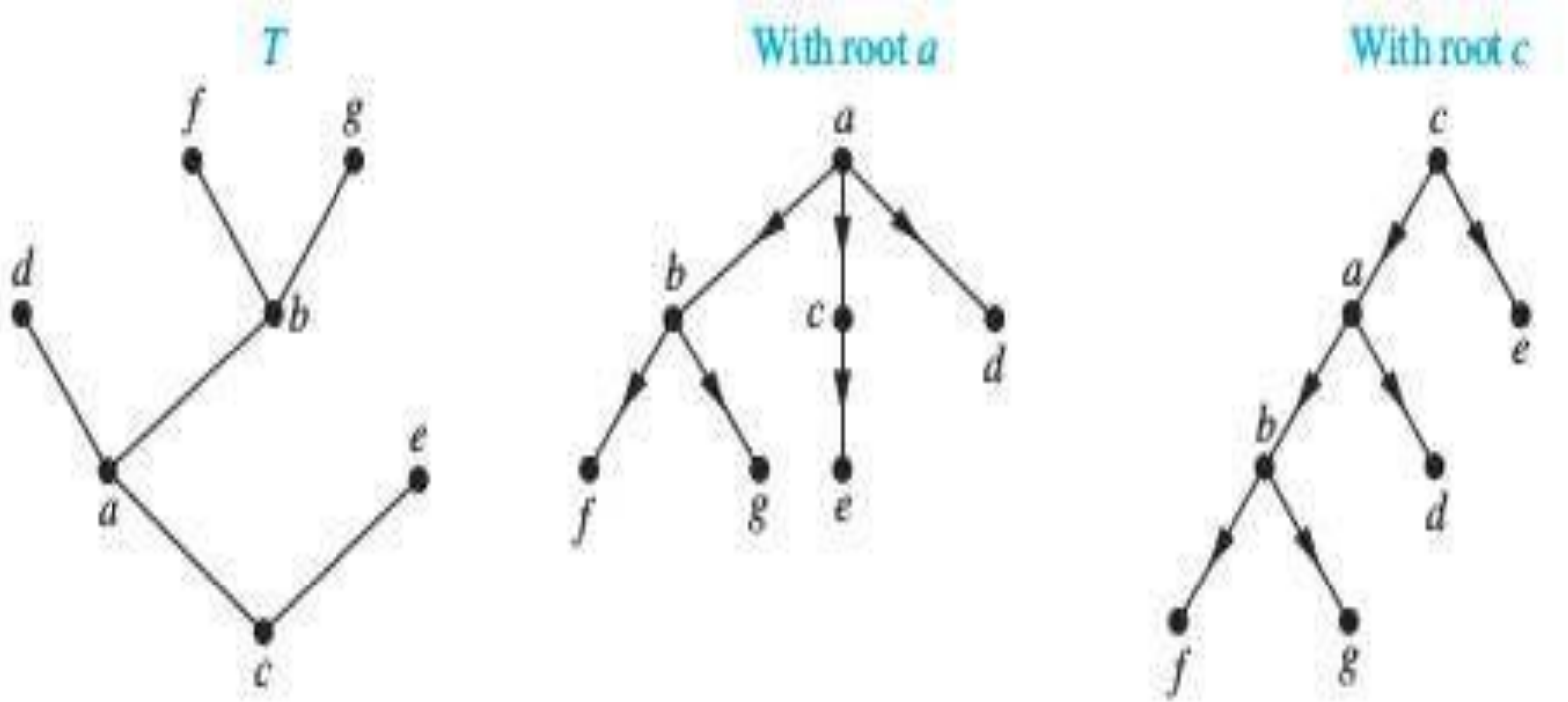
❖ **Example 2:** Are the following graphs trees?



Solution: Graph (a) and (d) are not tree while (b) & (c) tree

Rooted Trees

❖ A **rooted tree** is a tree in which one vertex has been designated as the root and every edge is directed away from the root.

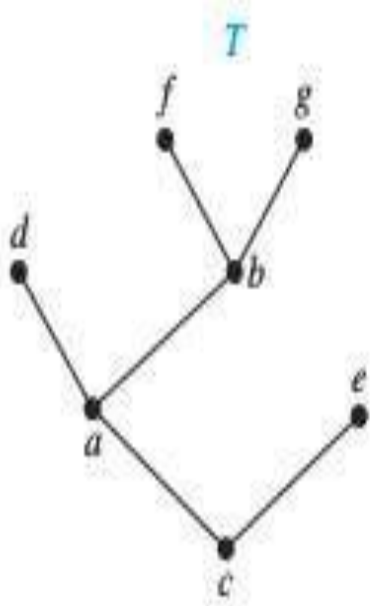


Terminology for rooted trees

- **Root** – The top node in a tree.
- **Parent** – The converse notion of *child*.
- **Siblings** – Nodes with the same parent.
- **Descendant** – a node reachable by repeated proceeding from parent to child.
- **Ancestor** – a node reachable by repeated proceeding from child to parent.
- **Leaf** – a node with no children.
- **Internal node** – a node with at least one child.
- **External node** – a node with no children.
- **Degree –** number of sub trees of a node.



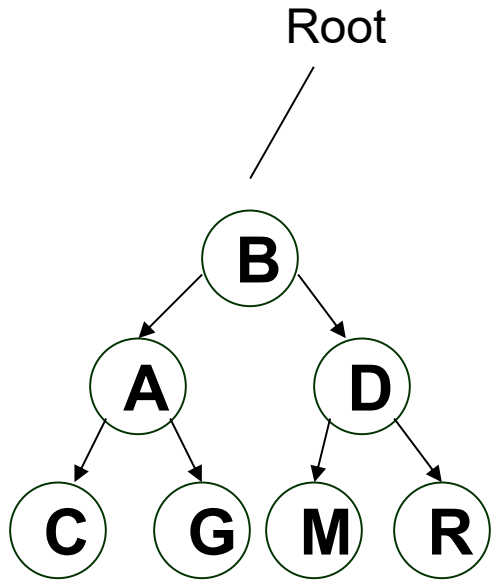
- The ancestors of a vertex other than the root are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root (i.e., its parent, its parent‟s parent, and so on, until the root is reached).
- The descendants of a vertex v are those vertices that have v as an ancestor.
- If a is a vertex in a tree, the subtree with a as its root is the subgraph of the tree consisting of a and its descendants and all edges incident to these descendants.



- **Edge** – connection between one node to another.
- **Path** – a sequence of nodes and edges connecting a node with a descendant.
- **Level** – The number of connections between the node and the root
- **Height of tree** –The height of a tree is the number of edges on the longest downward path between the root and a leaf.
- **Height of node** –The height of a node is the number of edges on the longest downward path between that node and a leaf.(Height of leaf is zero.)
- **Depth** –The depth of a node is the number of edges from the node to the tree's root node.(Depth of root is zero.)
- **Forest** – A forest is a set of $n \geq 0$ disjoint trees.

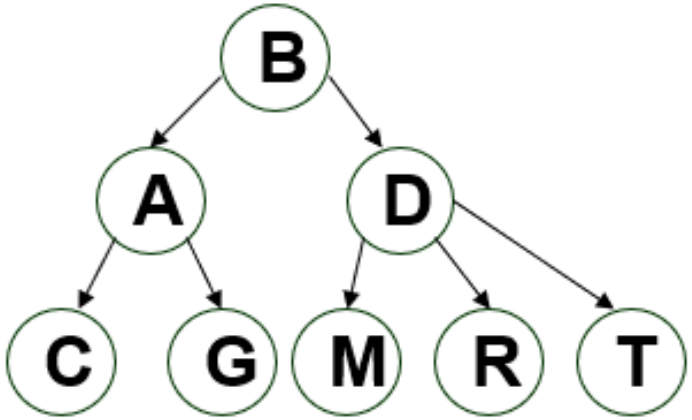
Tree Properties

- Tree can be a recursive data structure.
- It has one designated node, called the root, that has no parent.
- Every node, except the root, has exactly one parent.
- A node may have zero or more children.
- Unique directed path from the root .
- Tree with n vertices has n-1 edges

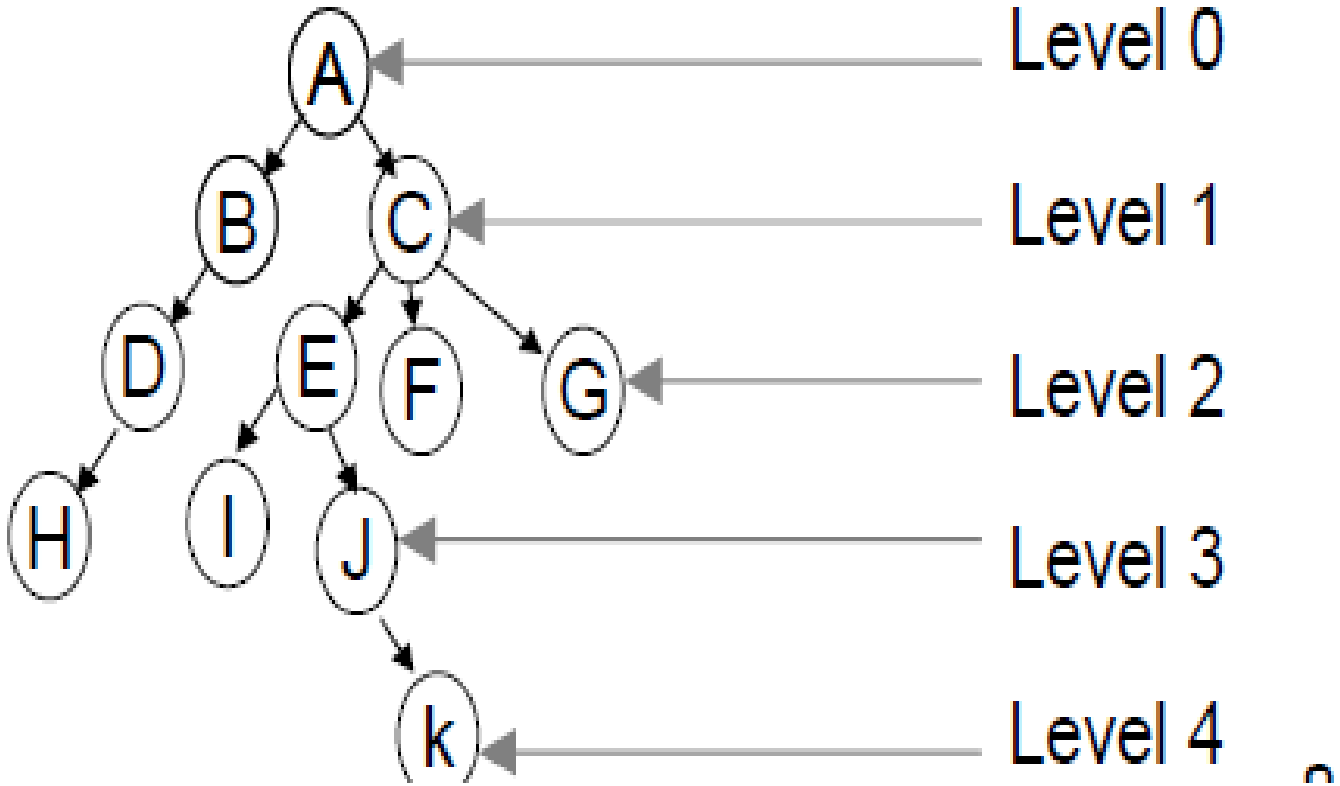


Basic Tree Concepts

- In-degree branch.
- Out-degree branch.
- Degree of the node.
- If the tree is not empty, the first node is called the root.



Basic Tree Concepts

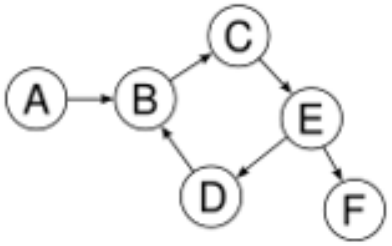


Tree as a Noncyclic Graph

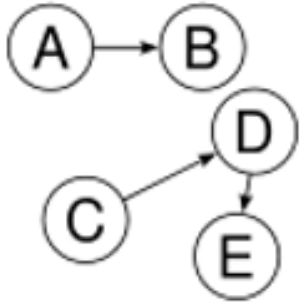
A tree is a (possibly non-linear) data structure made up of nodes or vertices and edges without having any cycle. The tree with no nodes is called the **null** or **empty** tree. A tree that is not empty consists of a root node and potentially many levels of additional nodes that form a hierarchy.



Not a tree:
cycle A→A



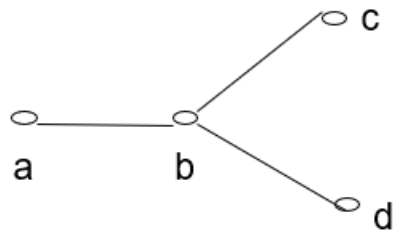
Not a tree: cycle
B→C→E→D→B



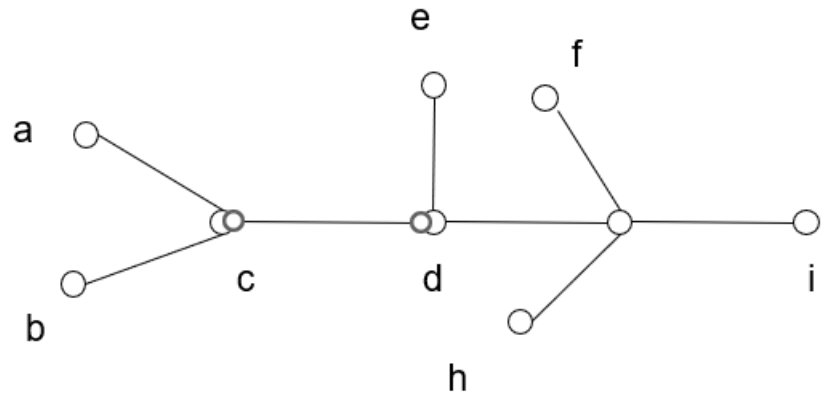
Not a tree: two non-connected parts, A→B and C→D→E

ECCENTRICITY OF VERTEX and Cut Point

E(v)=max d(vi,v)

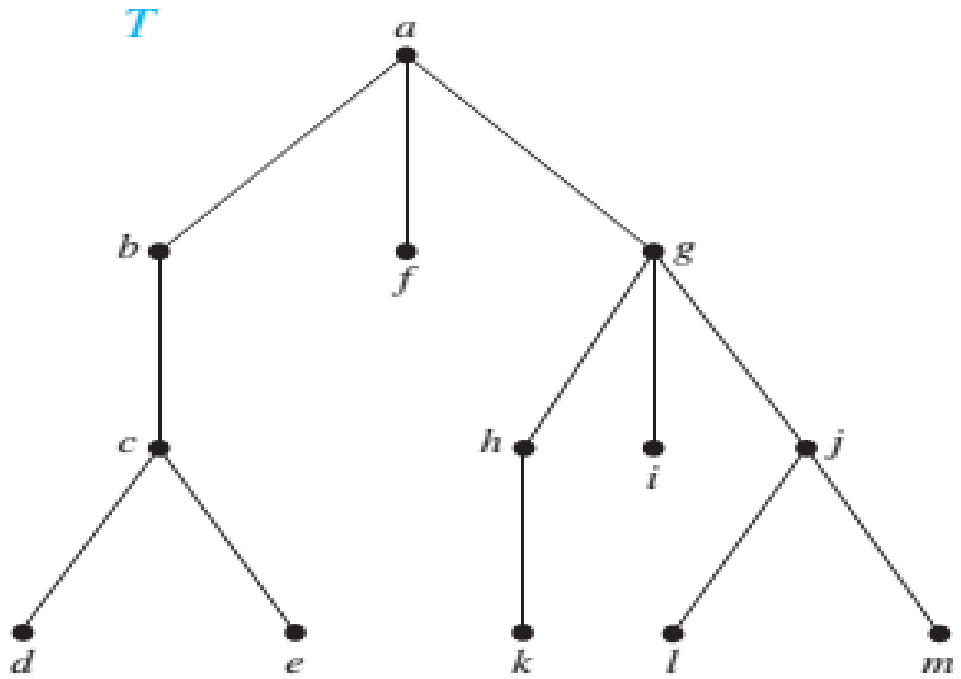


Vertex with minimum Eccentricity is called center of G.

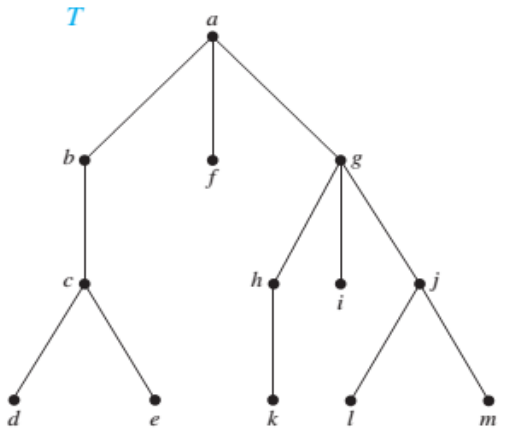


In any tree, all vertices except pendant vertices are cut vertices

❖ **Example 3:** In the rooted tree T(with root a) shown in Figure , find the parent of c, the children of g, the siblings of h, all ancestors of e, all descendants of b, all internal vertices, and all leaves. What is the subtree rooted at g?

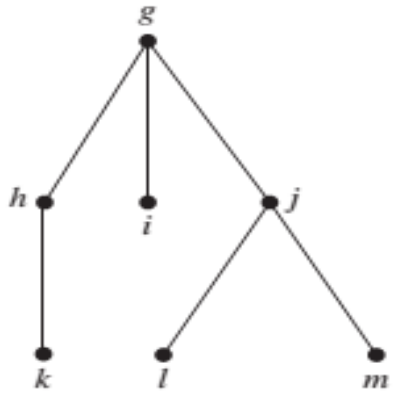


❖ **Example 3:** In the rooted tree T(with root a) shown in Figure , find the parent of c, the children of g, the siblings of h, all ancestors of e, all descendants of b, all internal vertices, and all leaves. What is the subtree rooted at g?



Solution: The parent of c is b. The children of g are h, i, and j. The siblings of h are i and j.

The ancestors of e are c, b, & a. The descendants of b are c, d, & e.



The internal vertices are a, b, c, g, h, & j. The leaves are d, e, f, i, k, l, and m. The subtree rooted at g is shown in Figure.

❖ **Example 4:** Answer these questions about the rooted tree illustrated in Figure.

a) Which vertex is the root?

b) Which vertices are internal?

c) Which vertices are children of j?

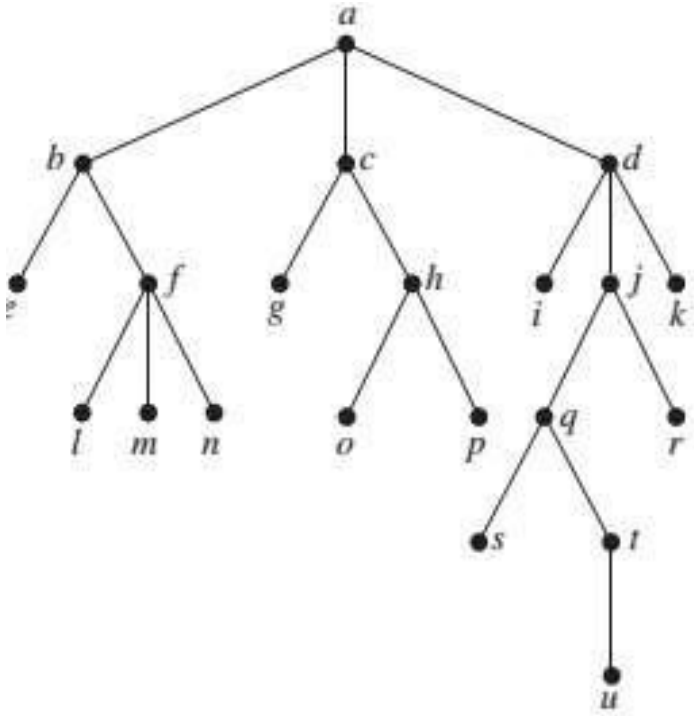
d) Which vertex is the parent of h?

e) Which vertices are siblings of o?

f) Which vertices are ancestors of m?

g) Which vertices are descendants of b?

h) Which vertices are leaves?



❖ **Example 4:** Answer these questions about the rooted tree illustrated in Figure.

a) Which vertex is the root? Vertex a

b) Which vertices are internal? a, b, c, d, f, h , j , q, & t.

c) Which vertices are children of j? q and r

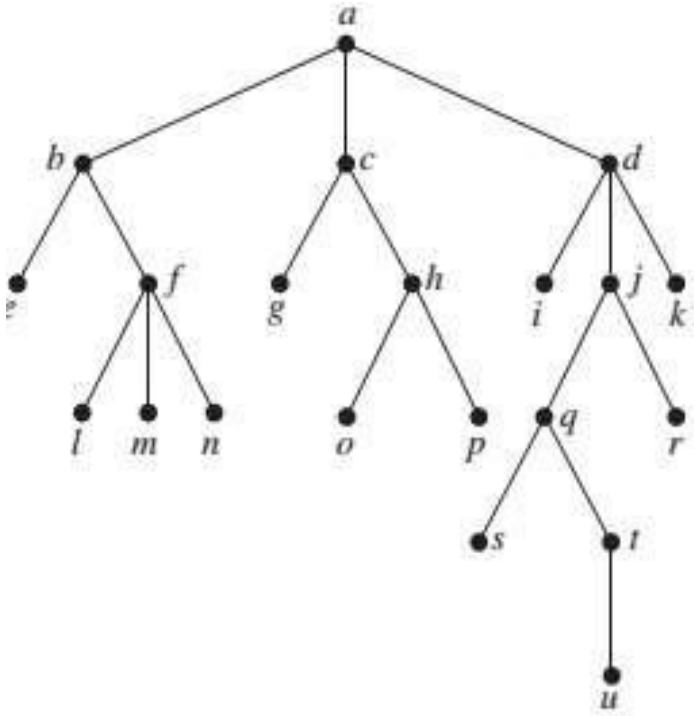
d) Which vertex is the parent of h? c

e) Which vertices are siblings of o? p

f) Which vertices are ancestors of m? f, b, and a

g) Which vertices are descendants of b? e, f, l, m, and n

h) Which vertices are leaves? e, g, i, k, l, m, n, o, p, r, s, & u.



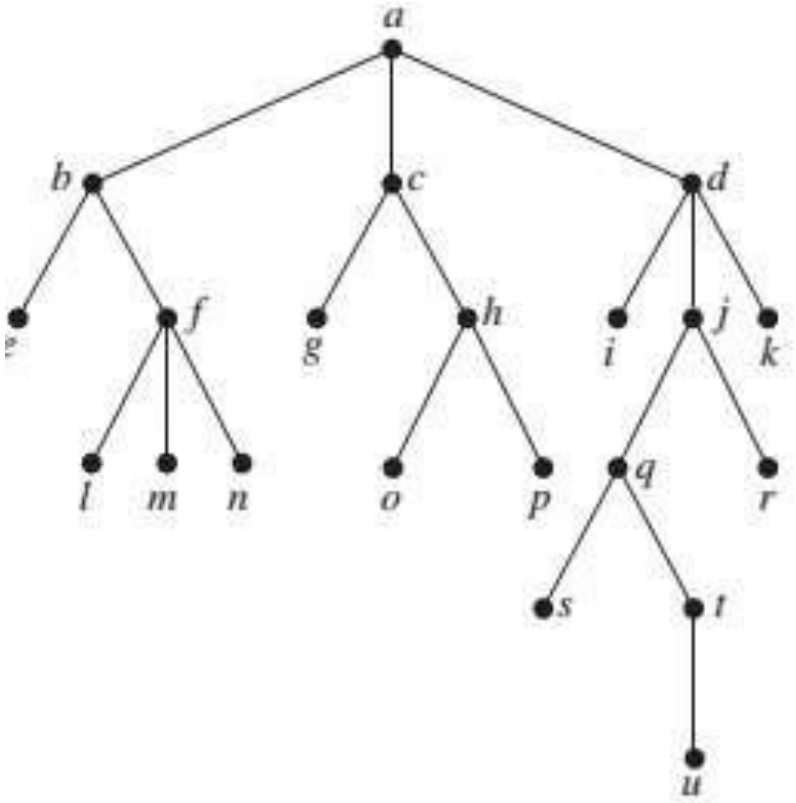
❖ **Example 5:** What is the level/length of each vertex of the rooted tree in Figure ?

Solution: We can easily determine the levels from the drawing.

The root a is at level 0.

The vertices in the row below a are at level 1, namely b, c, and d.

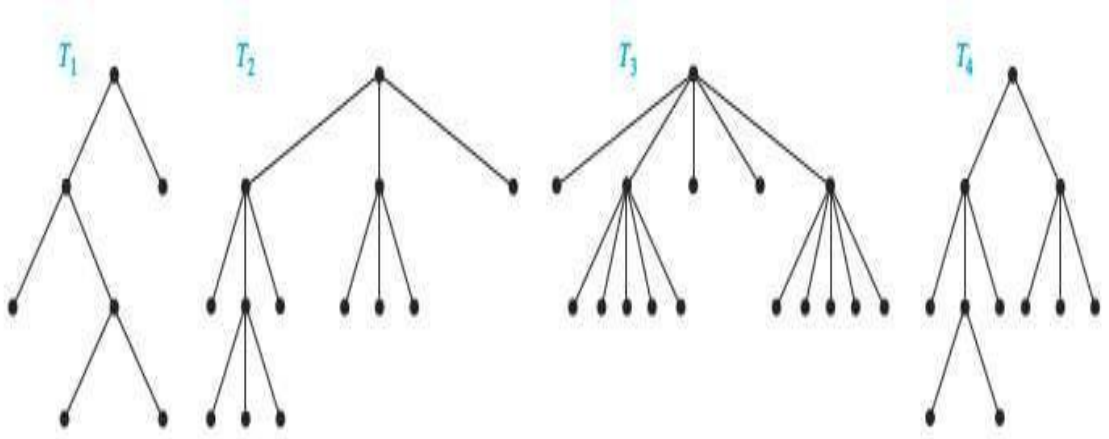
The vertices below that, namely e through k (in alphabetical order), are at level 2. Similarly l through r are at level 3, s and t are at level 4, and u is at level 5.



M-ary tree

- A rooted tree is called an m-ary tree if every internal vertex has no more than m children. The tree is called a full m-ary tree if every internal vertex has exactly m children. An m- ary tree with m = 2 is called a binary tree.
- **Example 6:** Are the rooted trees in Figure 8 full m-ary trees for some positive integer m?

Solution: T1 is a full binary tree because each of its internal vertices has two children. T2 is a full 3-ary tree because each of its internal vertices has three children. In T3 each internal vertex has five children, so T3 is a full 5-ary tree.T4 is not a full m-ary tree for any m because some of its internal vertices have two children and others have three children.



Properties of M-ary Trees

❖ **A full m-ary tree with i internal vertices contains n = mi + 1 vertices.**

❖ **Proof:** Every vertex, except the root, is the child of an internal vertex. Because each of the i internal vertices has m children, there are mi vertices in the tree other than the root. Therefore, the tree contains $n = mi + 1$ vertices.

❖ **A full m-ary tree with ::**

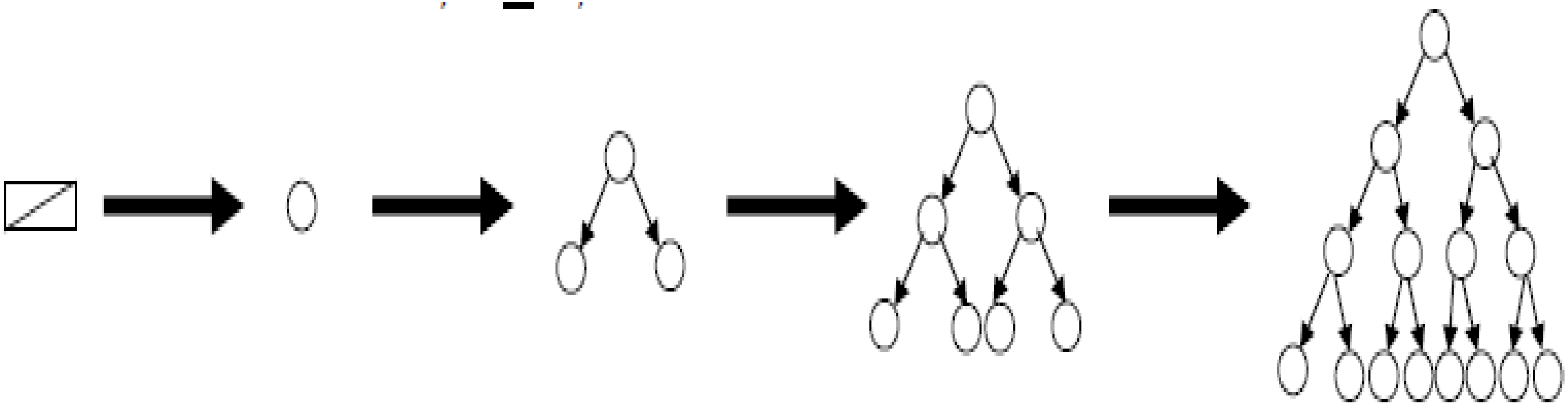
(i) n vertices has $i = (n-1)/m$ internal vertices and $l = [(m-1)n + 1]/m$ leaves,

(ii) i internal vertices has $n = mi + 1$ vertices and $l = (m-1)i + 1$ leaves,

(iii) l leaves has $n = (ml-1)/(m-1)$ vertices & $i = (l-1)/(m-1)$ internal vertices.

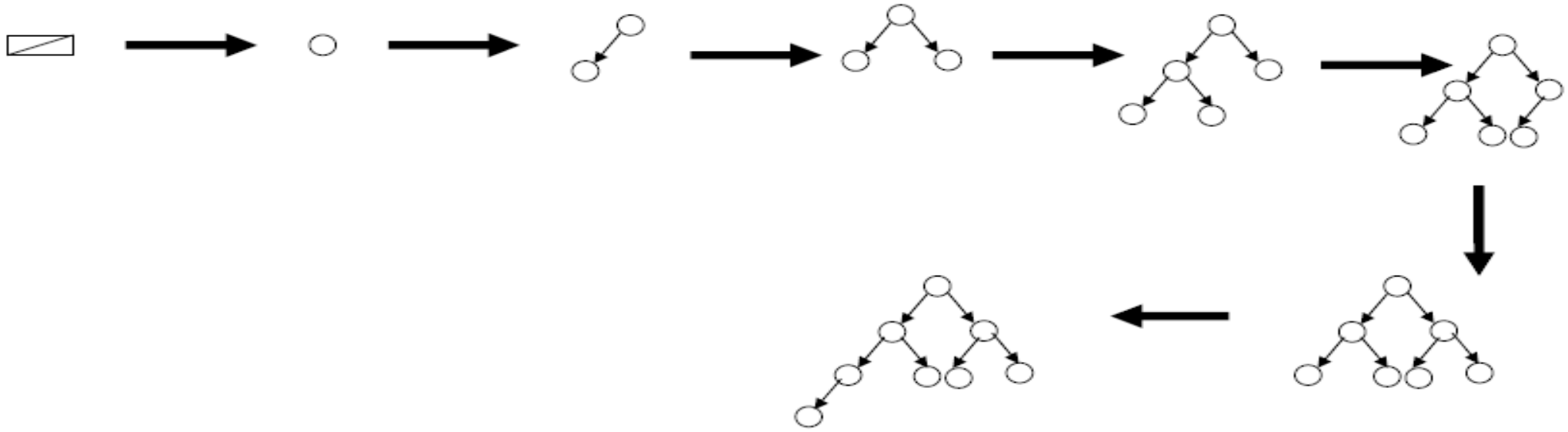
Full Binary tree

- A full binary tree is either an empty binary tree or a binary tree in which each level k, k >0, has 2^k nodes.



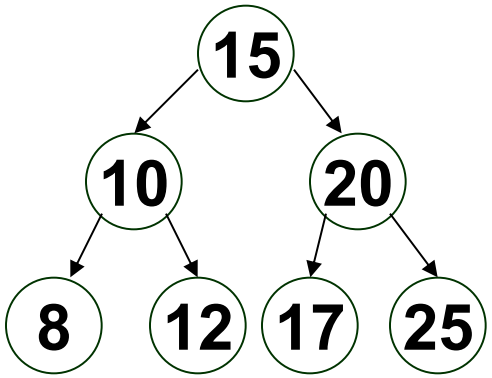
Complete Binary tree

- A complete binary tree is either an empty binary tree or a binary tree in which:
- Each level k, k >0, other than the last level contains the maximum number of nodes for that level, that is 2^k



Binary search tree

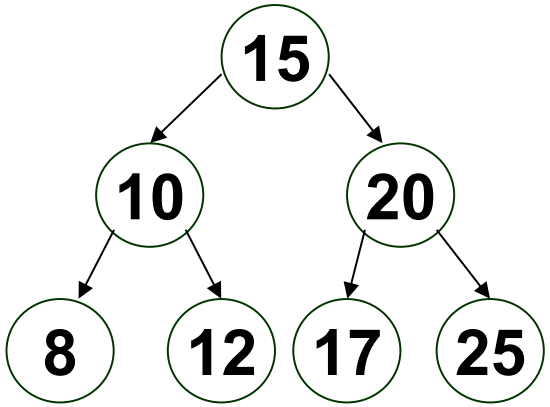
- A **Binary Search Tree** is a binary tree with the following properties:
- All items in the left sub-tree are less than the root.
- All items in the right sub-tree are greater or equal to the root.
- Each sub-tree is itself a binary search tree.



Insertion in Binary search tree

To insert node into BST the new node is inserted as a leaf in a tree.

- Compare the node value with the root node.
- If value > root node proceed to right child and it becomes root for right sub tee.
- If value < root node proceed to left child.
- Repeat above steps until we meet a node which has no left and right sub tree.
- Now if the item is greater than node then item is inserted as right child or if it is less than node then inserted as left child.
- E.g. insert node 19 in following BST.

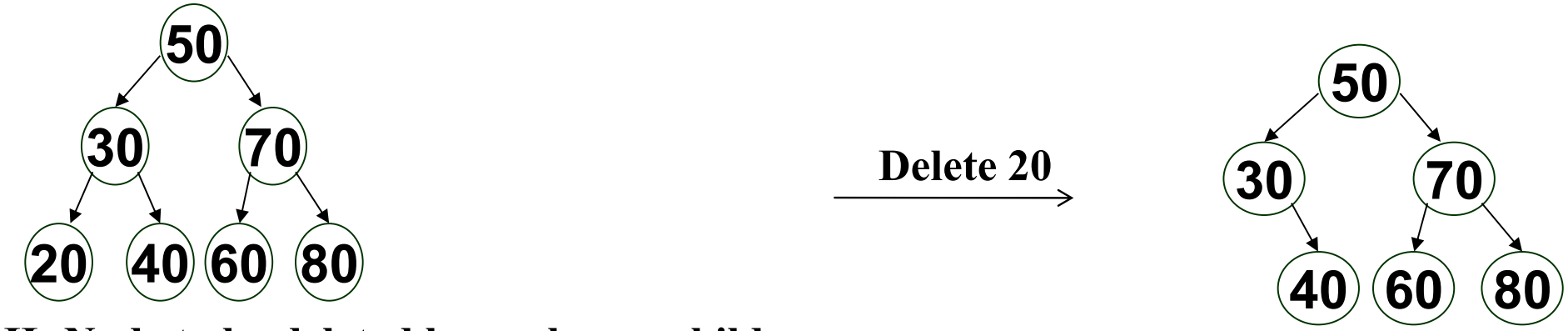


Deletion from Binary search tree

To delete a node from BST we need to consider three cases depending on no of children the node to be deleted has.

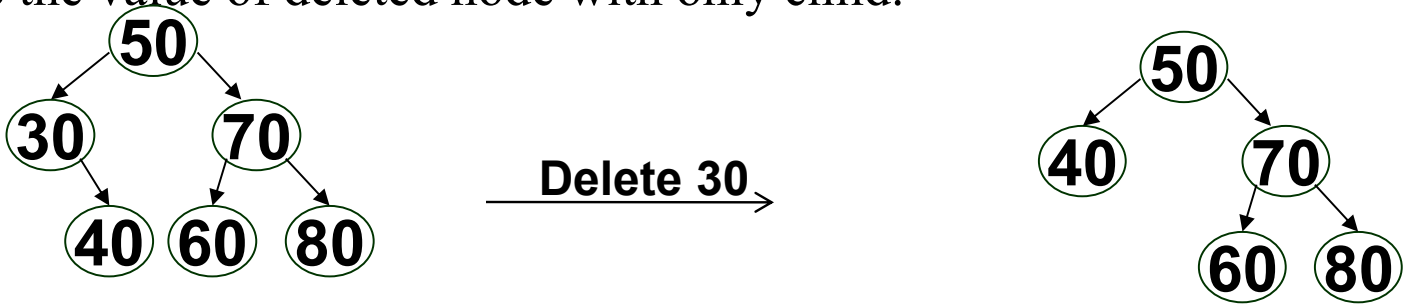
Case I: Node to be deleted has no child.

•This is very simple just replace the node with null.



Case II: Node to be deleted has only one child.

•Replace the value of deleted node with only child.

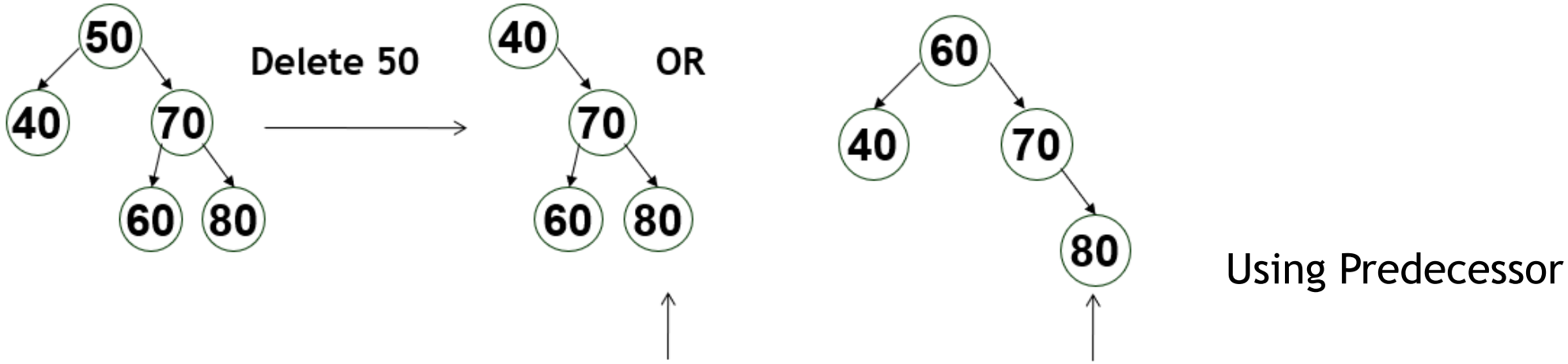


Deletion from Binary search tree

Case III: Node to be deleted has two children.

•In this case replace the deleted node with the node that is closest in the value to it. This node determined is called immediate predecessor or immediate successor. Now replace the value of deleted node with predecessor or successor and delete the replaced node by using case I or case II procedure.

Delete 50 **OR**



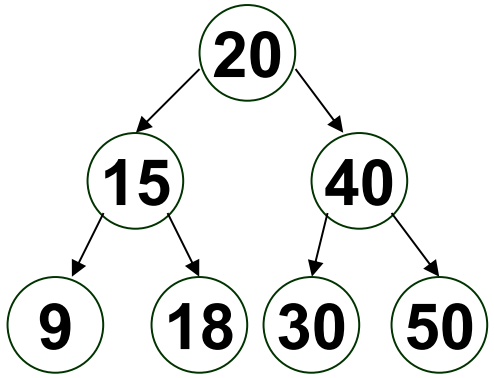
Using Predecessor

Using Successor

Binary tree traversal

TRAVERSAL OF A BINARY TREE IS TO ACCESS EVERY NODE OF BINARY TREE AT MOST ONCE.

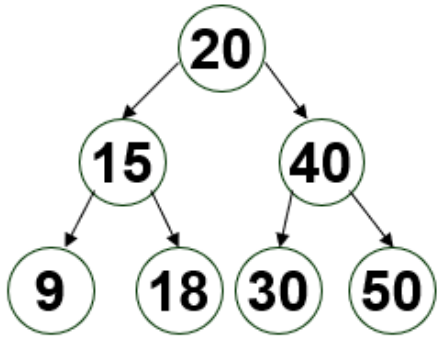
TRAVERSAL - PREORDER (ROOT-LEFT-RIGHT)



20 15 9 18 40 30 50

Binary tree traversal

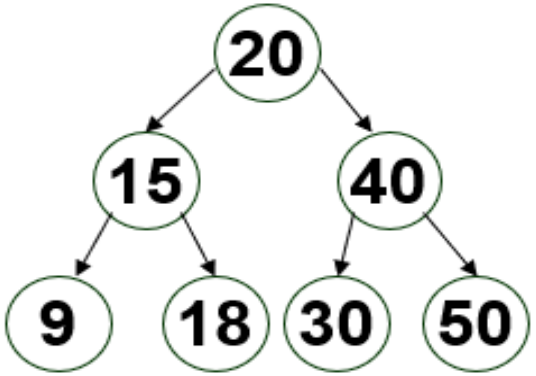
TRAVERSAL - POSTORDER (LEFT-RIGHT-ROOT)



9 18 15 30 50 40 20

Binary tree traversal

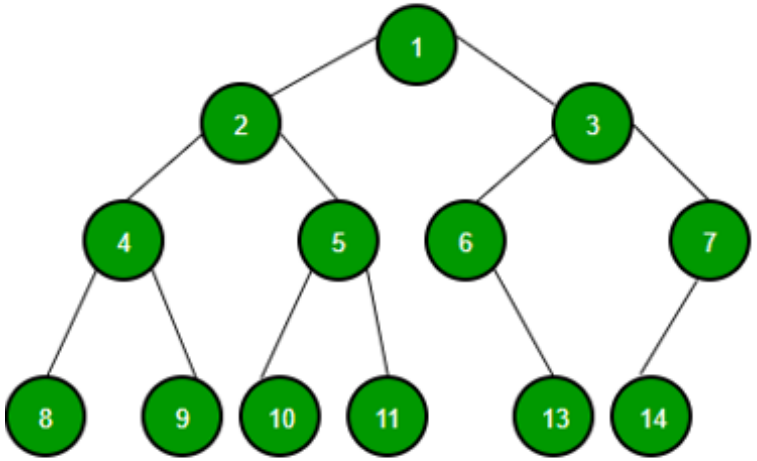
TRAVERSAL - INORDER (LEFT-ROOT-RIGHT)

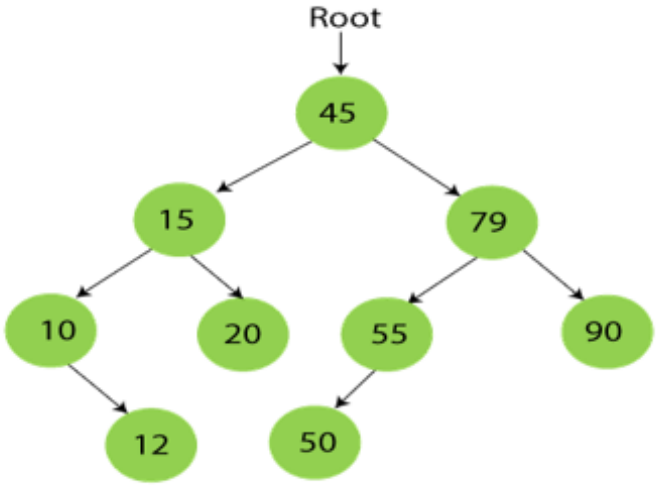


9 15 18 20 30 40 50

Exercise

Find Inorder, Preorder and Postorder Traversal of below tree

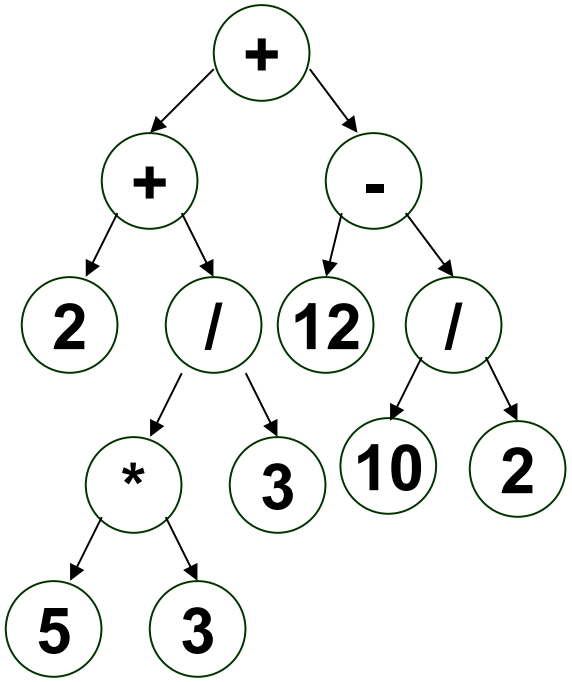




Expression Tree

Write and evaluate the expression tree shown below:

(2+(5*3)/3)+(12-(10/2))



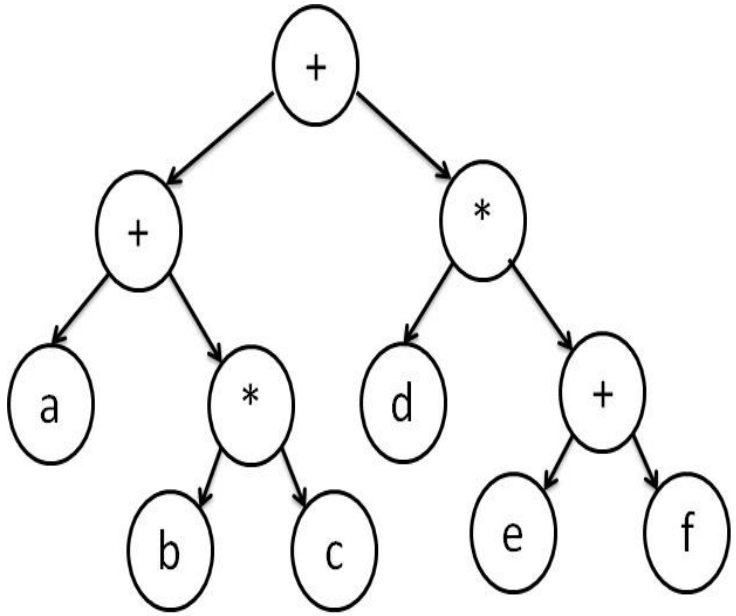
Exercise

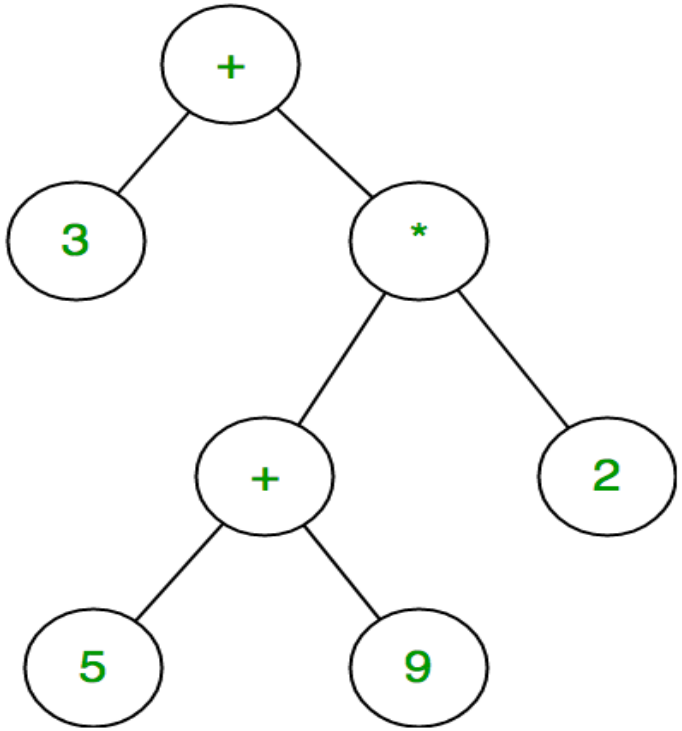
Create Expression tree using following expression

1) a + (b * c) + d * (e + f)

2) 3 + ((5+9)*2)

Solution





Prefix Code

- Consider the problem of using bit strings to encode the letters of the English alphabet (where no distinction is made between lowercase and uppercase letters). We can represent each letter with a bit string of length five, because there are only 26 letters and there are 32 bit strings of length five. The total number of bits used to encode data is five times the number of characters in the text when each character is encoded with five bits.
- Consider using bit strings of different lengths to encode letters. Letters that occur more frequently should be encoded using short bit strings, and longer bit strings should be used to encode rarely occurring letters. When letters are encoded using varying numbers of bits, some method must be used to determine where the bits for each character start and end.
- For instance, if e were encoded with 0,a with 1, and t with 01, then the bit string 0101 could correspond to *eat, tea, eaea, or tt.*

- One way to ensure that no bit string corresponds to more than one sequence of letters is to encode letters so that the bit string for a letter never occurs as the first part of the bit string for another letter. Codes with this property are called prefix codes.

- For instance, the encoding of e as 0, a as 10, and t as 11 is a prefix code.

- **A set of sequence is said to be prefix code if no sequence in the set is prefix of another.**

OR

- **A code is called a prefix (free) code if no codeword is a prefix of another one.**

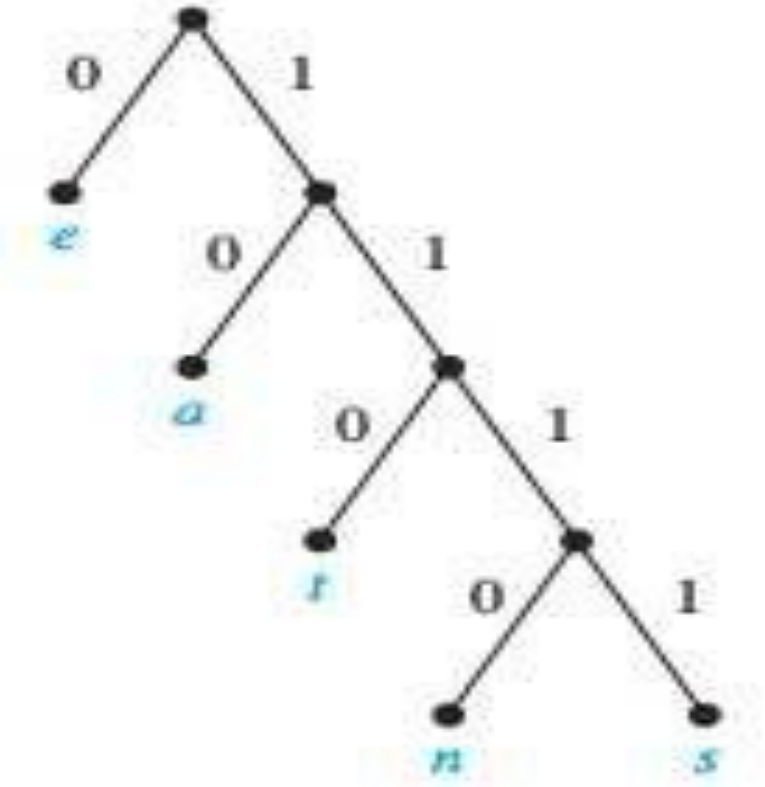
- A word can be recovered from the unique bit string that encodes its letters. For example, the string 10110 is the encoding of ate.

- To see this, note that the initial 1 does not represent a character, but 10 does represent a (and could not be the first part of the bit string of another letter). Then, the next 1 does not represent a character, but 11 does represent t. The final bit, 0 represents e.

i. e. the string 10110 is the encoding of ate

- **For Example:** The set {01,10,11,000} is a prefix code. The set {1,00,01,000,0001} is not prefix code because the sequence 00 is prefix of the sequence 000 and 0001.
- A prefix code can be represented using a binary tree, where the characters are the labels of the leaves in the tree. The edges of the tree are labeled so that an edge leading to a left child is assigned a 0 and an edge leading to a right child is assigned a 1.
- The bit string used to encode a character is the sequence of labels of the edges in the unique path from the root to the leaf that has this character as its label.

❖ In figure the encoding is represents as: e by 0, a by 10, t by 110, n by 1110, and s by 1111.



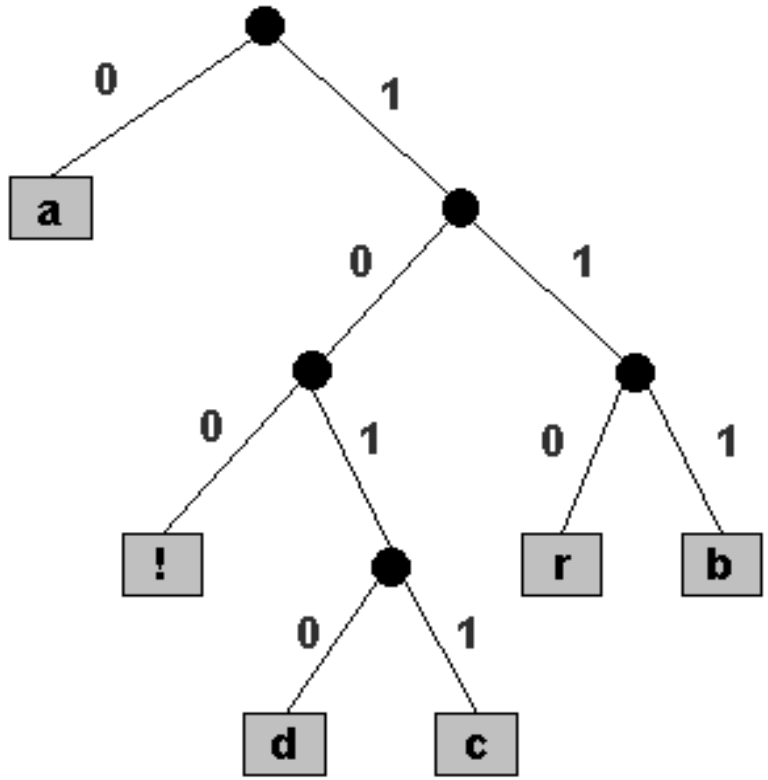
❖ Bit string 11111011100 using the code in figure encode the message the

" sane "

1111	10	1110	0
s	a	n	e

❖ In figure the encoding is represents as:

a by 0, b by 111, r by 110, c by 1011, d by 1010 and ! by 100.



❖ The bit string

0111110010110101001111100100

encodes the message

" abracadabra! "

0	111	110	0	1011	0	1010	0	111	110	0	100
a	b	r	a	c	a	d	a	b	r	a	!

❖ **Example 7:** Which of these codes are prefix codes?

➢ **i) a: 11, e: 00, t: 10, s:01**

✓ This is a prefix code, since no code is the first part of another.

➢ **ii) a:0, e:1, t: 01, s: 001**

✓ This is not a prefix code, since, for instance, the code for a is the first part of the code for t.

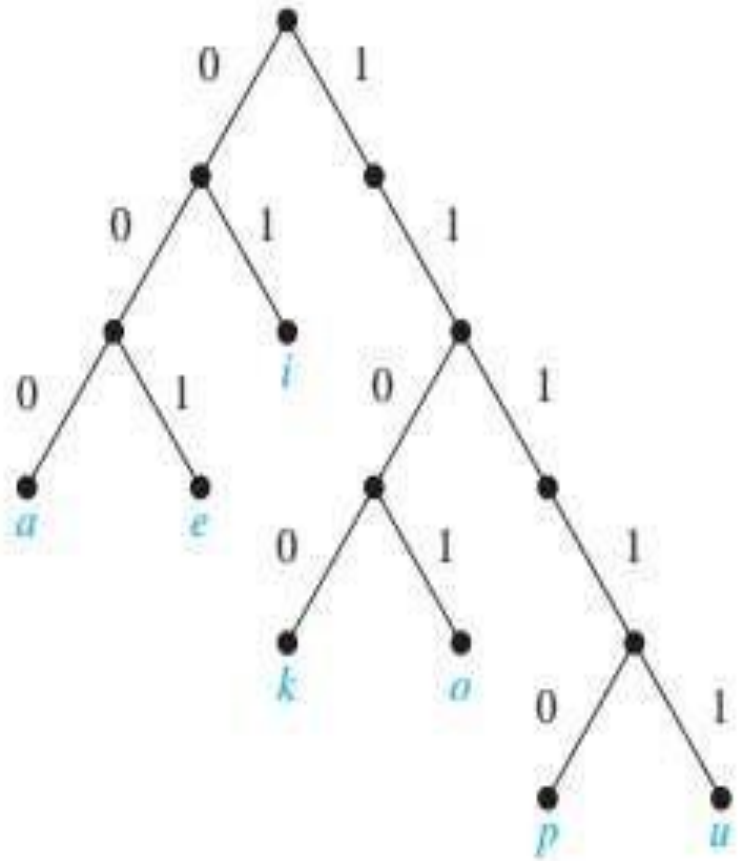
➢ **iii) a: 101, e: 11, t: 001, s: 011, n: 010**

✓ This is a prefix code, since no code is the first part of another.

➢ **iv) a: 010, e: 11, t: 011, s: 1011, n: 1001, i: 10101**

✓ This is a prefix code, since no code is the first part of another.

❖ **Example 8:** What are the codes for a, e, i, k, o, p, and u if the coding scheme is represented by this tree?



Solution:

a : 000 ;

e : 001;

i: 01;

k : 1100;

o : 1101;

p : 11110 and u : 11111.

Prefix Code-Homework

❖ **Example 9:** Construct the binary tree with prefix codes representing these coding schemes.

a) a: 11,e:0,t: 101,s: 100

b) a:1,e: 01,t: 001,s: 0001,n: 00001

c) a: 1010,e:0,t: 11,s: 1011,n: 1001,i: 10001

❖ **Example 10:** Given the coding scheme a:001, b:0001, e:1, r:0000, s:0100, t:011,x: 01010, find the word represented by

a) 01110100011.

b)0001110000.

c) 0100101010.

d)01100101010.

Optimal Tree

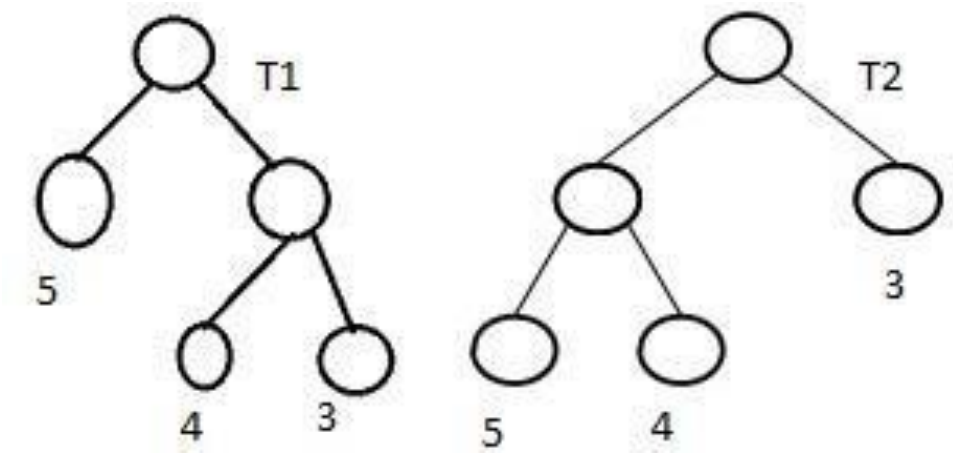
❖ Let T be any full binary tree and w1,w2,w3,….wt be the weights of the terminal vertices. Then the weight W of a binary tree is given by:

$$W(T) = \sum_{i=1}^{t} wili$$

Where li is the length of the leaf i from the root of the tree. The fully binary tree is called optimal if its weight is minimum.

❖ Optimal tree are used to construct variable length codes, where the letters of alphabets are represented by binary digit.

❖ **Example 11:** Figure show two fully binary tree T1 and T2, with the weights of the leaves as 3,4 and 5. Show which tree is optimal tree.



Solution:

$W(T1) = 3*2 + 4*2 + 5*1 = 6 + 8 + 5 = 19$

$W(T2) = 3*1 + 4*2 + 5*2 = 2 + 8 + 10 = 21$

Here $W(T1) < W(T2)$, Thus T1 is optimal tree for weights 3,4, and 5.

- Binary character code: each character is represented by a unique binary string.
- A data file can be coded in two ways:

	a	b	c	d	e	f
frequency(%)	45	13	12	16	9	5
fixed-length code	000	001	010	011	100	101
variable-length code	0	101	100	111	1101	1100

The first way needs $100 \times 3 = 300$ bits. The second way needs

$45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4 = 232$ bits.

Optimal Prefix Code-Huffman Algorithm

❖ A binary prefix code obtained from optimal tree is called **optimal prefix code.**

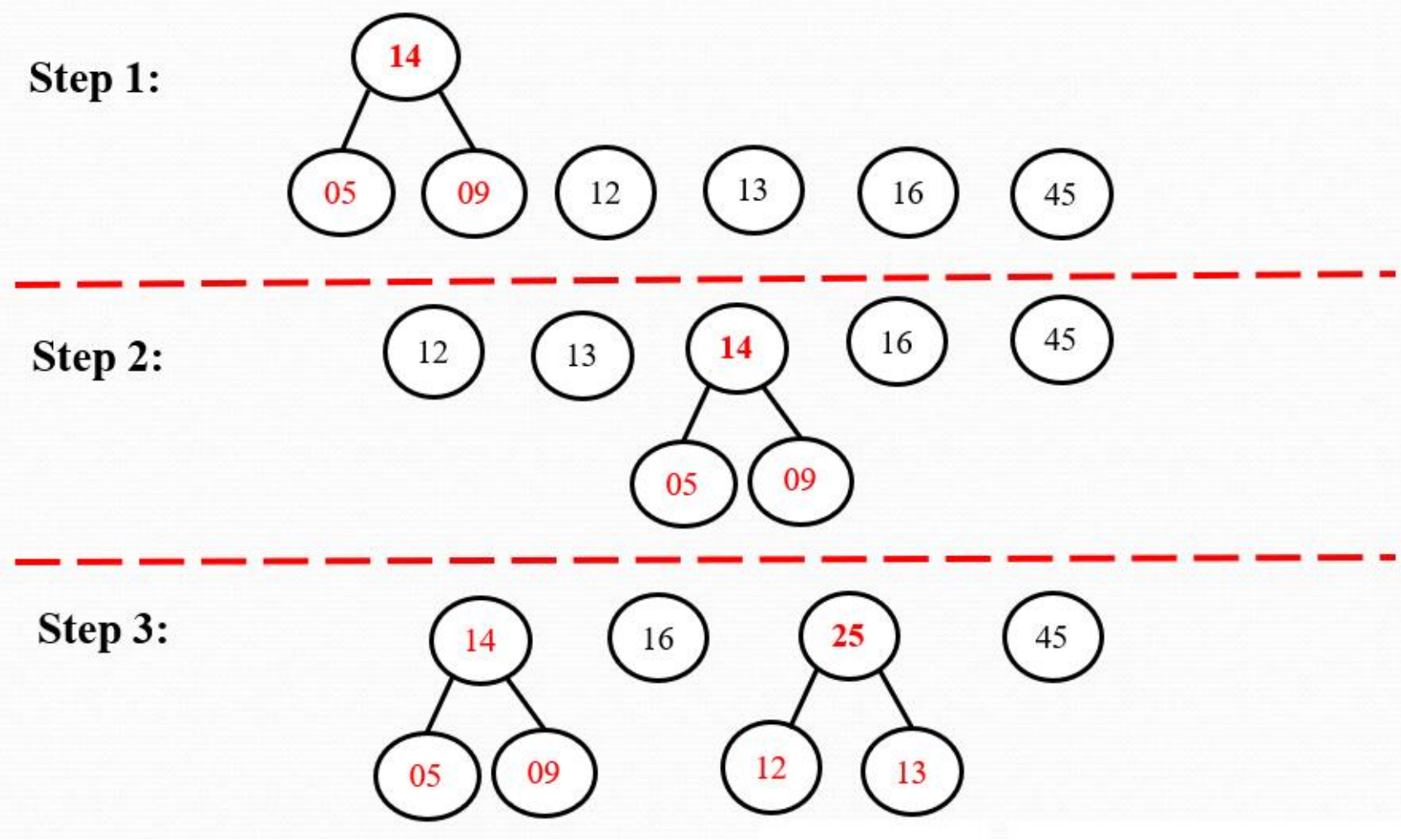
❖ Huffman Algorithm to find optimal tree::

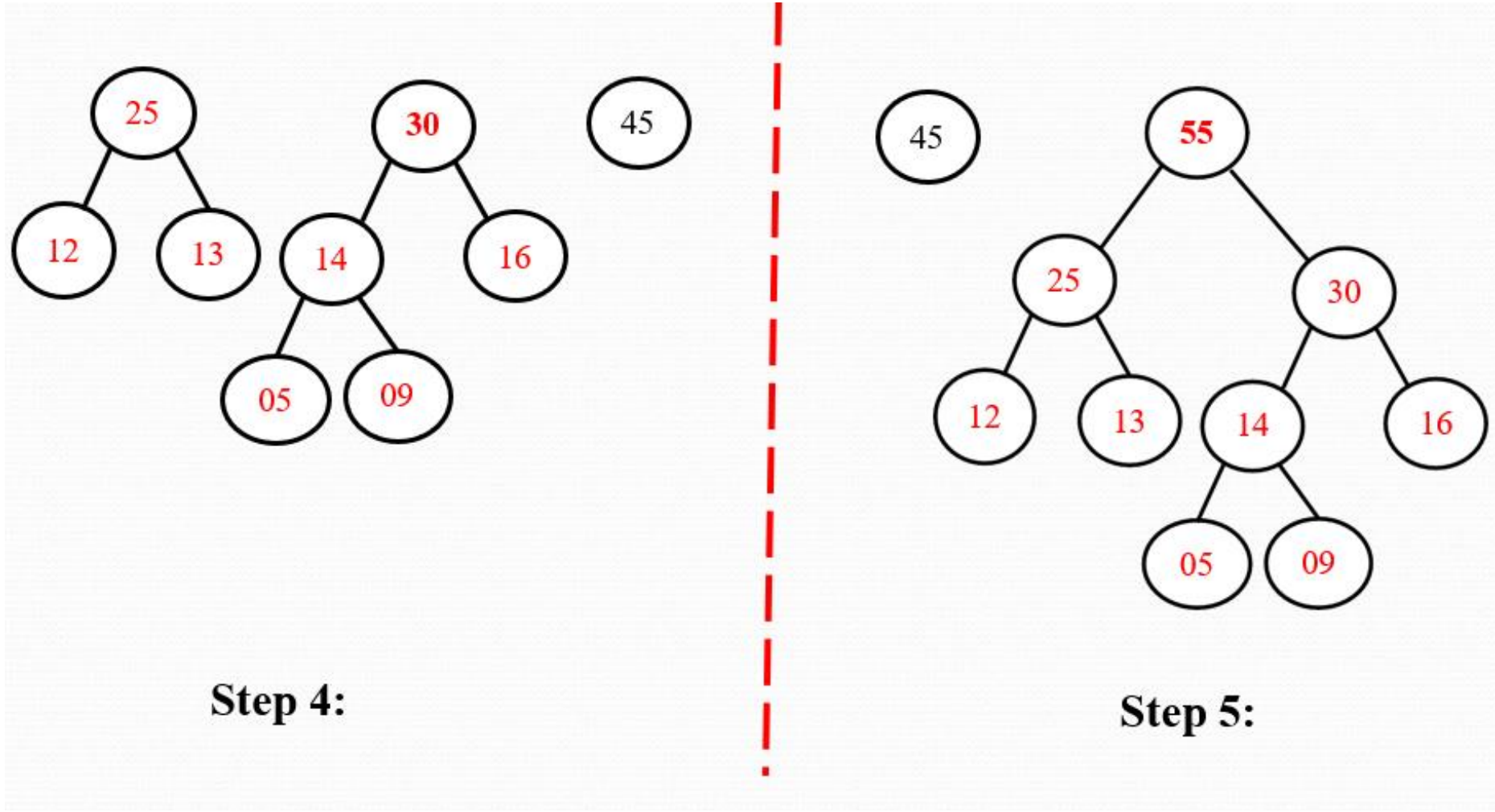
- A Huffman code is a particular type of optimal prefix code that is commonly used for lossless data compression.
- **Given:** A set of symbols and their weights (usually proportional to probabilities).
- **Find:** A prefix-free binary code (a set of codewords) with minimum expected codeword length (equivalently, a tree with minimum weighted path length from the root).
- Let W1,W2,W3,….,Wt be the weights of the leaves and it is required to construct an optimal binary tree.

- ✓ **Step 1:** Arrange the weights of a tree in increasing order.
- ✓ **Step 2:** Consider two leaves with minimum weights W1 and W2. Replace two leaves and their parent

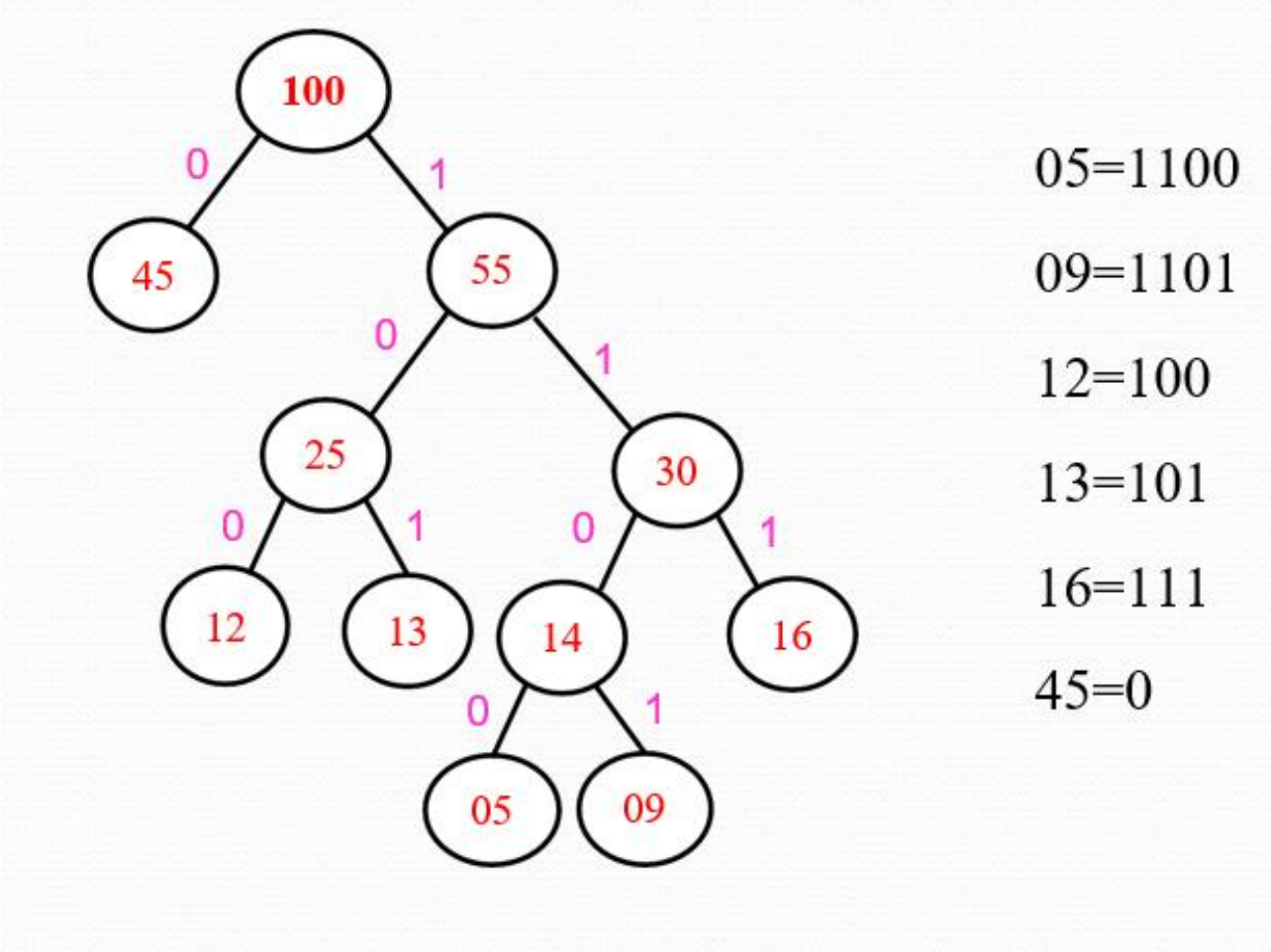
  by the leaf. Assign weight W1 + W2 to this new leaf.
- ✓ **Step 3:** Repeat step 2 for the weights W1,W2,W3,….,Wt until no weight remains.
- ✓ **Step 4:** The tree obtained in this method is an optimal tree for given weights and stop the procedure.

❖ **Example 12:** Construct Huffman code for the data: 45, 13, 12, 16, 9, 5.



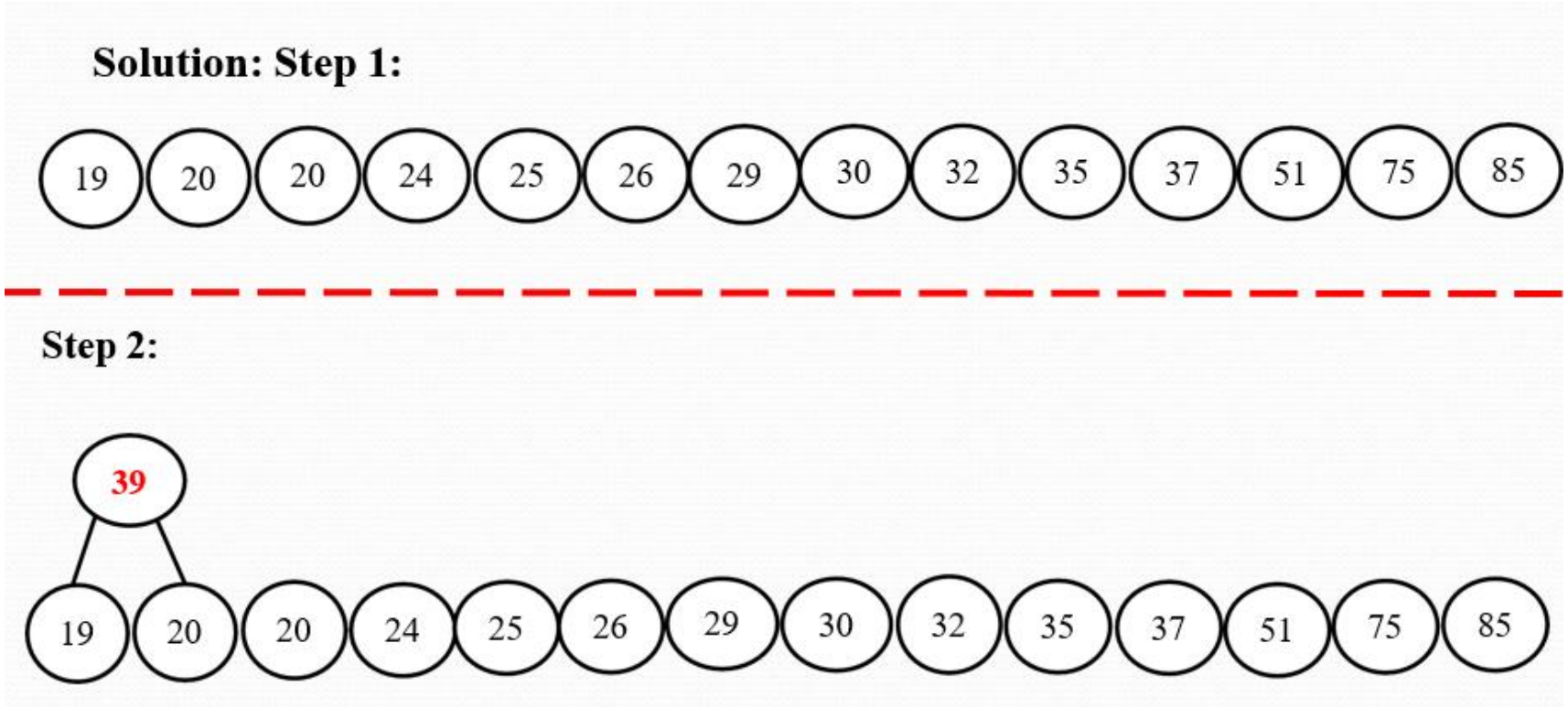

25
30
45
12
13
14
16
05
09
Step 4:
45
55
25
30
12
13
14
16
05
09
Step 5:

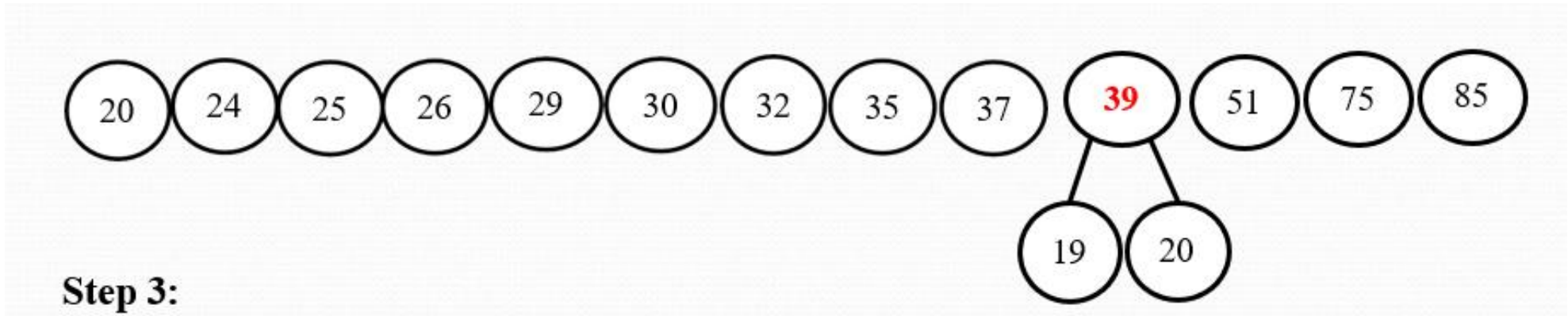
Step 6:


100
0
1
45
55
0
1
25
30
0
1
0
1
12
13
14
16
0
1
05
09
05=1100
09=1101
12=100
13=101
16=111
45=0

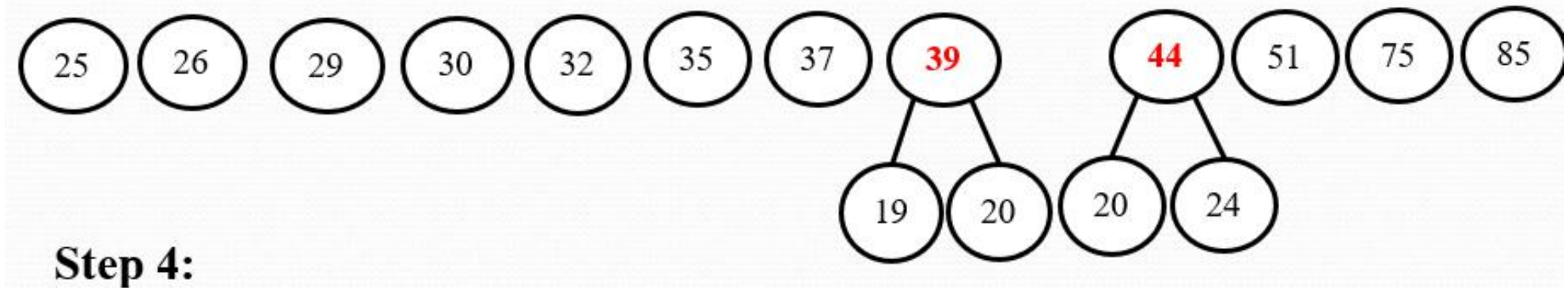
❖ **Example 13:** A secondary storage media contains information in files with different formats. The frequency of different types of files is as follows. Exe (20), bin(75), bat(20), jpeg(85), dat(51), doc(32), sys(26), c(19), cpp(25), bmp(30), avi(24), prj(29), 1st(35), zip(37). Construct the Huffman code of this.

Solution: Step 1:

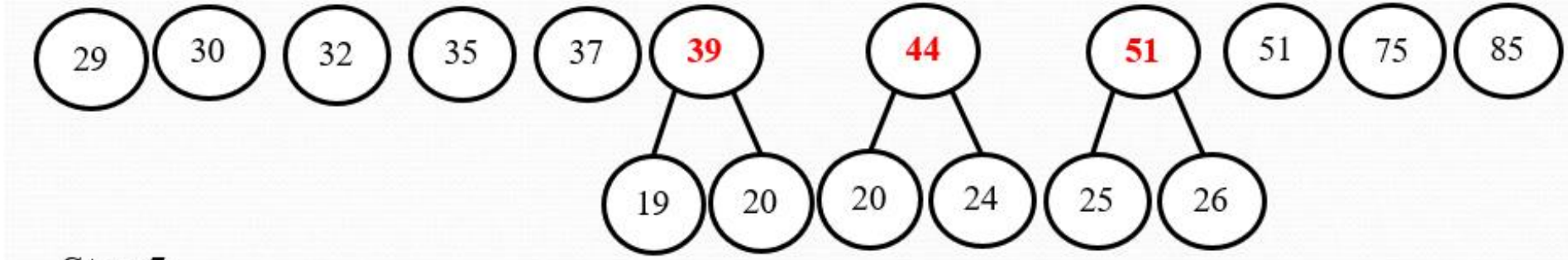



20
24
25
26
29
30
32
35
37
39
51
75
85
19
20

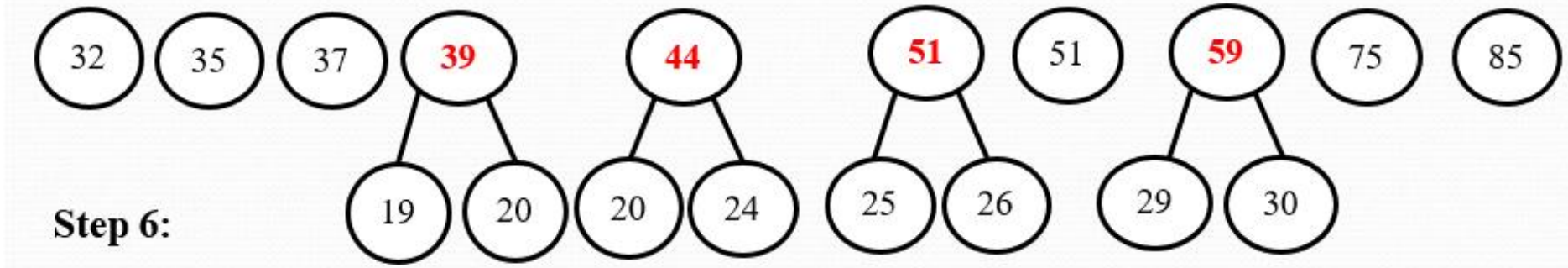
Step 3:


25
26
29
30
32
35
37
39
44
51
75
85
19
20
20
24

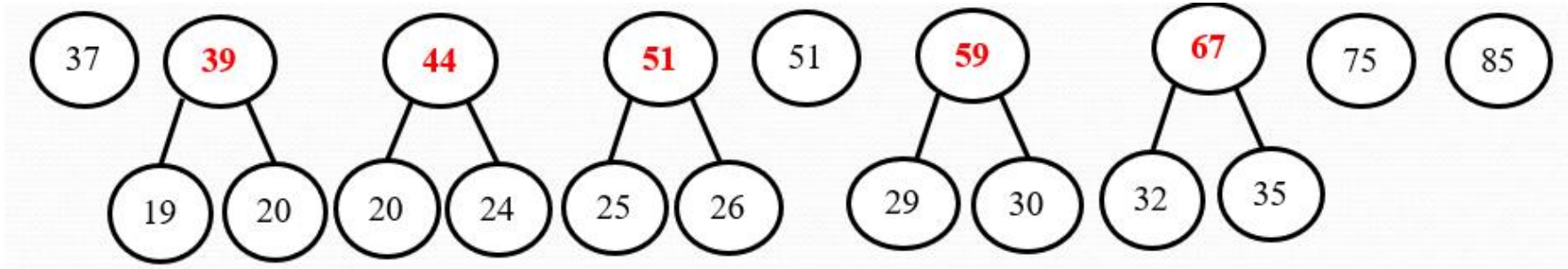
Step 4:


29
30
32
35
37
39
44
51
51
75
85
19
20
20
24
25
26

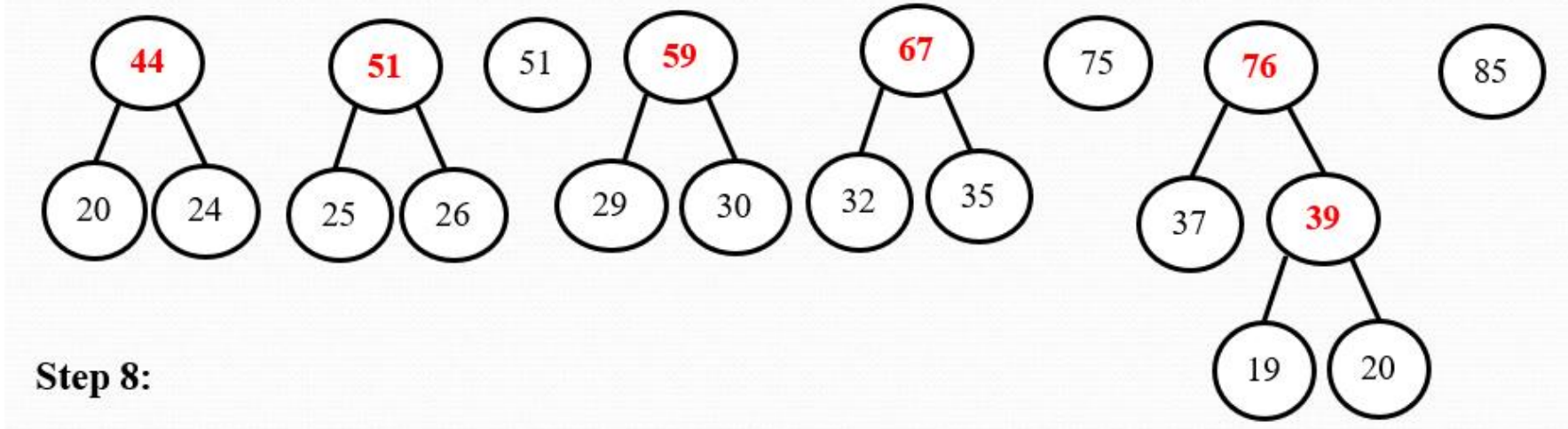
Step 5:


32
35
37
39
44
51
51
59
75
85
19
20
20
24
25
26
29
30

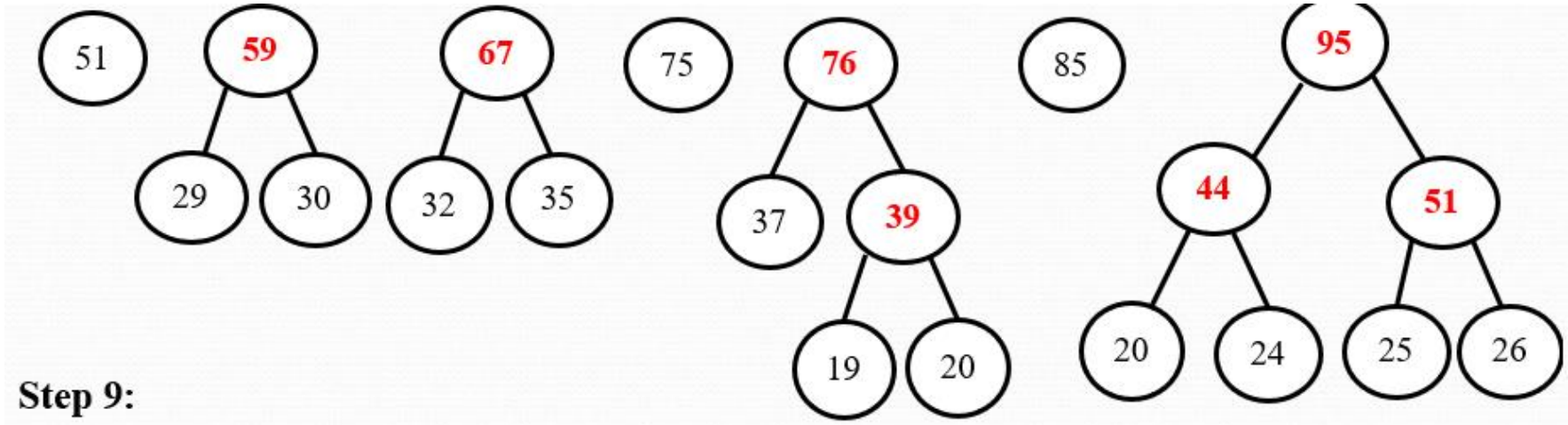
Step 6:


37
39
44
51
51
59
67
75
85
19
20
20
24
25
26
29
30
32
35

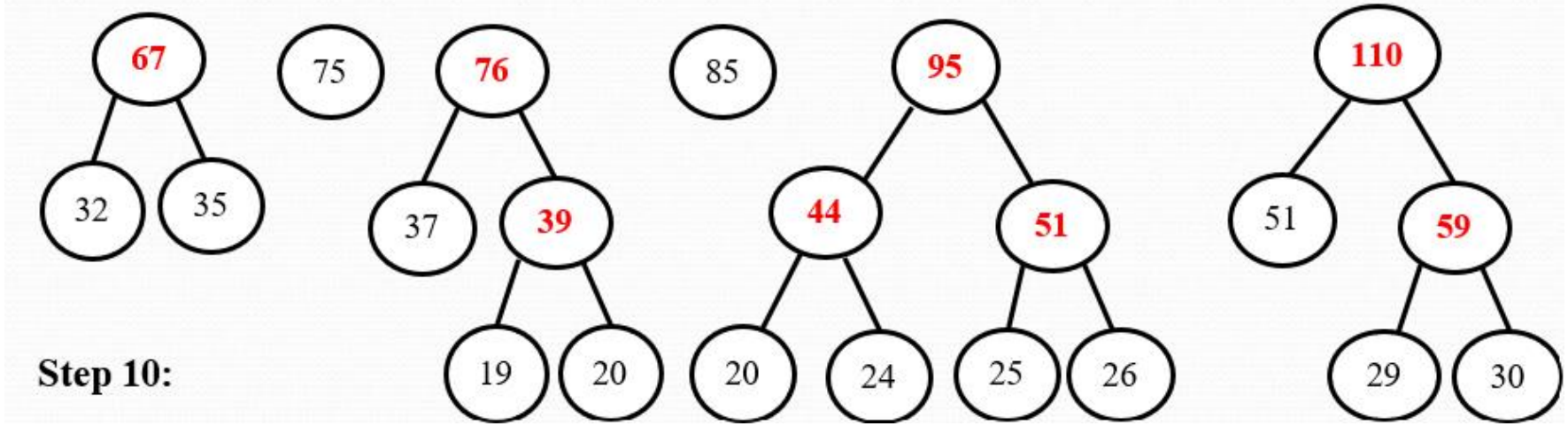
Step 7:


44
51
51
59
67
75
76
85
20
24
25
26
29
30
32
35
37
39
19
20

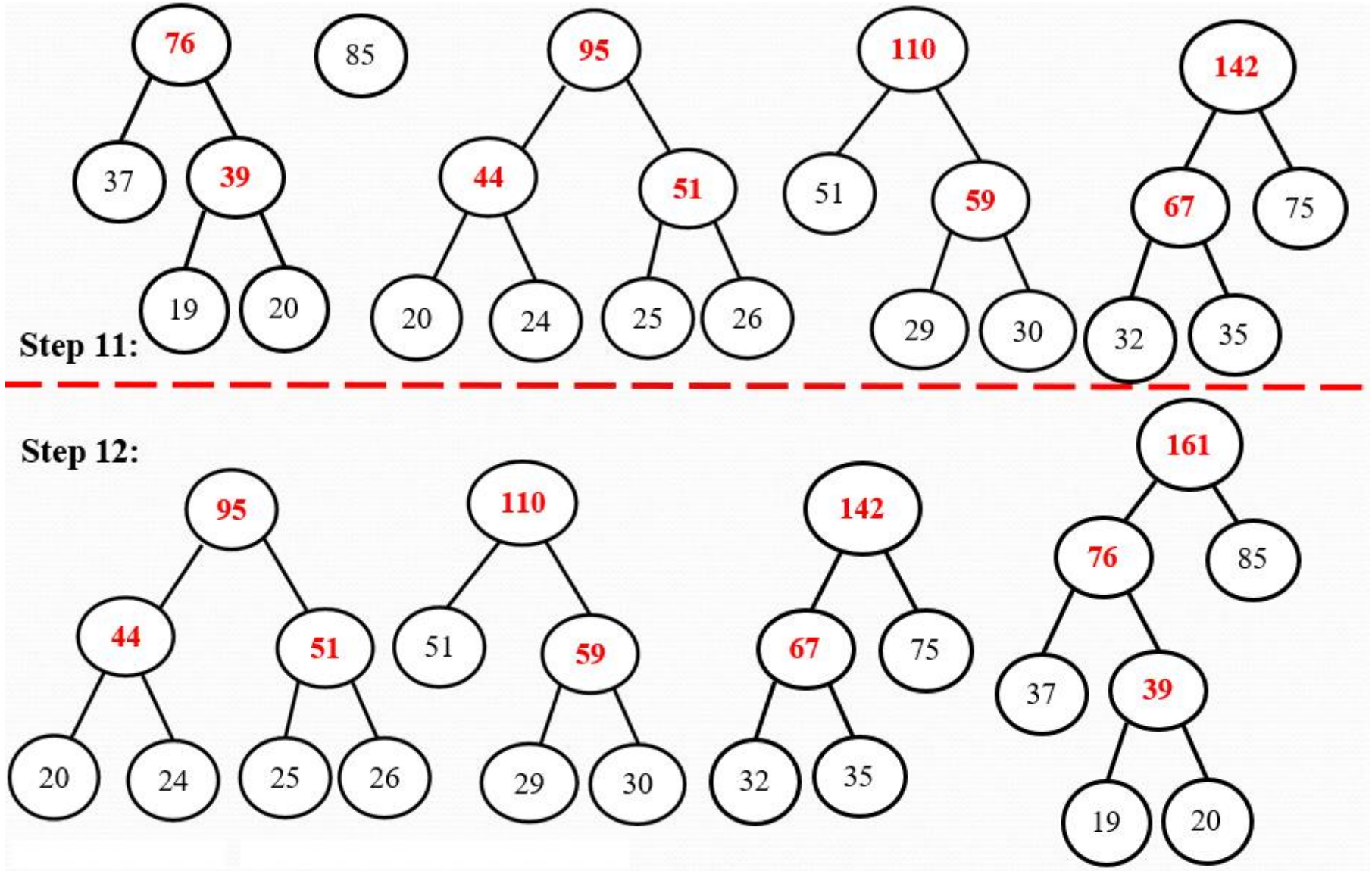
Step 8:



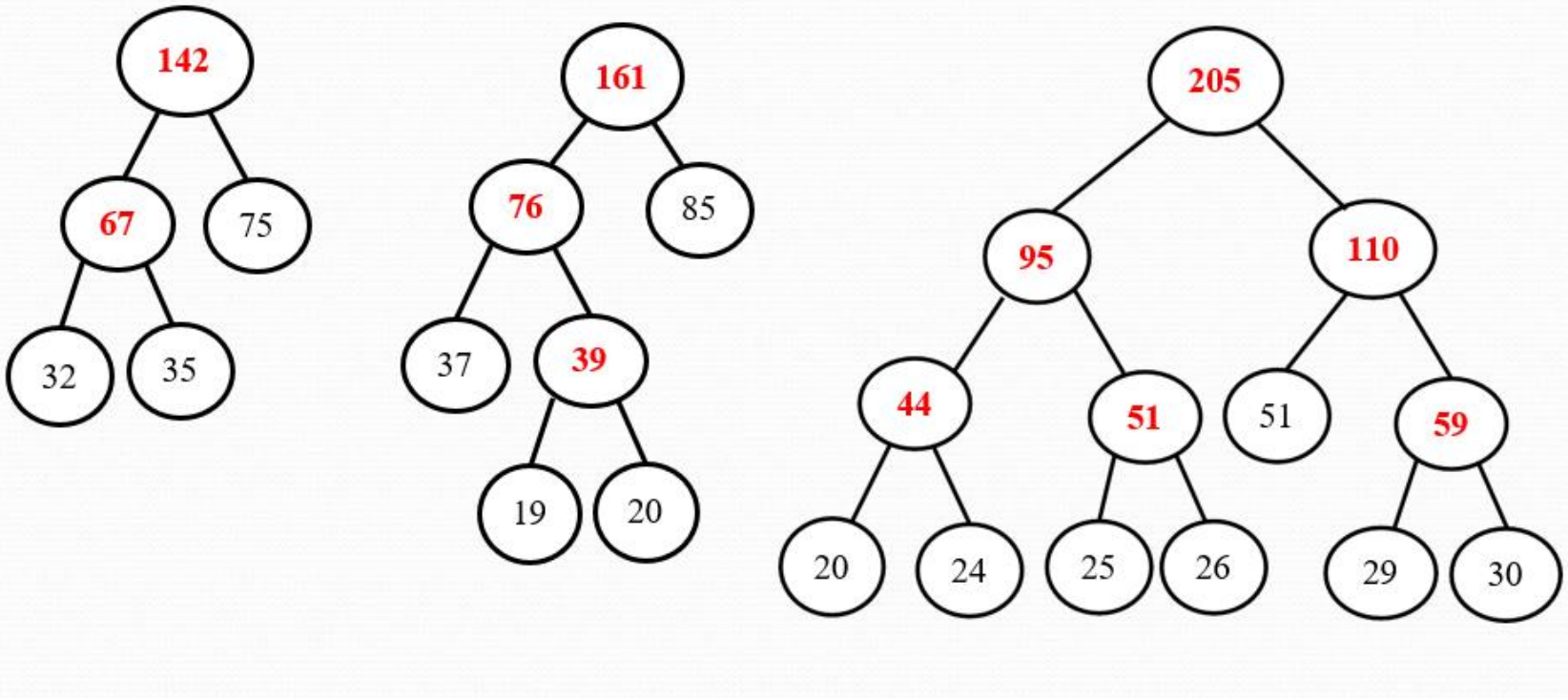
Step 9:



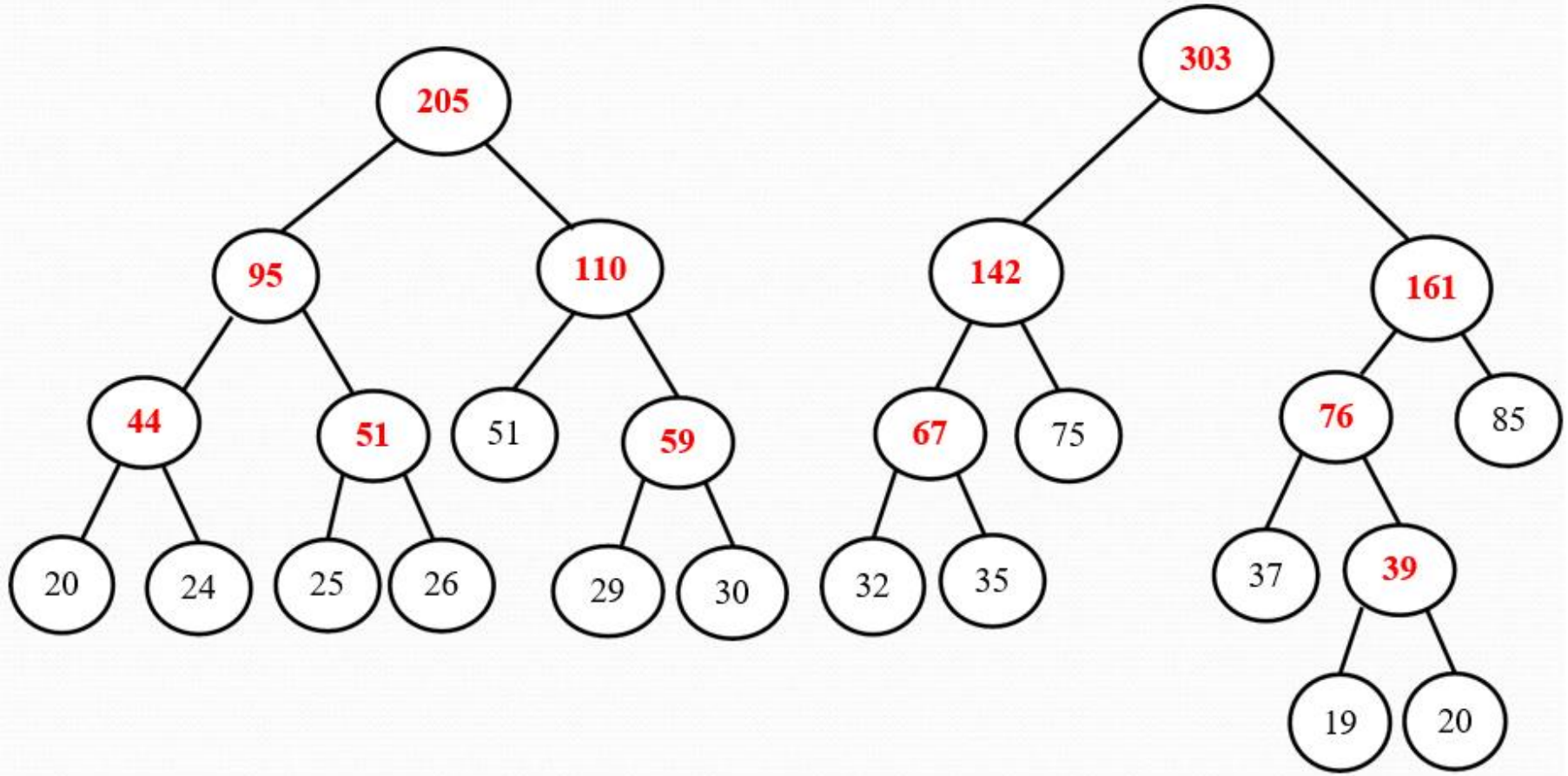
Step 10:

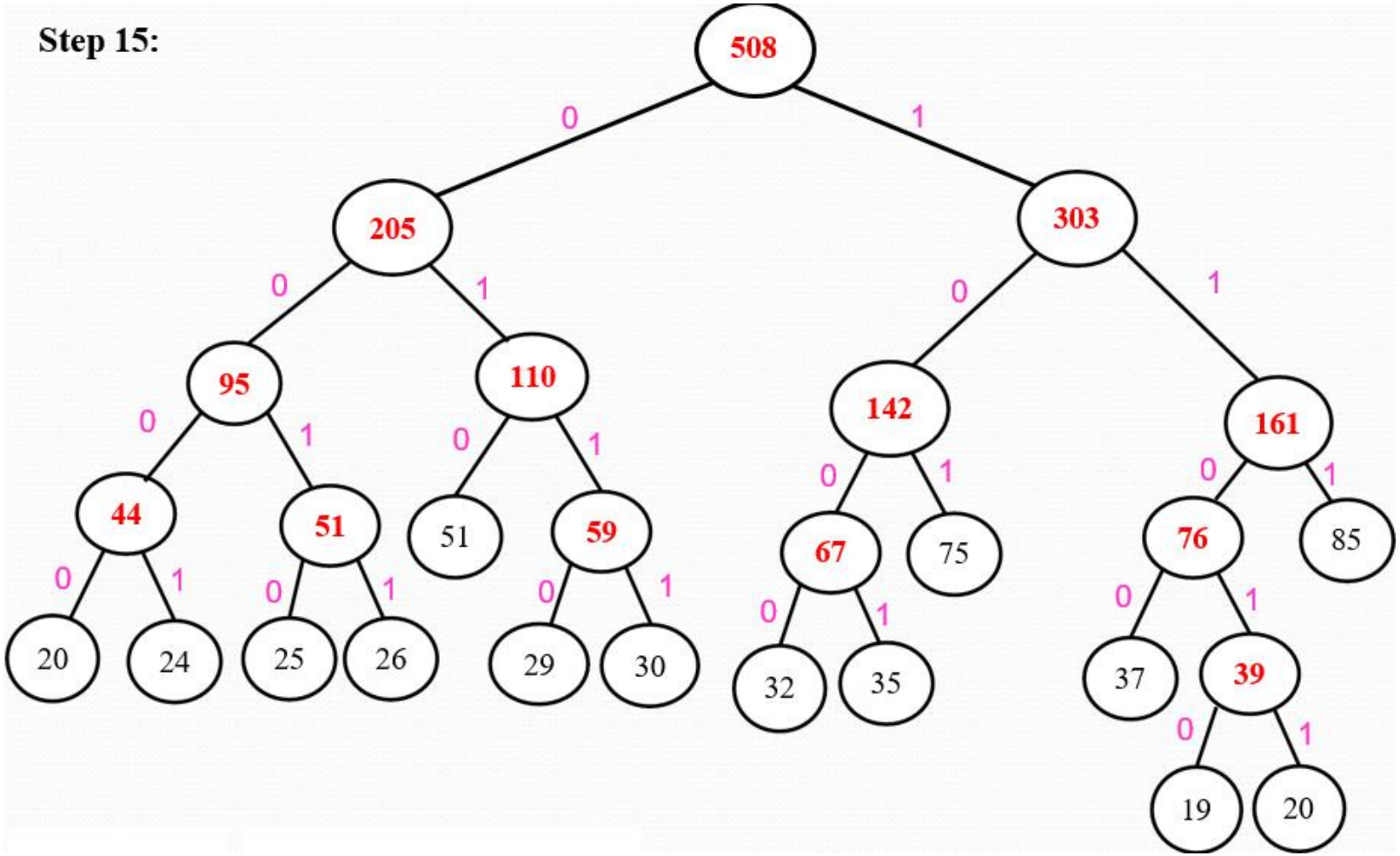

Step 11:
76
85
95
110
142
37
39
44
51
51
59
67
75
19
20
20
24
25
26
29
30
32
35
Step 12:
95
110
142
161
44
51
51
59
67
75
76
85
20
24
25
26
29
30
32
35
37
39
19
20

Step 13:


142
67
75
32
35
161
76
85
37
39
19
20
205
95
110
44
51
51
59
20
24
25
26
29
30

Step 14:


303
205
142
161
95
110
44
51
51
59
67
75
76
85
20
24
25
26
29
30
32
35
37
39
19
20


Step 15:
508
0
1
205
303
0
1
0
1
95
110
142
161
0
1
0
1
0
1
0
1
44
51
51
59
67
75
76
85
0
1
0
1
0
1
0
1
0
1
20
24
25
26
29
30
32
35
37
39
0
1
19
20

❖ **Example 14:** Construct Huffman code for the data: A: 0.08, B:0.10, C: 0.12, D: 0.15, E: 0.20, F: 0.35. What is the average number of bits used to encode a character?

Solution: (Draw tree):

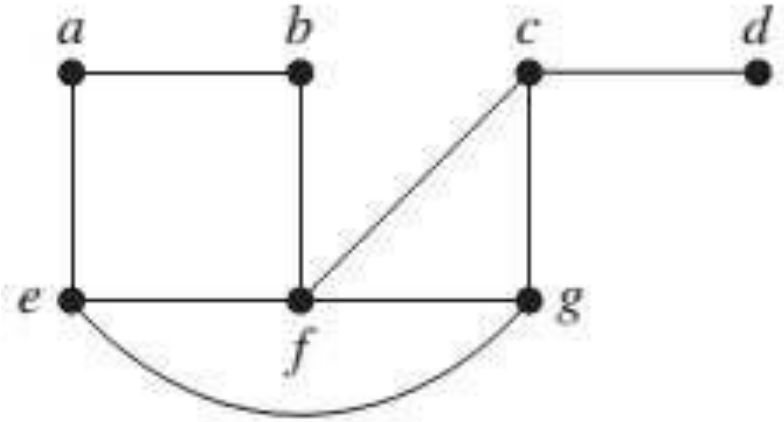
The average number of bits used to encode a symbol using this encoding is

$$3*0.08 + 3*0.10 + 3*0.12 + 3*0.15 + 2*0.20 + 2*0.35 = 2.45$$

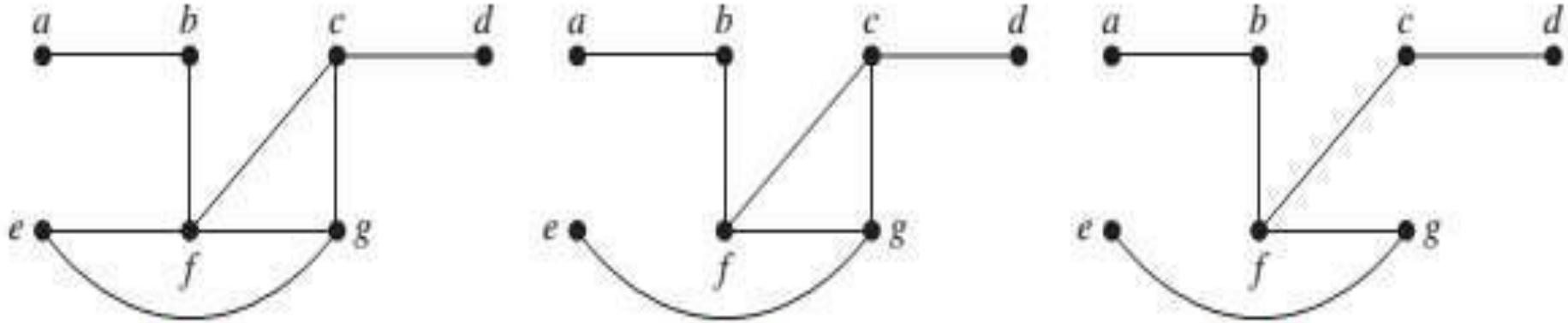
Spanning Tree

- Let G be a simple graph. A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.
- In general, a graph may have several spanning trees, but a graph that is not connected will not contain a spanning tree
- A simple graph with a spanning tree must be connected, because there is a path in the spanning tree between any two vertices.
- The converse is also true; that is, every connected simple graph has a spanning tree.
- **Theorem:** A simple graph is connected if and only if it has a spanning tree.

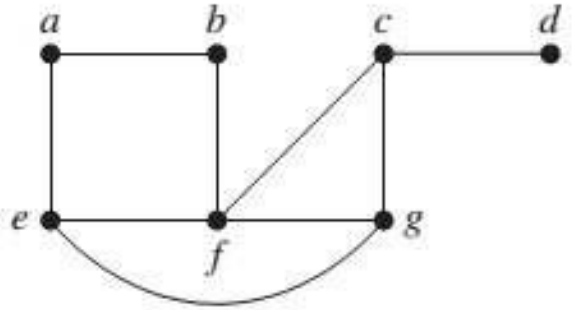
❖ **Example 17:** Find a spanning tree of the simple graph G shown in Figure.



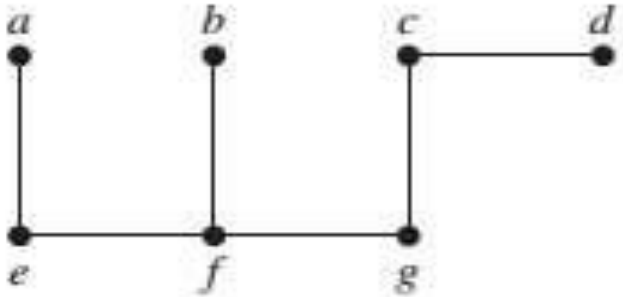
Solution: Fig c below is spanning tree T.

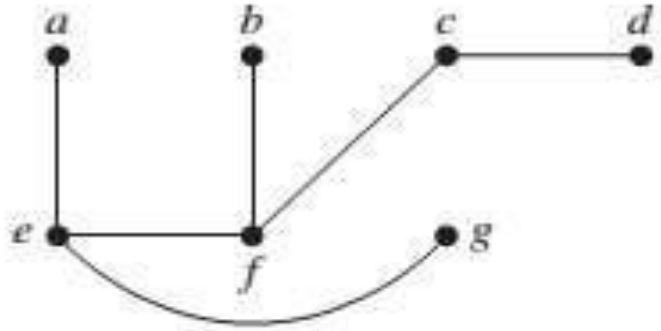


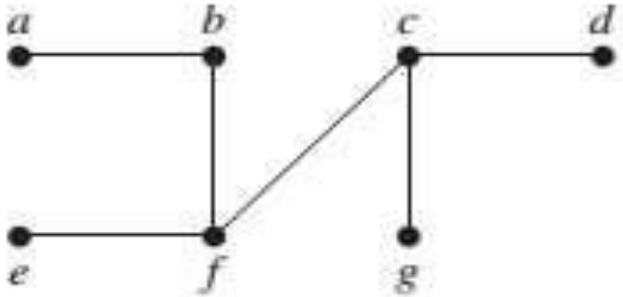
❖ **Example 17:** Find a spanning tree of the simple graph G shown in Figure.

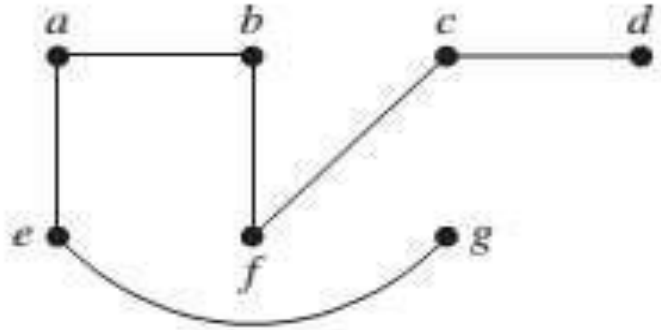


Solution:









❖ **Example 18:** How many edges must be removed from a connected graph with v vertices and e edges to produce a spanning tree?

Solution:

The graph has e edges. The spanning tree has v - 1

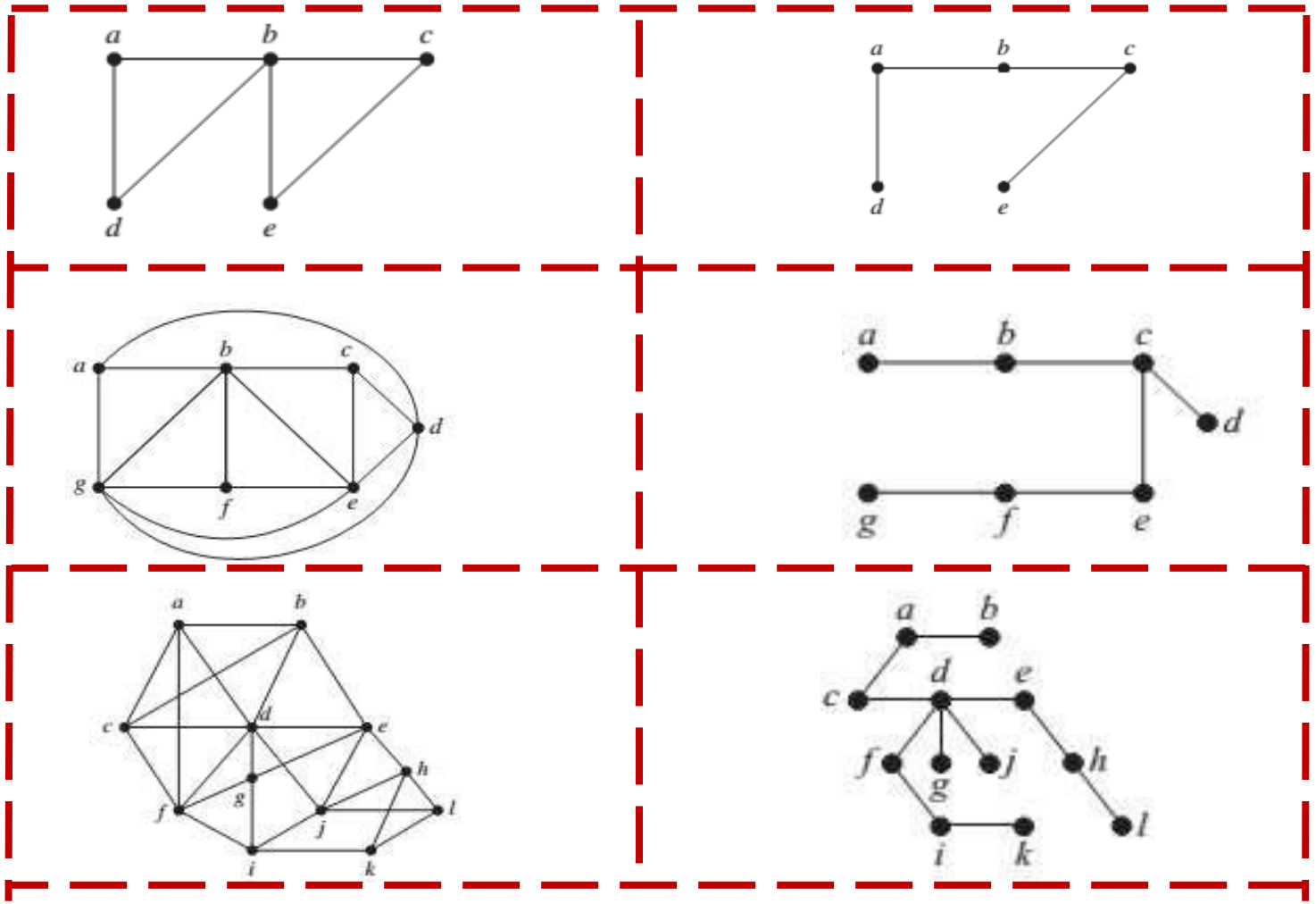
edges. Therefore we need to remove : e - (v - 1) edges.

Hence Graph has : e – v + 1 edges

❖ **Example 19:** Find a spanning tree for each of these graphs.

a) K5 b)K4,4 c) K1,6 d)Q3 e) C5 f) W5

❖ **Example 20:** Find a spanning tree for each simple graph G shown in Fig:



Minimum Spanning Tree

- A minimum spanning tree in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.

- A minimum spanning tree (MST) or minimum weight spanning tree is then a spanning tree with weight less than or equal to the weight of every other spanning tree.

- There are two algorithm to find minimum spanning tree.
  - Prim‟s Algorithm
  - Krushal‟s Algorithm

Prim's Algorithm

- Prim's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected weighted undirected graph.
- It finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized.
- This algorithm is directly based on the MST property.
- The algorithm may informally be described as performing the following steps:

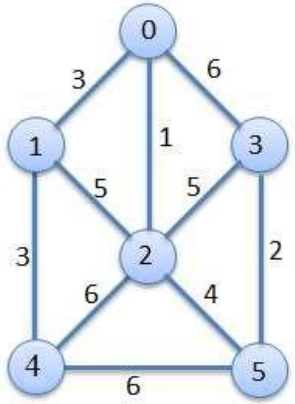
1. Initialize a tree with a single vertex, chosen arbitrarily from the graph.
2. Grow the tree by one edge: of the edges that connect the tree to vertices not yet in the tree, find the minimum-weight edge, and transfer it to the tree.
3. Repeat step 2 (until all vertices are in the tree).

❖ Following are details step of implementation of Prim‟s algorithm:

- Let G(V,E) be connected weighted graph
- **Step 1:** Select any vertex V0 in graph. Set T={ V0, ∅}
- **Step 2:** Find edge ei= (V0, Vi) in E such that its one vertex is V0 ∈ T and its weight is minimum. Therefore New Set T={{ V0, V1},{ei}}
- **Step 3:** Choose next edge ek = (Vk, Vj) in such a way that it‟s one vertex Vk ∈ T and other vertex Vj ∉ T and weight of ek is as small as possible. Again join vetex Vj and edge ek to T.
- **Step 4**: Repeat step 3 until T contain all vertices of graph G. The result will be MST of graph G.

❖ **Example 21:** Using Prim‟s Algorithm Find MST for given graph.

Solution:





❖ **Step 1:** Select Vertex 0 as a starting vertex. Therefore T = {{0},∅}



❖ **Step 2:** Vertex 0 is adjacent to vertex 1, 2 and 3.

Edge (0, 1) = 3, edge (0, 2) = 1 and edge (0, 3) = 6. Select edge with minimum weight.

i.e. say edge e1 = (0,2) = 1 Therefore T = {{0, 2}, e1}.

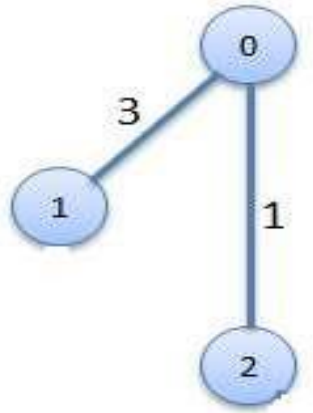
❖ **Step 3:** Now Vertex 0 is adjacent to vertex 1, and 3. Edge (0, 1) = 3 and edge (0, 3) = 6.

Vertex 2 is adjacent to 1, 3, 4, and 5

Edge (2, 1) = 5, edge (2, 3) = 5, edge (2, 4) = 6 and edge (2, 5) = 4

Select edge with minimum weight. i.e. say edge e2 = (0, 1) = 3.

Therefore T = {{0, 2, 1}, e1, e2}.



❖ **Step 4:** Now Vertex 0 is adjacent to vertex 3. Edge (0, 3) = 6.

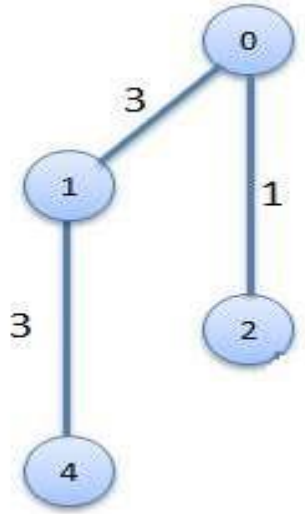
Vertex 2 is adjacent to 3, 4, and 5

Edge (2, 3) = 5 ,edge (2, 4) = 6 and edge (2, 5) = 4

Vertex 1 is adjacent to vertex 4. Edge (1, 4) = 3.

Select edge with minimum weight. i.e. say edge e3 = (1, 4) = 3.

Therefore T = {{0, 2, 1, 4}, e1, e2, e3}.



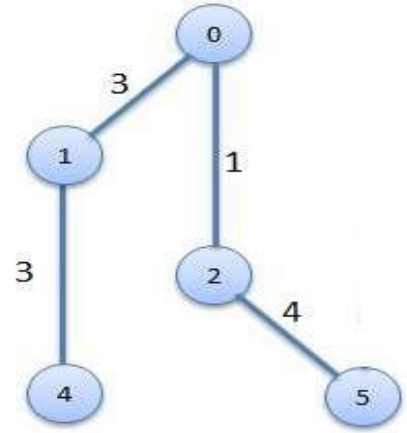
❖ **Step 5:** Now Vertex 0 is adjacent to vertex 3 Edge (0, 3) = 6.

Vertex 2 is adjacent to 3, and 5 Edge (2, 3) = 5 and edge (2, 5) = 4.

Vertex 4 is adjacent to 5 Edge (4, 5) = 6.

Select edge with minimum weight. i.e. say edge e4 = (2, 5) = 4

Therefore T = {{0, 2, 1, 4, 5}, e1, e2, e3, e4}.



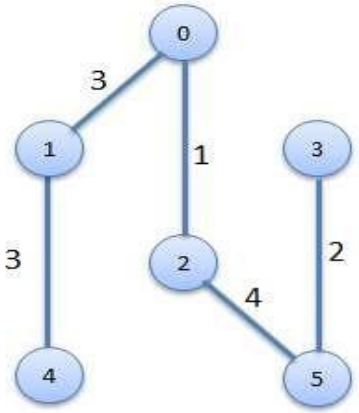
❖ **Step 6:** Now Vertex 0 is adjacent to vertex 3. Edge (0, 3) = 6.

Vertex 2 is adjacent to 3. Edge (2, 3) = 5

Vertex 5 is adjacent to 3. Edge (5, 3) = 2

Select edge with minimum weight. i.e. say edge e5 = (5, 3) =

2 Therefore T = {{0, 2, 1, 4, 5, 3}, e1, e2, e3, e4, e5}.



❖ **Step 7:** All vertex are include in spanning tree, and graph obtained is the minimum spanning tree. **Minimum Cost = 1 + 2 + 3 + 3 + 4 = 13.**

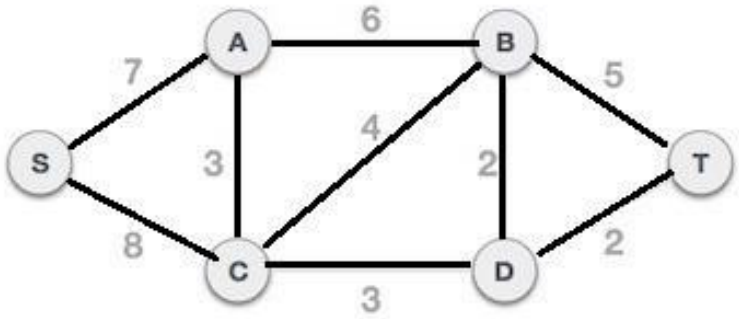
Krushal's Algorithm

- Kruskal's algorithm is a minimum-spanning-tree algorithm which finds an edge of the least possible weight that connects any two trees in the forest.
- It is a greedy algorithm in graph theory as it finds a minimum spanning tree for a connected weighted graph adding increasing cost arcs at each step.
- This means it finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimised.
- If the graph is not connected, then it finds a minimum spanning forest.

- Following are details step of implementation of Krushal‟s algorithm.
- Let G(V,E) be weighted connected graph.
- **Step 1:** Pick up the e_i of the Graph G such that its weights W(e,i) is minimum.(If there are more edges of the minimum weight then select all those edges which do not form circuit)
- **Step 2:** If edges e1,e2,e3,…..en have been chosen then pick an edge e_{n+1} such that:
  - i. for i=1,2,3,…,n
  - ii. The edges $e_1,e_2,e_3,\ldots..e_n$ e_{n+1} do not form circuit.
  - iii. $W(e_{n+1})$ is as small as possible subject to condition (ii).
- **Step 3:** Stop procedure when step two cannot be implemented further.

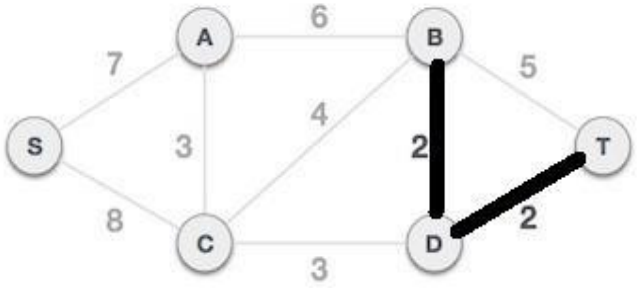
❖ **Example 22**: Using Krushal algorithm Find MST for given graph.

Solution: Arrange all edges in their increasing order of weight.

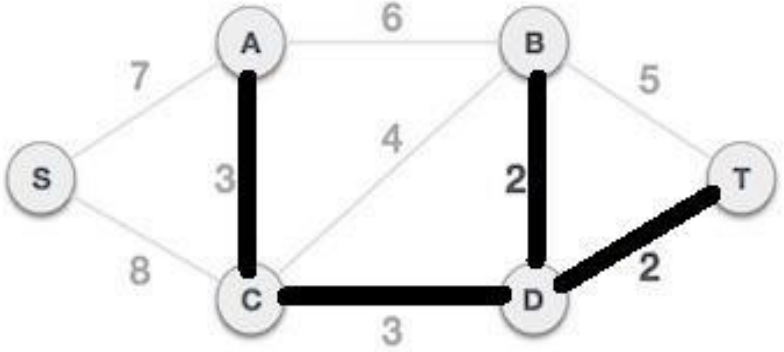


BD	DT	AC	CD	CB	BT	AB	SA	SC
2	2	3	3	4	5	6	7	8

Step 1: Now we start adding edges which has the least weight. The least cost is 2 and edges involved are B,D and D,T. We add them. Adding them does not violate spanning tree properties, so we continue to our next edge selection.



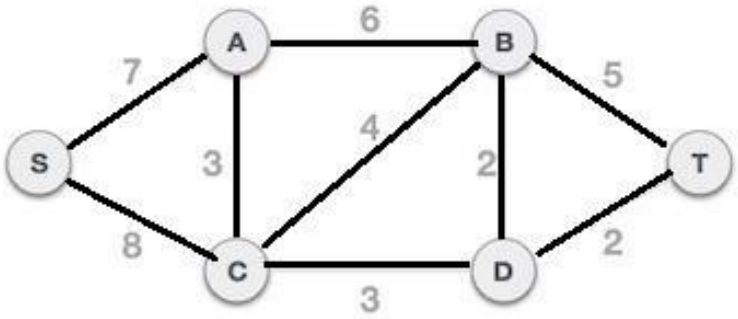
Step 2: Next cost is 3, and associated edges are A,C and C,D. We add them again.



❖ **Example 22**: Using Krushal algorithm Find MST for given graph.

Solution: Arrange all edges in their increasing order of weight.

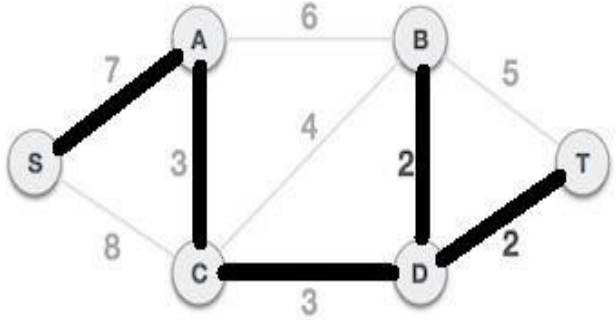
BD	DT	AC	CD	CB	BT	AB	SA	SC
2	2	3	3	4	5	6	7	8



Step 3: Next cost in the table is 4, and we observe that adding it will create a circuit in the graph. We ignore it. In the process we shall ignore/avoid all edges that create a circuit. We observe that edges with cost 5 and 6 also create circuits. We ignore them and move on.

Step 4: Now we are left with only one node to be added. Between the two least cost edges available 7 and 8, we shall add the edge with cost 7.

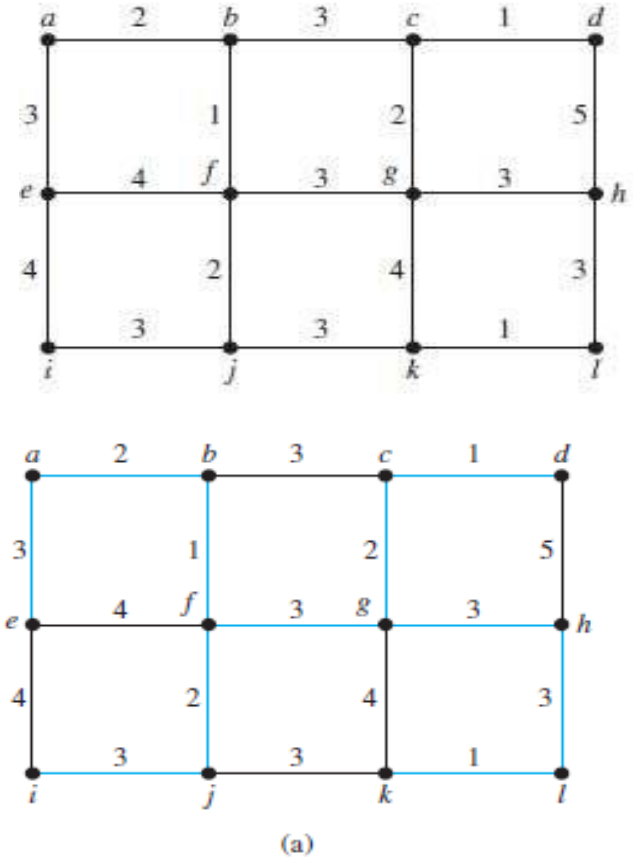
By adding edge S, A we have included all the nodes of the graph and we now have minimum cost spanning tree.



Minimum Cost Spanning Tree = 2 + 2 + 3 + 3 + 7 = 17.

MST Example-Prim's algorithm

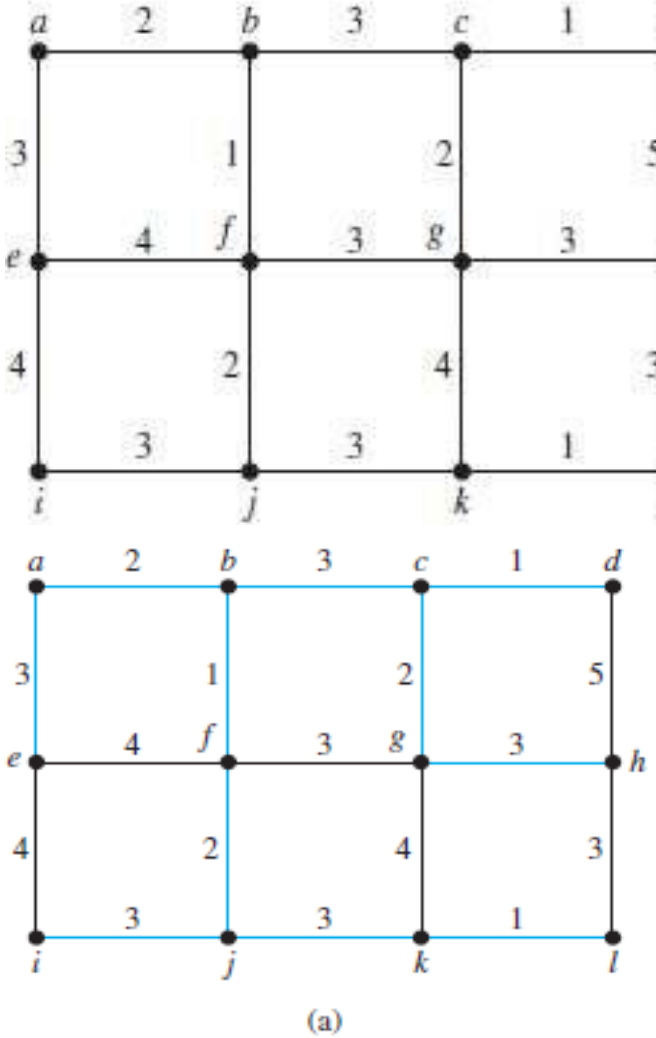
❖ Use Prim"s algorithm to find a minimum spanning tree in the graph shown in Figure.



(a)

Choice	Edge	Weight
1	b, e	1
2	a, b	2
3	f, j	2
4	a, e	3
5	i, j	3
6	f, g	3
7	c, g	2
8	c, d	1
9	g, h	3
10	h, l	3
11	k, l	1
Total		**24**

❖ Use Krushal‟s algorithm to find a minimum spanning tree in the graph shown in Figure.



(a)

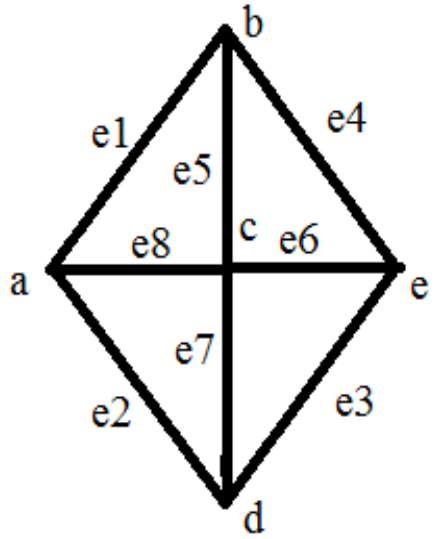
Choice	Edge	Weight
1	c, d	1
2	k, l	1
3	b, f	1
4	c, g	2
5	a, b	2
6	f, j	2
7	b, c	3
8	j, k	3
9	g, h	3
10	i, j	3
11	a, e	3
Total		**24**

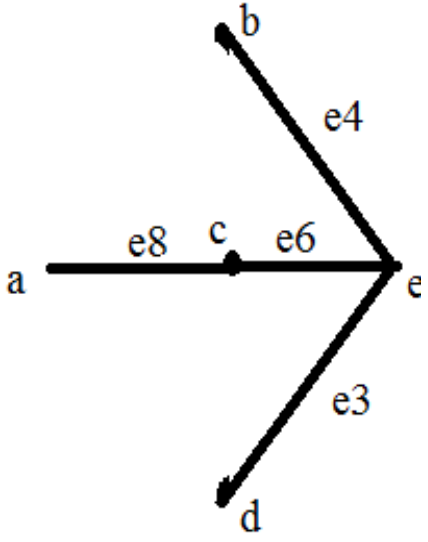
Fundamental Circuit

- For a Graph G and its associated spanning tree T, the fundamental circuit of a chord „e" of T is given by the circuit formed due to addition of „e" in T.
- In other words Let G be connected graph and Let T be spanning tree of G. Consider a chord of T i.e. an edge of G which is not in T. Then „T + e" contain a unique circuit called as **fundamental circuit.**
- For different spanning tree of graph G, the fundamental circuit will be different.
- The number of fundamental circuit in any spanning tree is equal to the numbers of chords of that spanning tree.
- Hence if the connected graph G has v number of vertices and e number of edges then spanning tree T has (v-1) branches and (e - v + 1) numbers of chords.
- Thus there will be (e - v + 1) number of fundamental circuit in G with respect to the spanning tree T.

❖ **Example 23:** Find the fundamental circuit for the graph show in figure below w.r.t the spanning tree T.

Solution:





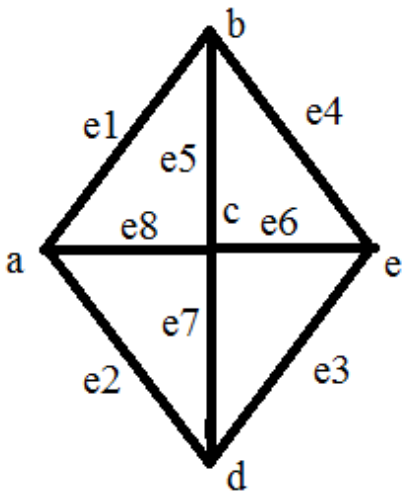
Chord	Fundamental Circuit
e1	{ e1, e4, e6, e8 }
e2	{ e2, e3, e6, e8 }
e5	{ e5, e4, e6 }
e7	{e7, e3, e6 }

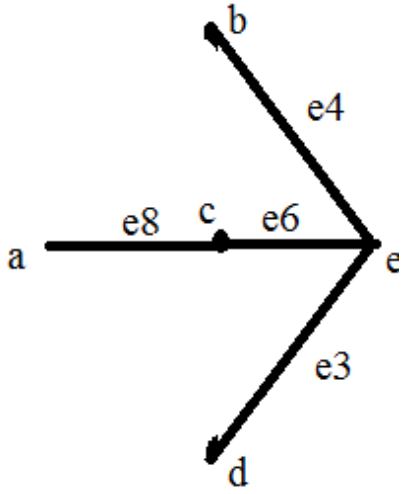
Fundamental Cut-sets

- Let T be the spanning tree of the connected graph G.
- The removal of any branch from a spanning tree, breaks the spanning tree into two trees i.e there is division of the vertices in the graph into two subsets corresponding to the vertices in the two trees.
- It follows that for every branch in a spanning tree there is a corresponding cut set called fundamental cut set.
- The fundamental cut set of the graph G with respect to the spanning tree T is called **fundamental system of cut set.**
- The number of fundamental cut sets is equal to the number of branches in the spanning tree.
- If the connected graph G has v number of vertices and e number of edges, then spanning tree has (v-1) branches which is the same as the number of fundamental circuit.

❖ **Example 24:** Find the fundamental circuit for the graph show in figure below w.r.t the spanning tree T.

Solution:

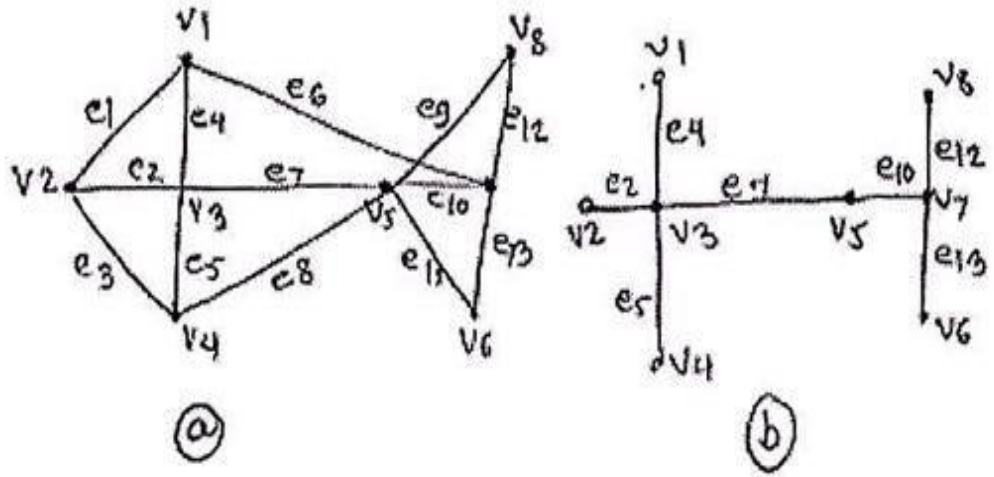




Branch	Fundamental cutset
e4	{ e1, e5, e4 }
e3	{ e2, e7, e3 }
e6	{ e1, e5, e6, e7, e2 }
e8	{e1, e8, e2 }

Example 35: Find the fundamental system of cutset for the graph show in figure below w.r.t the spanning tree T.

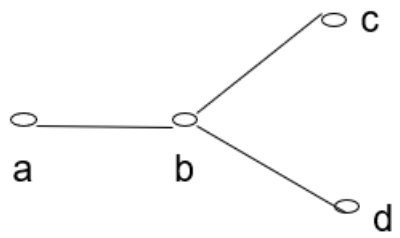
Solution: The spanning tree T has 7 branches {e2,e4,e5,e7,e10,e12,e13}. Therefore its has 7 fundamental cutset.



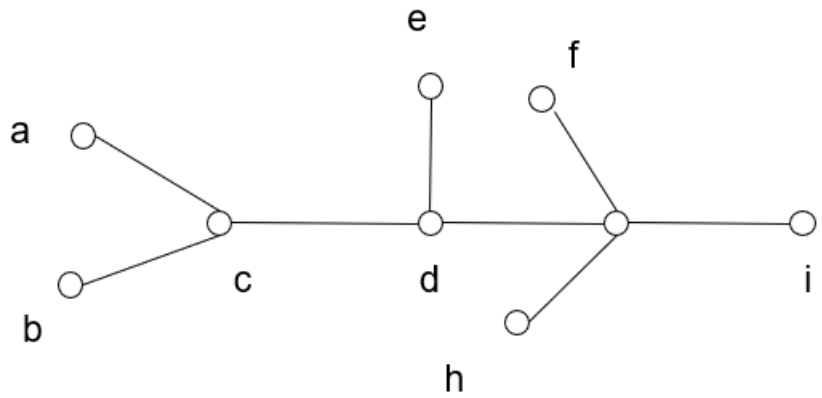
Branch	Fundamental cutset
e2	{e1,e2,e3}
e4	{e1,e4,e6}
e5	{e3,e5,e8}
e7	{e6,e7,e8}
e10	{e6,e9,e10,e11}
e12	{e9,e12}
e13	{e11,e13}

Eccentricity of Vertex & Cut Points

$E(v)=\max d(v_i,v)$



Vertex with minimum Eccentricity is called center of G.



In any tree, all vertices except pendant vertices are cut vertices

Tree Application

They are suitable for:

–Hierarchical structure representation, e.g.,

- File directory.
- Organizational structure of an institution.
- Class inheritance tree.

–Problem representation, e.g.,

- Expression tree.
- Decision tree.

–Efficient algorithmic solutions, e.g.,

- Search trees.
- Efficient priority queues via heaps

Tree Application